

BIG DATA

Machine learning Case Study

Spark MLLib

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Significant information in the slide deck presented through the Unit 1 of the course have been created by **Dr. K V Subramaniam's** and would like to acknowledge and thank him for the same. There have been some information which I might have leveraged from **Dr. H L Phalachandra's** slide contents too. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

Lecture Overview

- ML example problem
- ML Workflow and Challenges
- Solving the ML workflow challenges
- ML Pipelines
- Deep Learning with Big Data

Motivating example

Motivational Problem: Text Classification

Goal : Given a text Document , Predict its topic.

Dataset: "20 Newsgroups"
From UCI KDD Archive

Features

```
Subject: Re: Lexan Polish?
Suggest McQuires #1 plastic
polish.  It will help somewhat
but nothing will remove deep
scratches without making it
worse than it already is.
McQuires will do something...
```

text, image, vector, ...

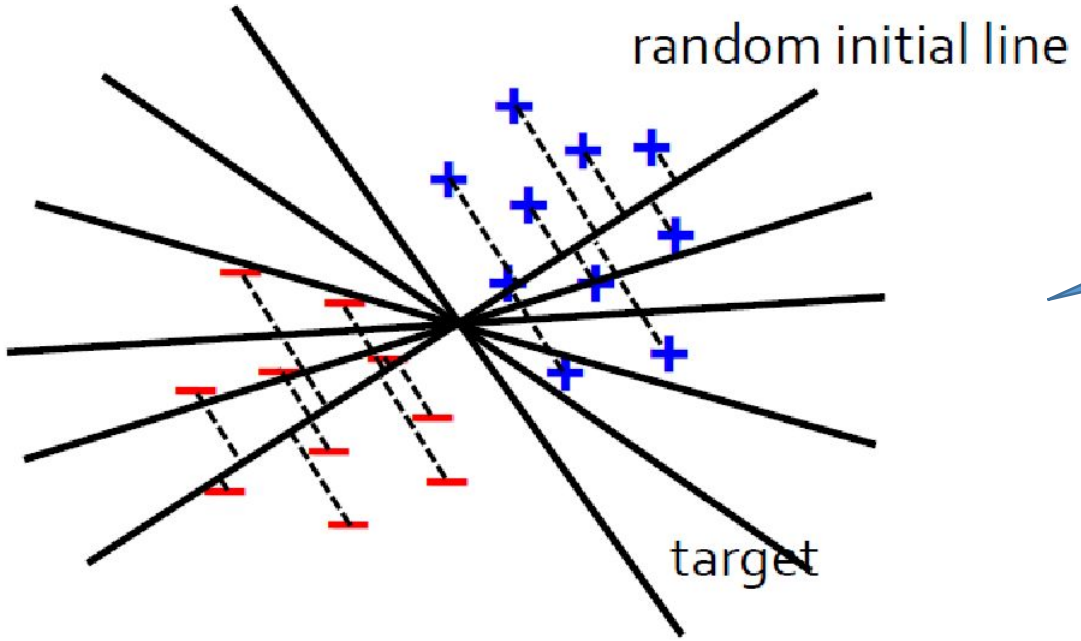
Label

1: about science
0: not about science

CTR, inches of rainfall, ...

Logistic Regression

Goal : Find best line separating two sets of points.



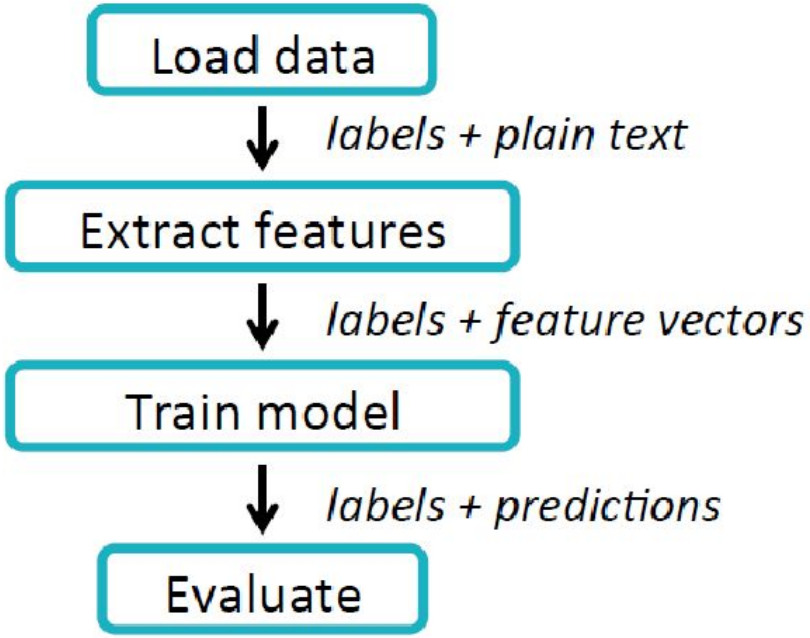
Classify into science and non-science. Each point represents a document.

Further details on logistic regression can be found at - https://en.wikipedia.org/wiki/Logistic_regression

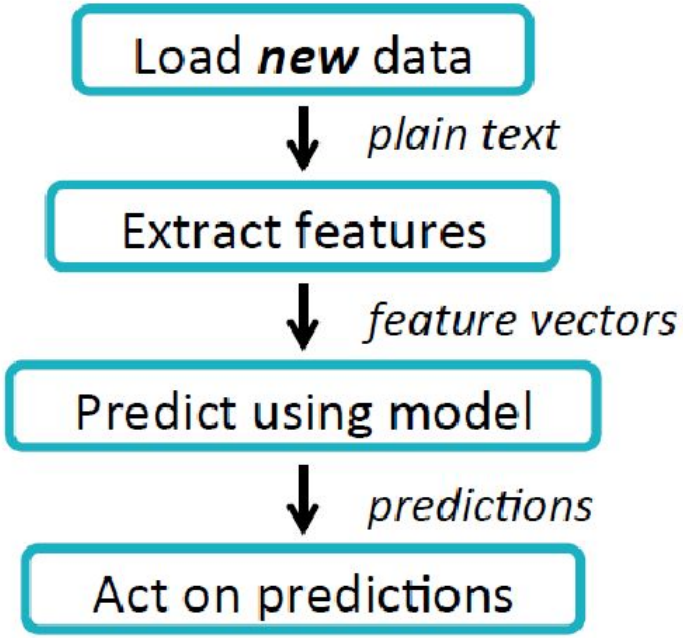
Machine Learning Workflow and Challenges

Example ML Workflow

TRAINING



TESTING/PRODUCTION



Almost identical workflow

What are the pain points?

- Create and Handle Many RDDs and data types
  - Labels, features, predictions..
- Write as a script
  - Whole pipeline needs to be coded as a script
  - Not modular
- Tune parameters
  - Key part of ML
  - Training many models
    - For different splits of data
    - Different sets of parameters

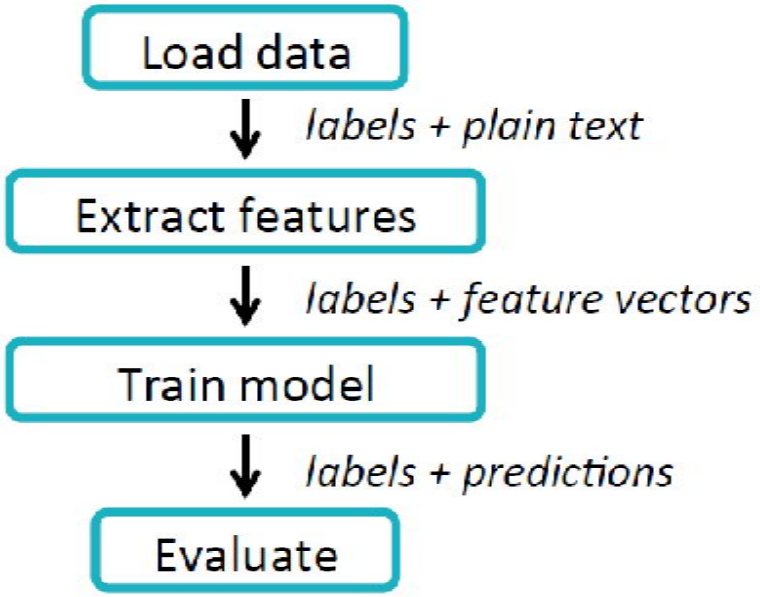
PES UNIVERSITY
CELEBRATING 50 YEARS

Solving the Machine learning challenges

How can the task be made easier?

- Make RDDs easier to read
  - Have to explicitly break up the fields in RDD
  - E.g., break line into blank separated tokens
- As developers we would like to just
  - Program to extract features
  - Specify the model to be used
- However, ML needs additional work
  - Write a script to do all the steps
  - Train the model
  - Evaluate the error of the model by testing

TRAINING



Key concepts

- Reading RDDs: DataFrame

Solves the RDD creation pain-point

- ML Pipeline
  - Transformers
  - Estimators
  - Evaluators

Solves the Scripting..

- Parameters
  - API
  - Tuning

Solves the parameter tuning pain point

BIG DATA

Dataframes

- Recall
- Announced Feb 2015
- Inspired by data frames in R and Pandas in Python
- Works in:



http://training.databricks.com/intro.pdf

What is a Dataframe?

- a distributed collection of data organized into named columns
- Like a table in a relational database

Dataframes

PES UNIVERSITY
CELEBRATING 50 YEARS

Features

- Scales from KBs to PBs
- Supports wide array of data formats and storage systems (Hive, existing RDDs, etc)
- State-of-the-art optimization and code generation via Spark SQL Catalyst optimizer
- APIs in Python, Java

Dataframe

Dataframe : RDD + Schema + DSL

Named columns with types

```
label: Double
text: String
words: Seq[String]
features: Vector
prediction: Double
```

label	text	words	features
0	This is ...	["This", "is", ...]	[0.5, 1.2, ...]
0	When we ...	["When", ...]	[1.9, -0.8, ...]
1	Knuth was ...	["Knuth", ...]	[0.0, 8.7, ...]
0	Or you ...	["Or", "you", ...]	[0.1, -0.6, ...]

Domain-Specific Language

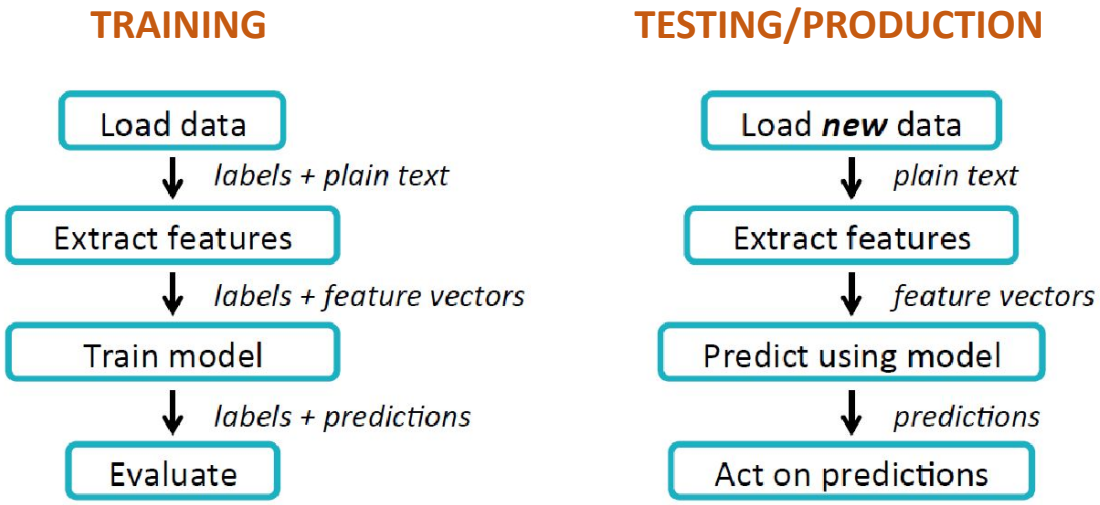
```
# Select science articles
sciDocs =
  data.filter("label" == 1)

# Scale labels
data("label") * 0.5
```

ML Pipelines

ML Pipelines

- Introduced in Spark 1.2 and 1.3
- Allows developers to just
  - Program to extract features
  - Specify the model to be used
- Automates the process of
  - Write a script to do all the steps
  - Train the model
  - Evaluate the error of the model by testing
  - Or deploy in production



The ML Pipeline

Transformers

- Extract features from DataFrame
- Features are stored in a new DataFrame

Estimators

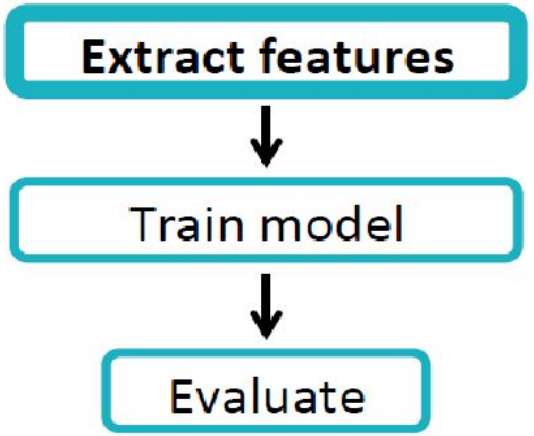
- ML Algorithms
- MLLib has standard defined ML algorithms (e.g., Logistic Regression)
- User can add his own

Evaluators

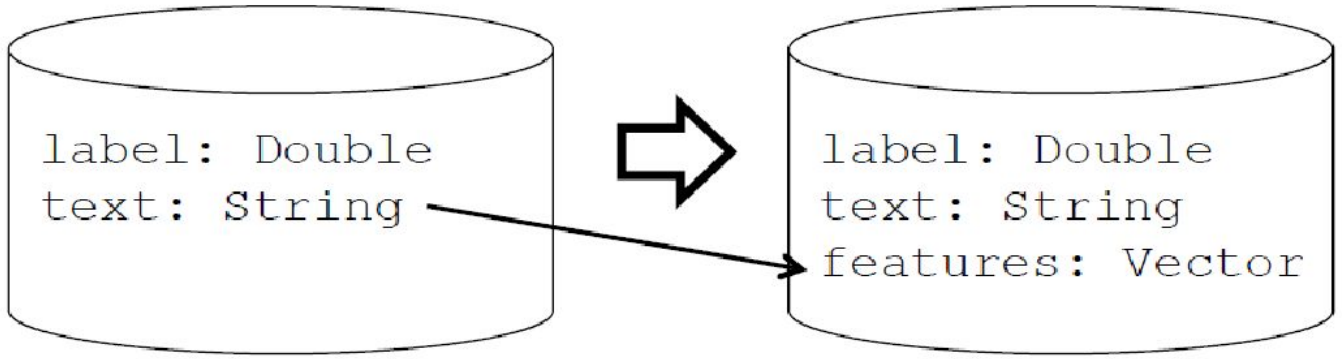
- Compute predictions and estimate metrics such as error
- Tune algorithm parameters
- Evaluator depends upon estimator
  - Evaluator that trains Logistic Regression cannot be used for Decision Trees

Transformers

TRAINING




```
def transform(DataFrame): DataFrame
```

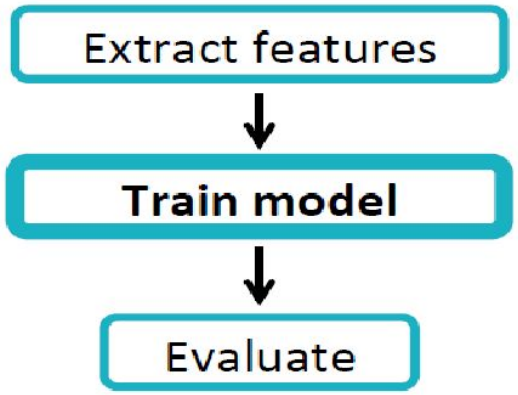


Label	Text
0	
1	

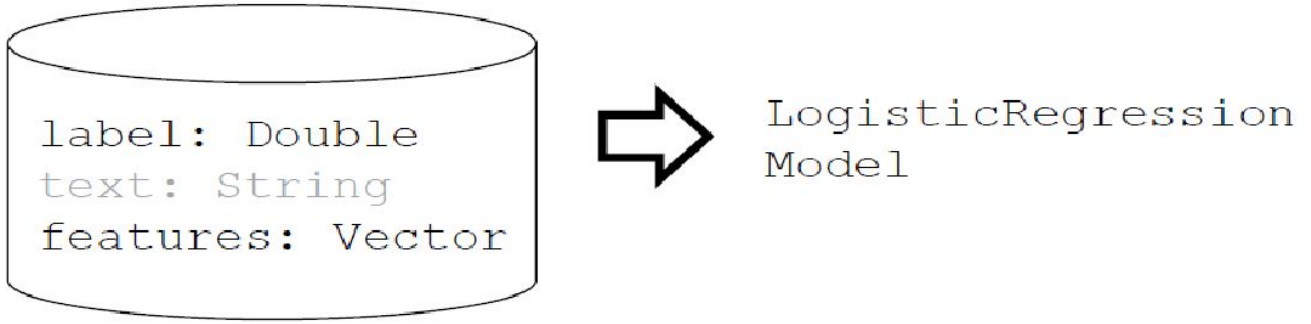
Label	Text	Features
0		
1		

Estimator

TRAINING



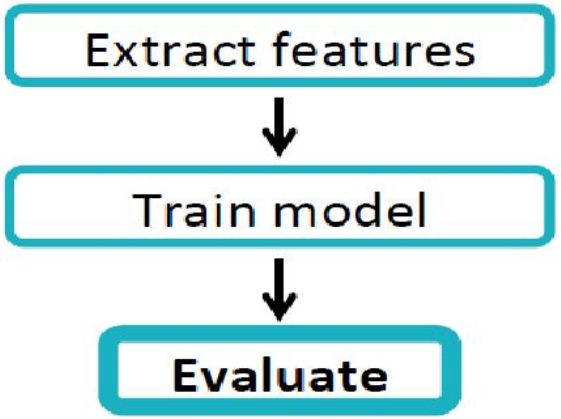

```
def fit(DataFrame): Model
```



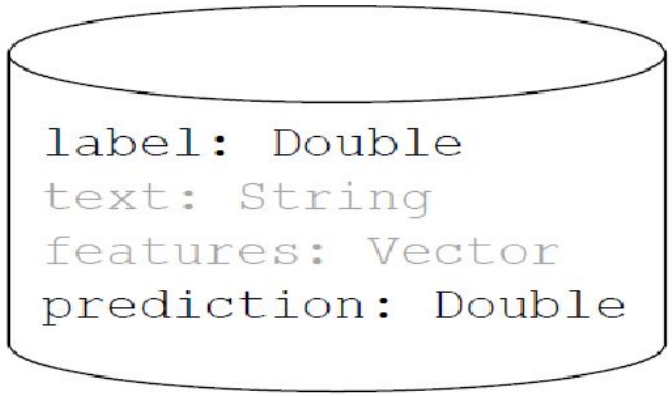
Label	Text	Features
0		
1		

Evaluator

TRAINING




```
def evaluate(DataFrame): Double
```

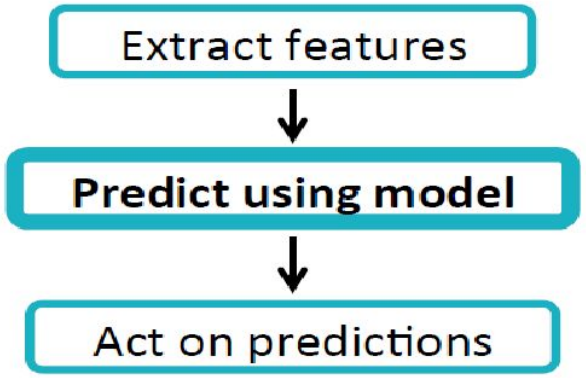


Metric:
- accuracy
- AUC
- MSE
- ...

Label	Text	Features	Prediction
0			
1			

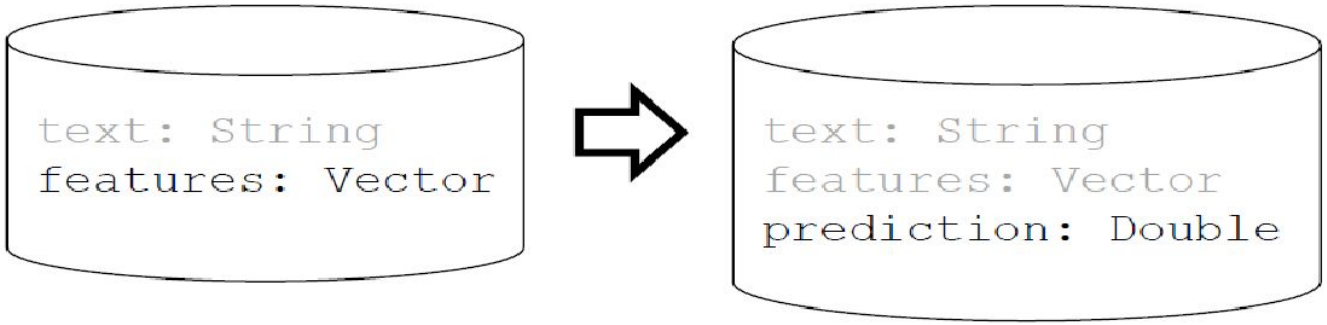
Model

TESTING/PRODUCTION



Model is a type of Transformer

```
def transform(DataFrame): DataFrame
```

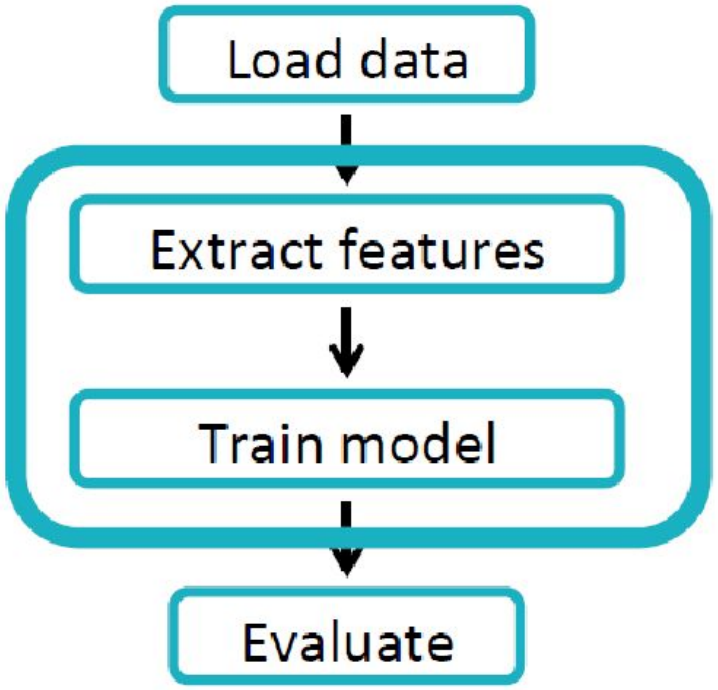


Text	Features

Text	Features	Prediction

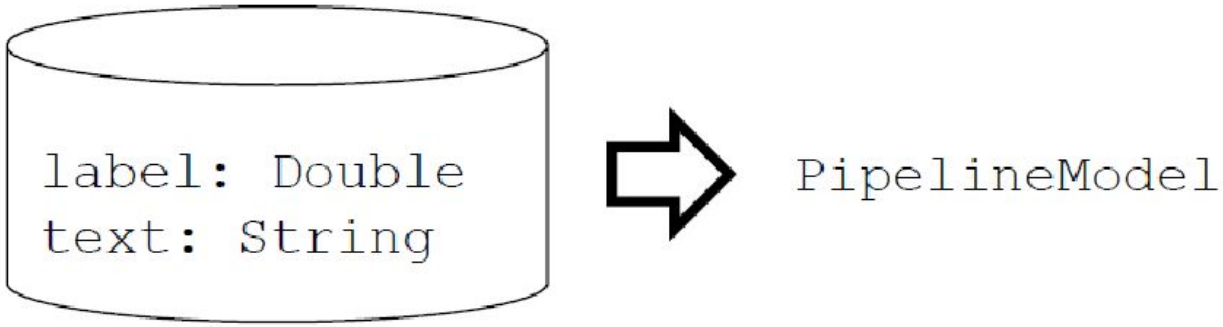
The training Pipeline

TRAINING



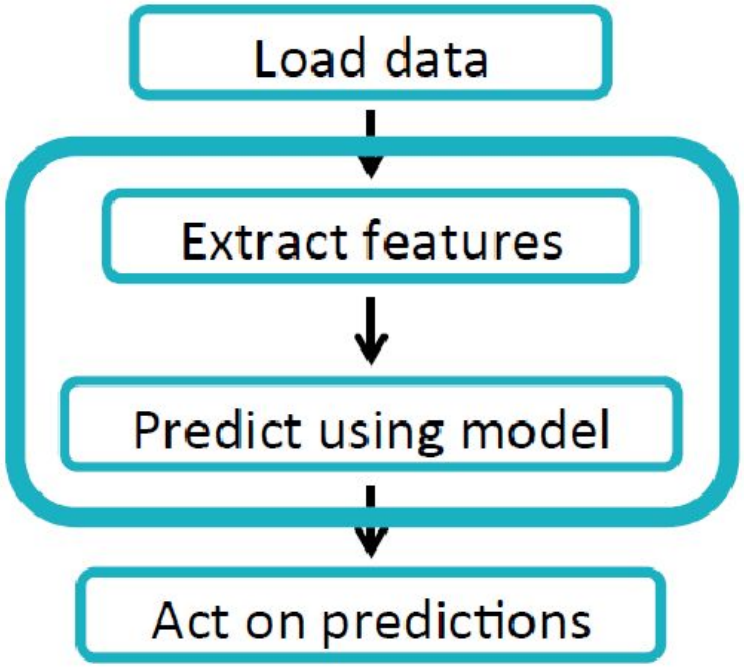
Pipeline is a type of Estimator

```
def fit(DataFrame): Model
```



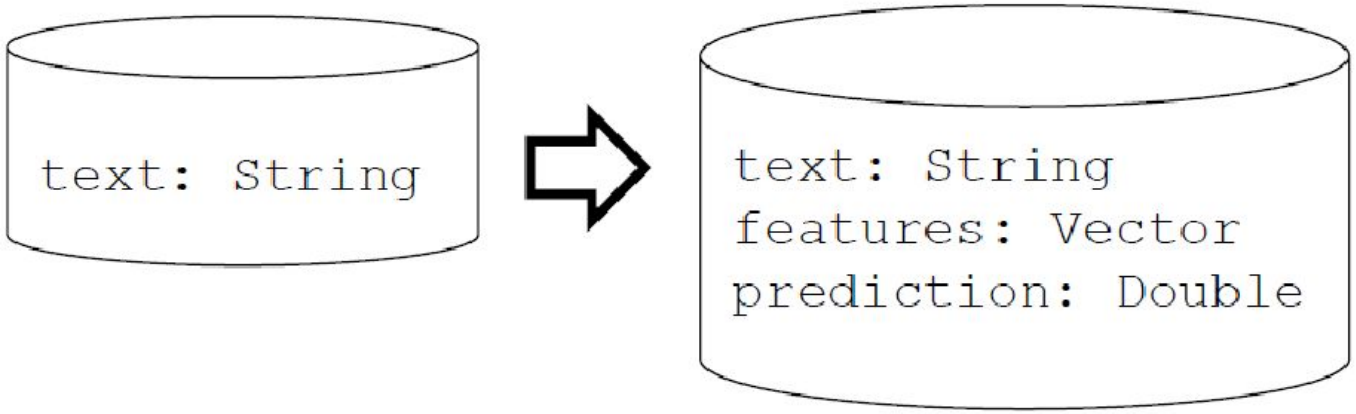
The testing pipeline/model

TESTING/PRODUCTION

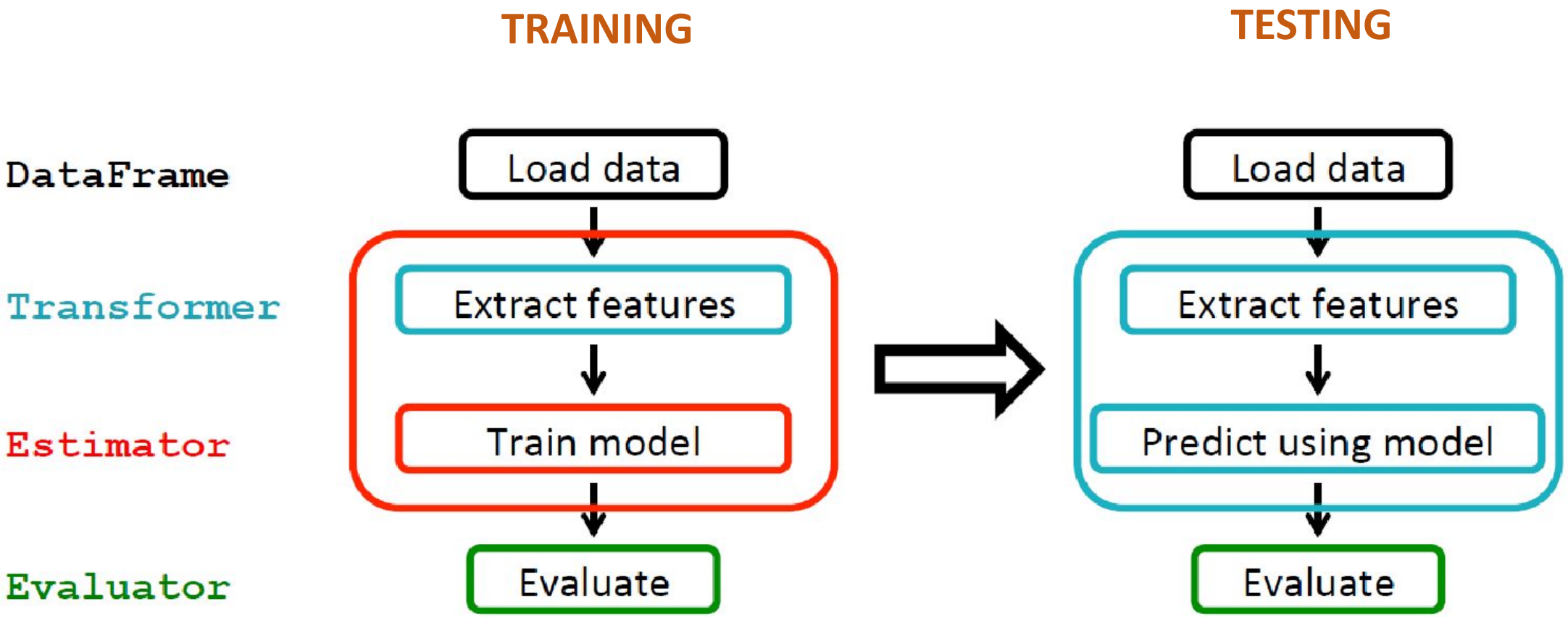


PipelineModel is a type of Transformer

```
def transform(DataFrame): DataFrame
```



Putting it all together



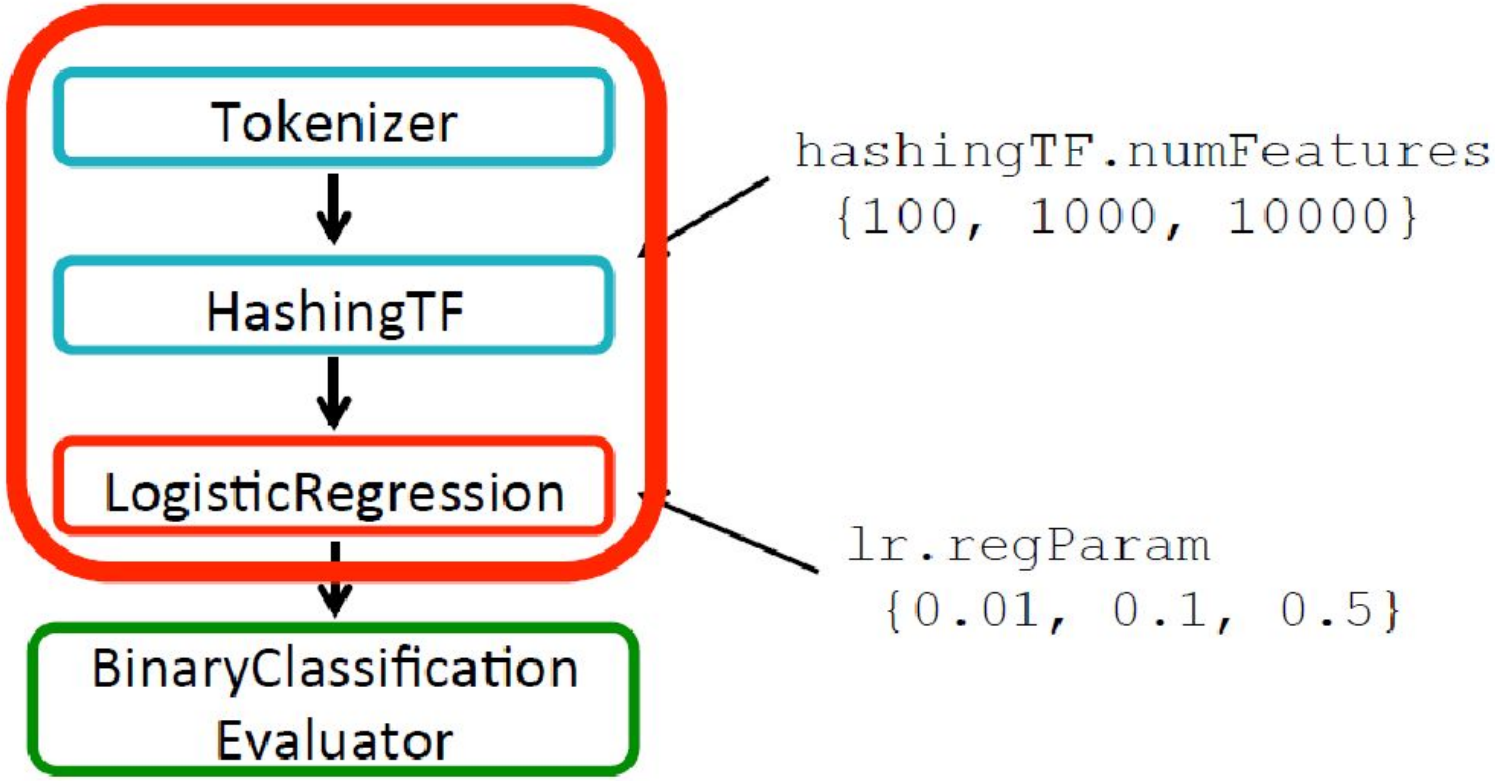
Parameter Tuning

Given:

- Estimator
- Parameter Grid
- Evaluator

Find the Best parameters

CrossValidator



Exercise 4: 10 minutes

- Suppose we have a dataset in which each line has a recording of a noise, and its classification
- E.g., <bell.wav>, bell



- What would be the input DataFrame be?
- Suppose we want to recognize sounds by
  - Extracting the frequencies from the wav file
  - Gaussian model
    - Find the average frequency of each sound
    - For a new sound, calculate average frequency
    - Find closest matching sound
- What are the DataFrames, Evaluators, etc needed.

Exercise 4: Solution

- Suppose we have a dataset in which each line has a recording of a noise, and its classification
- E.g., <bell.wav>, bell

- Input DataFrame
  - <bell.wav>, bell
- Feature DataFrame
  - <bell.wav>, bell, frequencies
- Transformer (use same transformer for train/predict)
  - <bell.wav> Bell, average frequency
- Model
  - train(FeatureDataFrame)
    - Associate average frequency for “Bell”
  - predict(PredictDataFrame)
    - Output closest matching sound

Mllib algorithms

- Classification
  - Logistic Regression
  - Decision Tree
  - Random Forest
  - Gradient boosted tree
  - Multilayer Perceptron
  - SVM
  - Naïve Bayes
- Clustering
  - K-Means
  - LDA
  - GMM
- Collaborative Filtering
  - ALS
- Frequent Pattern Mining

PES UNIVERSITY
CELEBRATING 50 YEARS

Deep Learning with Big Data

Challenges for Deep Learning

- Heterogenous cluster
- Deep Learning (Tensorflow)
  - Iterative
  - Matrix vector multiplication – Linear algebra
- Initially evolved on a single machine – only scale up
- Then had its own cluster
  - Typically heterogenous with CPUs, GPUs, TPUs

Challenges for Deep Learning

- But data resides on HDFS and big data platform uses Spark
- How should the two work together.
- Typically the two clusters are different

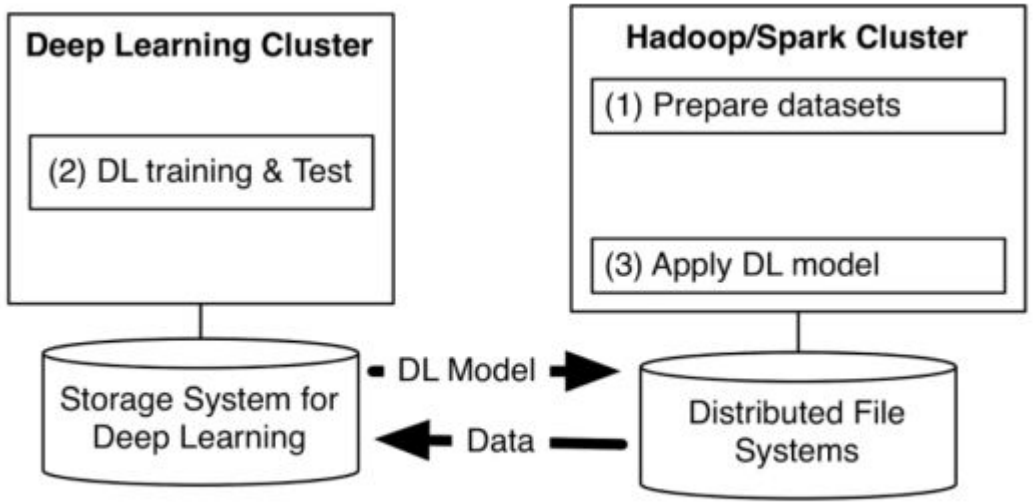


Figure 1: ML Pipeline with multiple programs on separated clusters

https://developer.yahoo.com/blogs/157196317141/

Challenges for Deep Learning

- Can we use the same cluster?

- Tensorflow on Spark
  - From Yahoo

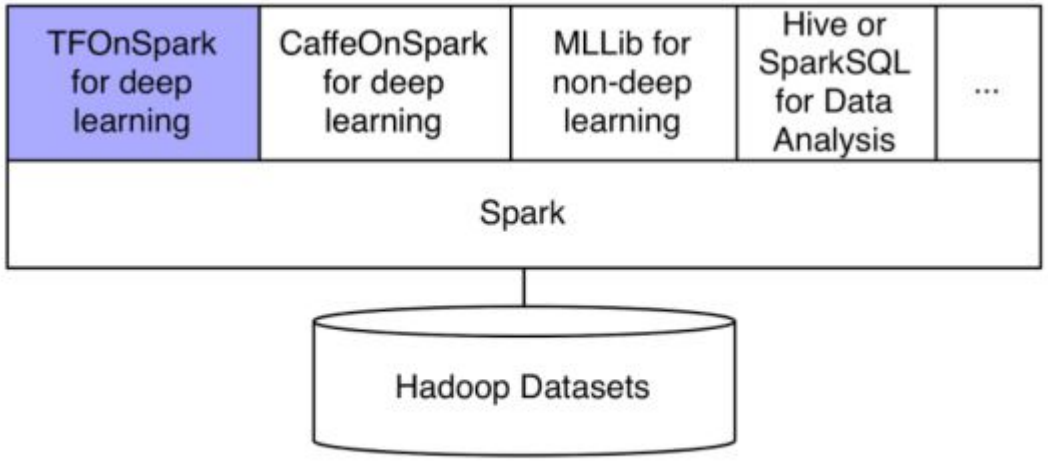


Figure 2: TensorFlowOnSpark for deep learning on Spark clusters

https://developer.yahoo.com/blogs/157196317141/

BIG DATA

Tensorflow on Spark Architecture

- Supports both
  - Model parallelism
  - Data parallelism
- <10 lines of code change reqd
- Algorithm and parameter server run on Spark executors
  - Can read data directly from HDFS
  - Spark RDD data is fed to spark executor which passes it to Tensorflow
- RDMA faster network transfers

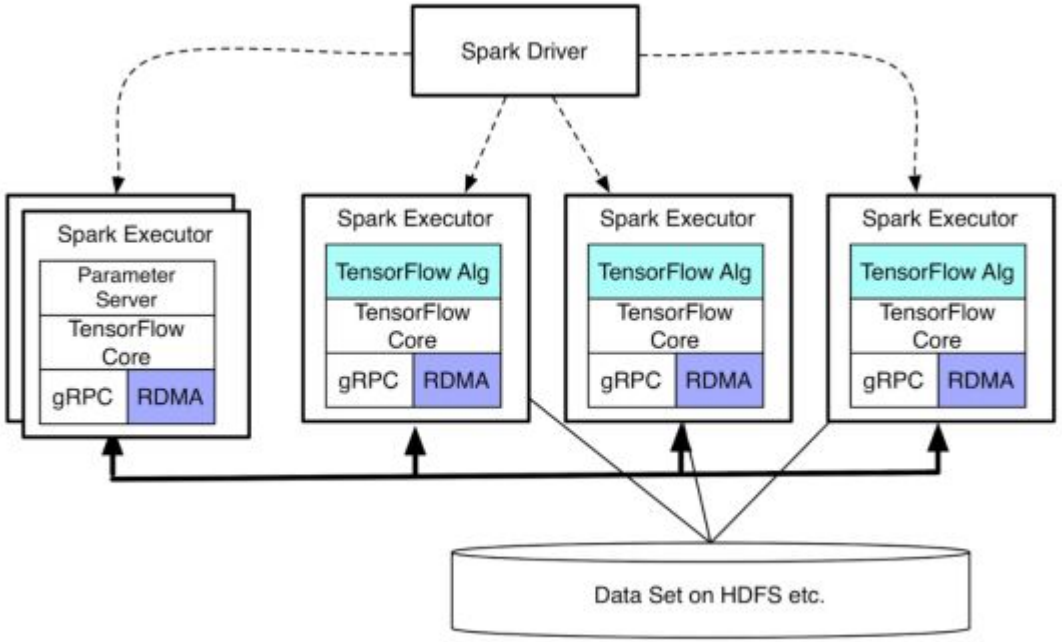


Figure 3: TensorFlowOnSpark system architecture

https://developer.yahoo.com/blogs/157196317141/

Other solutions

- SystemML – systemml.apache.org
  - IBM
  - SystemML: Declarative Machine Learning on Spark http://www.vldb.org/pvldb/vol9/p1425-boehm.pdf
  - Uses a declarative ML language
  - Translated to MR/Spark
- Intel BigDL
  - Modeled on Torch

https://www.intel.com/content/www/us/en/developer/tools/bigdl/overview.html

https://www.intel.com/content/dam/develop/external/us/en/documents/bigdl-distributed-dl1.pdf



THANK YOU

Prafullata Kiran Auradkar
Department of CSE
prafullatak@pes.edu



BIG DATA

Streaming Algorithms – Sampling

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Significant information in the slide deck presented through the Unit 1 of the course have been created by **Dr. K V Subramaniam's** and would like to acknowledge and thank him for the same. There have been some information which I might have leveraged from **Dr. H L Phalachandra's** slide contents too. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

BIG DATA

Overview: Streaming Algorithms

- Why study streaming algorithms?
- Sampling Algorithms

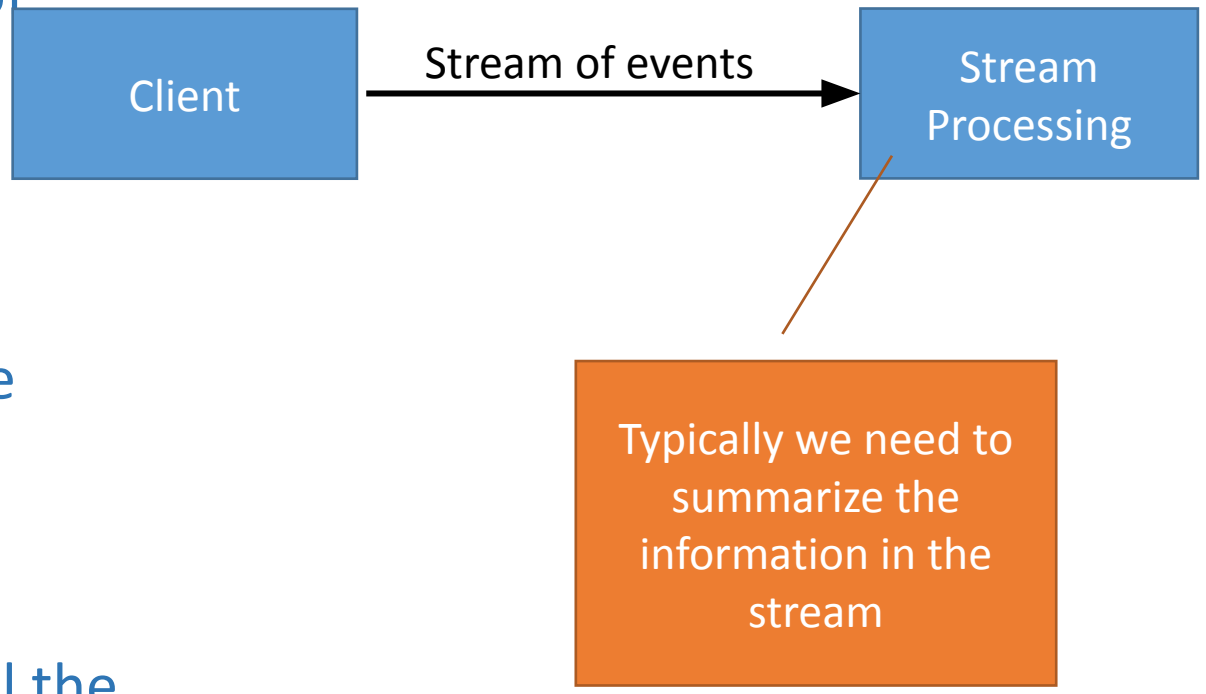
Streaming Algorithms overview

BIG DATA

The need for processing events

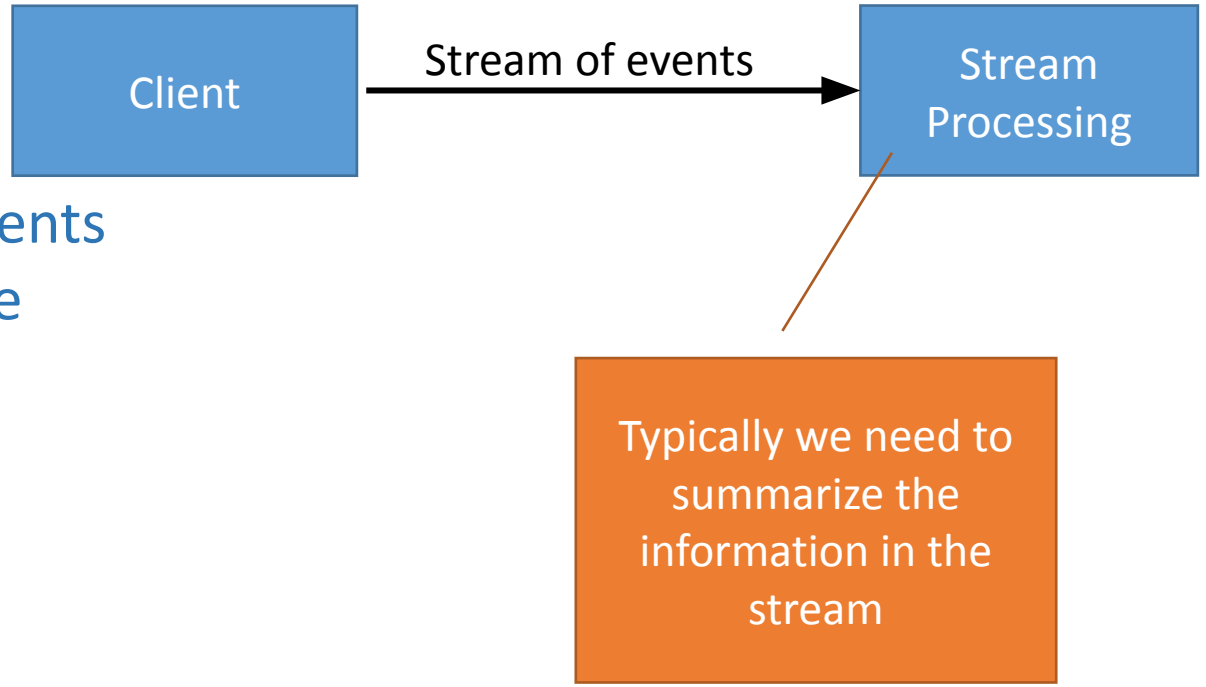
- Stream processing requires processing of events in a never ending stream
- We need to compute summaries of the stream
- For example
  - How many unique elements have we seen in the stream?

- In a relational table this is just COUNT(DISTINCT key), but that needs all the data stored. A never-ending stream cannot be stored in full.



The need for processing events

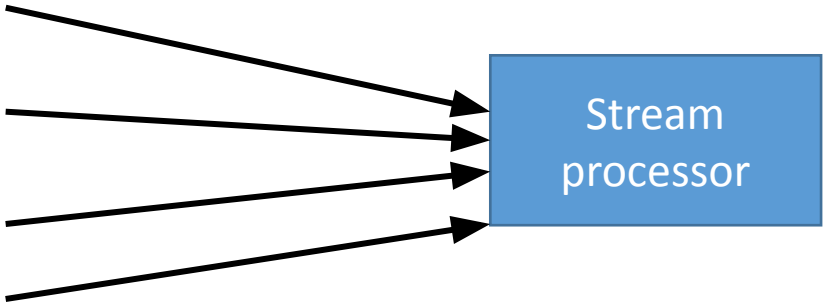
- One approach
  - Breakup stream into a window of events
  - Window = the last n elements, or the last t seconds
  - Allows us to perform relational operations
  - Similar to Apache Spark model



Issues in stream processing

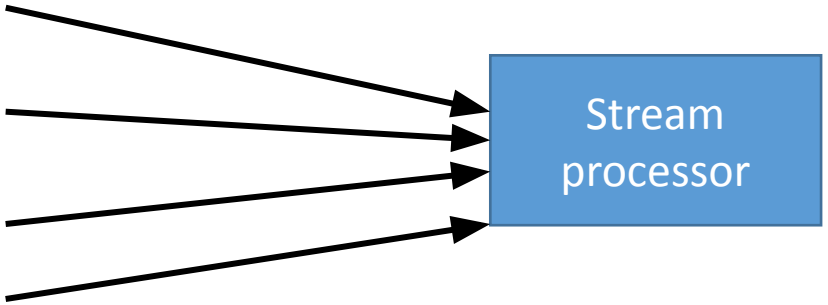
- Velocity of the stream
  - The rate at which data is being sent
  - Some applications need a decision for each element as it arrives
  - Different streams may have different rates
  - For ex: mission critical systems

- #streams
  - Many streams, each needing a little memory, can together exhaust the memory



Stream processing algorithms

- Must process data in memory
  - Not off disk (too slow)
- More efficient to get *approximate* solution rather than an exact one.
- Hashing gives randomness that is repeatable: the same key always gets the same hash.



Sampling Algorithms

Flaw in obvious algorithm



Flaw in obvious algorithm

- Suppose user has issued a particular search query s twice
- There's a probability 1/100 (not 1/10) that both queries will show up in our sample
- There's only a probability 1/100 that we will know query s was repeated

So our sampling will be wrong

Query number *m* is *s*

p(m sampled) = 1/10

Query number *n* is also *s*

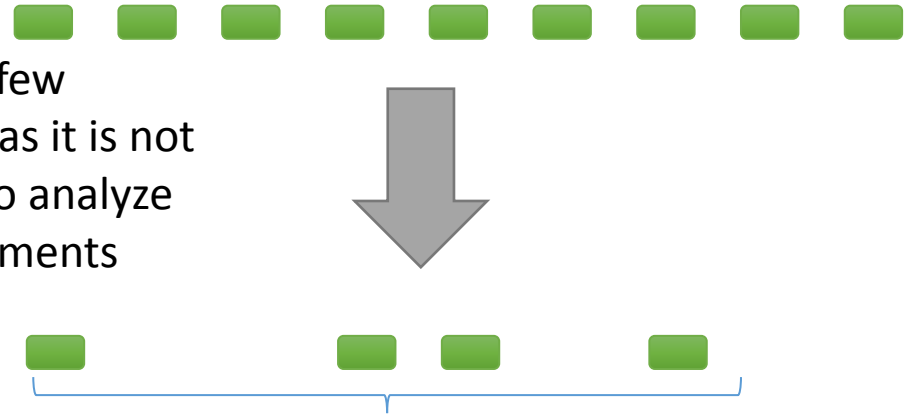
p(n sampled) = 1/10

p(m and n sampled) = 1/100

BIG DATA

Extracting reliable samples

- Given a long stream of data
- How to create a representative sample?



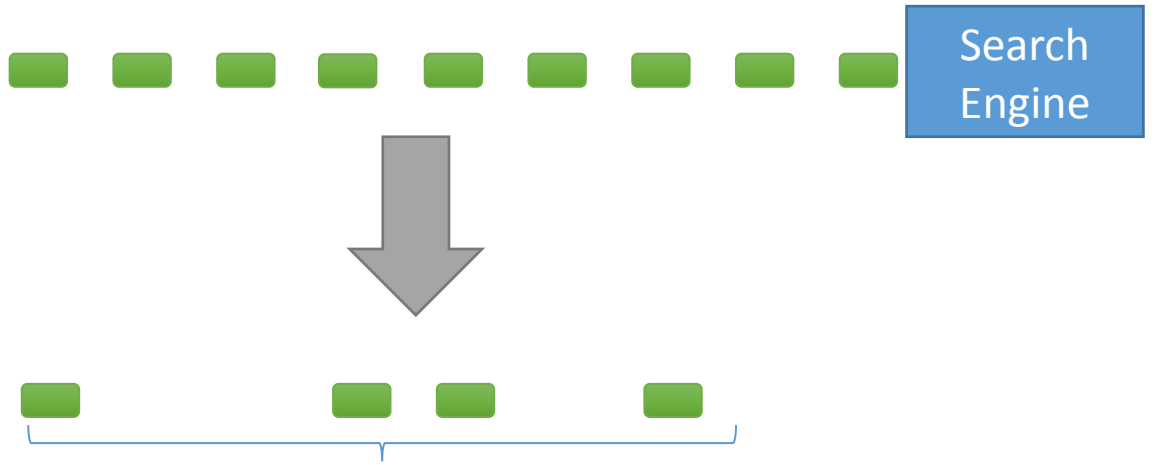
Sample a few elements as it is not possible to analyze all the elements

Need to ensure that analyzing the sample is representative of analyzing the entire stream

Motivating Example

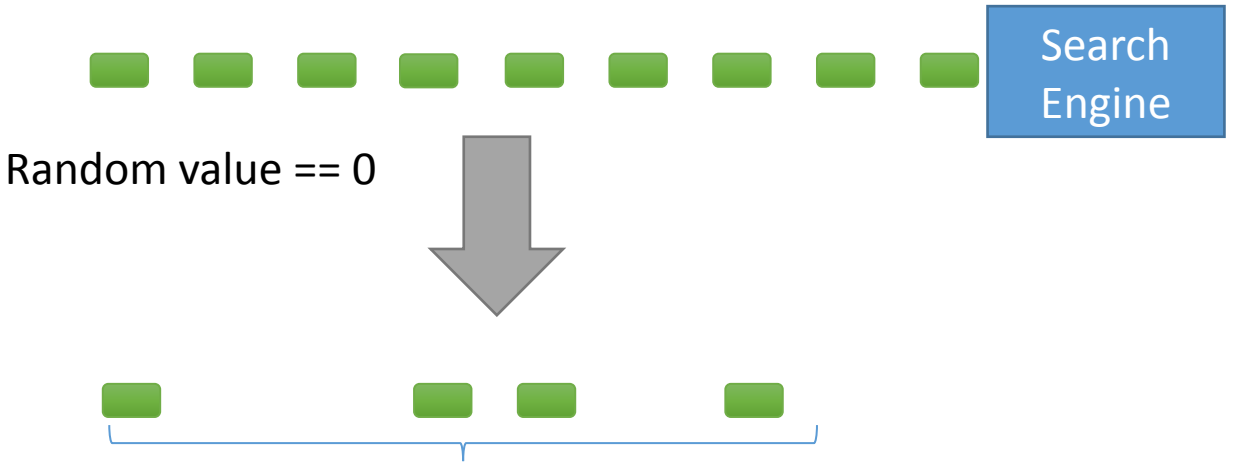
- Each element of the stream is a search query, a tuple (user, query, time)
- How to create a representative sample?

- We want to answer the question?
  - "What fraction of the typical user's queries were repeated over the past month?"
- What's obvious algorithm?



Obvious Algorithm

- For every stream tuple seen..
- Generate a random integer between 0..9
- If value is 0
  - Then store(use) the tuple
- Otherwise – discard.
- Each query is kept with probability 1/10, independently of all other queries

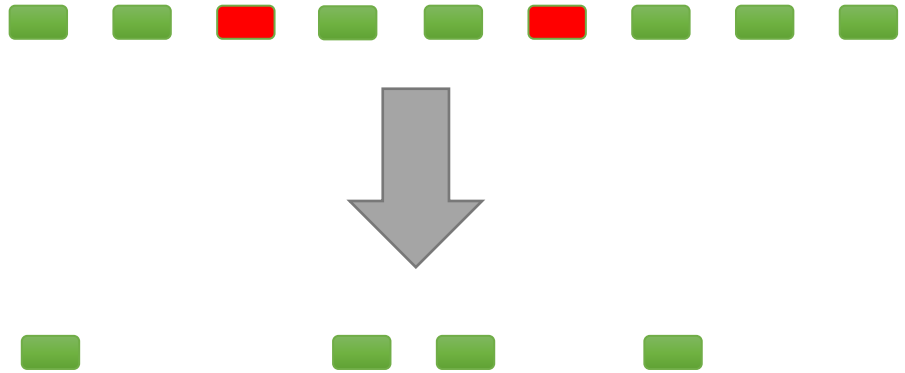


Flaw in obvious algorithm

Flaw in obvious algorithm

- Suppose user has issued a particular search query s twice
- There's a probability 1/100 (not 1/10) that both queries will show up in our sample
- There's only a probability 1/100 that we will know query s was repeated

So our sampling will be wrong



Sampling Algorithms - refinement

Refined algorithm

- Sample 1/10th of the **users**, and keep **all** queries of a sampled user
- Details
  - When a query arrives
  - Look up user to see if they are in sample.
  - If so, add query to sample
  - If first time we have seen the user, generate a random number 0..9
    - Add user to sample if number is 0

Drawback: we must store every user seen so far with an in/out decision.

Optimization to algorithm

- Checking if we have seen the user
- Requires a search through a data structure
- Not really required
- Just hash(username) □ 0..9
  - If hash is 0, select the userid

Generalization of the algorithm

- We used “user” here to select
- How to extend this algorithm generically?
- Identify the *key components* of the query.
  - In previous example tuple is *<user, query, time>*
  - Here the key is only the user. Time and query are not hashed.
- Hash key components in the range *(0..b)*
- To get sample fraction(a/b)
  - Select query if *hash(key components) < a*

Exercise

- Suppose we want a sample dataset to debug a program that profiles transactions by user and country
- How would I generate a 1/20 sample?

BIG DATA

Exercise - Solution

- There isn’t a unique solution
- Suppose I want a sample dataset to debug a program that profiles transactions by user and country
- How would I generate a 1/20 sample? Hash userid and country in the range 0..19; select if the hash is 0
- The above method will give me 1/20 of all user-country pairs

Varying Sample size

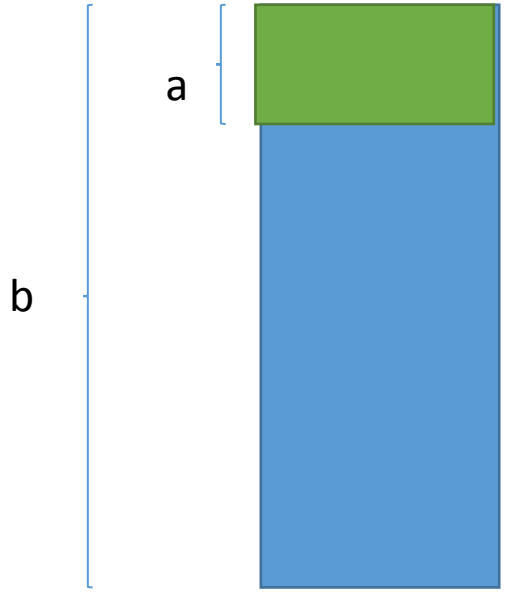
BIG DATA

Varying the sample size

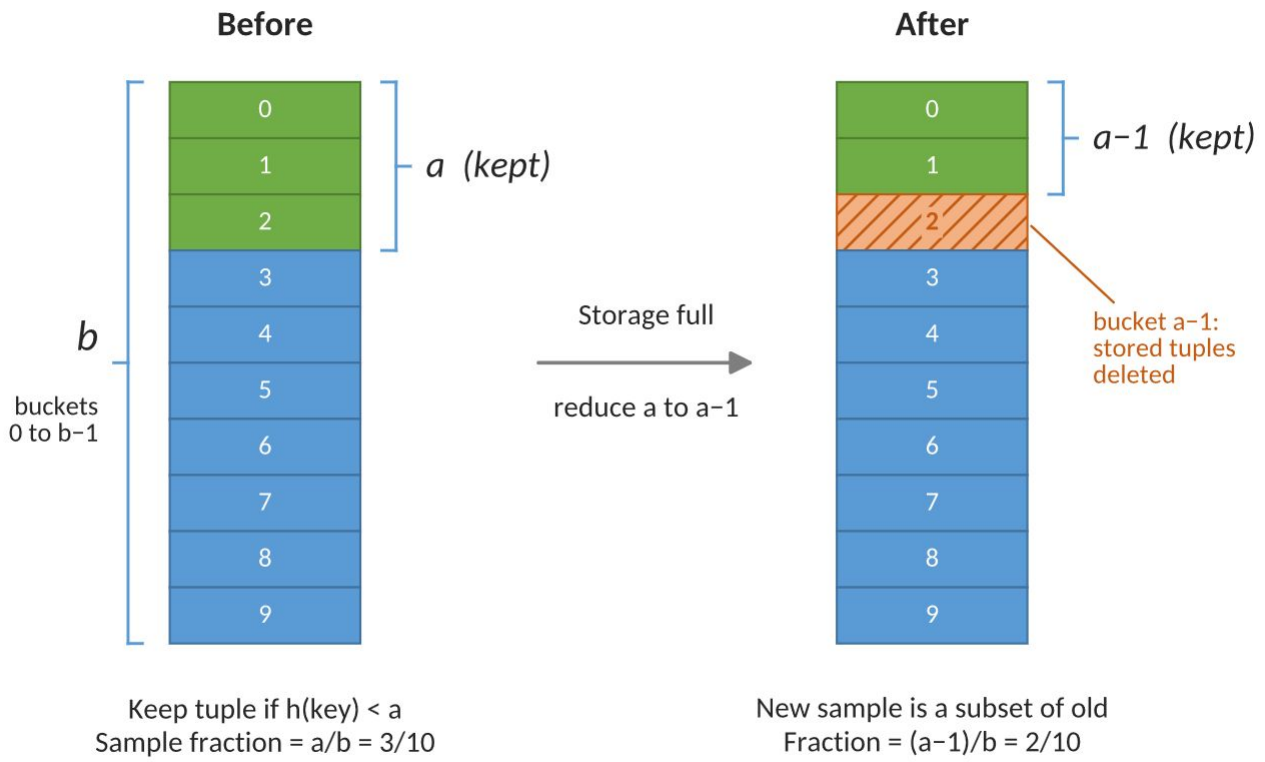
- We are selecting *a/b* samples from data arriving from outside.

- As more data is added to the system, the number of keys in the system also increases
  - Additional storage space required

- If there is a limit on #keys that can be stored, then how to handle it, so that the sample stays representative
- Reduce *a* to *a-1:* Lower the threshold from a to a−1 and delete the stored tuples whose key hashes to a−1. The new sample is a subset of the old one and is still a uniform a−1/b sample of keys



Varying the sample size



References

- Spark streaming -https://www.youtube.com/watch?v=g171ndOHgJ0
- https://spark.apache.org/streaming/
- https://yurongfan.wordpress.com/2017/01/10/introduction-of-a-big-data-machine-learning-tool-sparkml/
- https://towardsdatascience.com/learn-how-to-use-spark-ml-and-spark-streaming-3a731485d052
- **Refer chapter 4.1 and 4.2 from Mining of Massive datasets book**



THANK YOU

Prafullata Kiran Auradkar
Department of CSE
prafullatak@pes.edu



BIG DATA

Streaming Spark

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Significant information in the slide deck presented through the Unit 1 of the course have been created by **Dr. K V Subramaniam's** and would like to acknowledge and thank him for the same. There have been some information which I might have leveraged from **Dr. H L Phalachandra's** slide contents too. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

BIG DATA

Overview of lecture

- Introduction to streaming analysis
- Streaming query types
- Streaming analysis requirements
- Streaming spark – Dstreams
- Job execution
- Stateless Transformations
- Stateful processing
- Fault Tolerance
- Performance
- Putting it all together

Introduction to streaming analysis

BIG DATA

Examples of Streaming

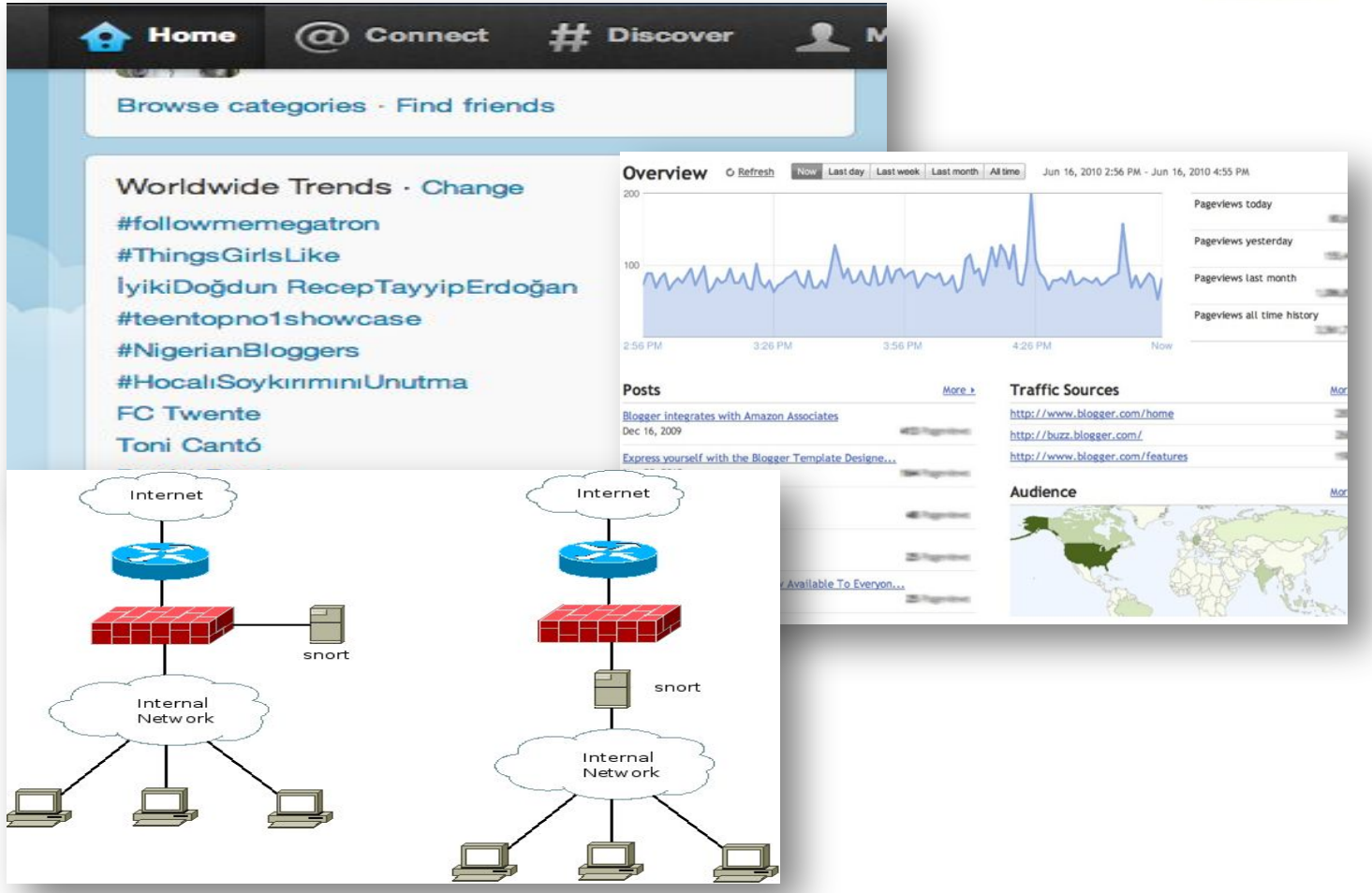
- Sensor data, e.g.,
  - Temperature sensor in the ocean
    - Reports GPS
    - 1 sensor every 150 square miles => 1,000,000 sensors
    - 10 readings/sec => 3.5 TB/day
- Images
  - London: 6 million video cameras
- Internet / Web traffic
  - Google: Billions of queries/day
  - Yahoo: Billions of clicks/day

BIG DATA

Motivation

- Many important applications must process large streams of live data and provide results in near-real-time
  - Social network trends
  - Website statistics
  - Intrusion detection systems
  - Transportation system - Uber
  - etc.
- Require large clusters to handle workloads

- Require latencies of few seconds



Streaming data model and queries

Stream Data Model

- Multiple streams
- Different rates, not synchronized
- Archival store
  - Offline analysis, not real-time
- Working store
  - Disk or memory
  - Summaries
  - Parts of streams
- Queries
  - Standing queries
  - Ad-hoc queries

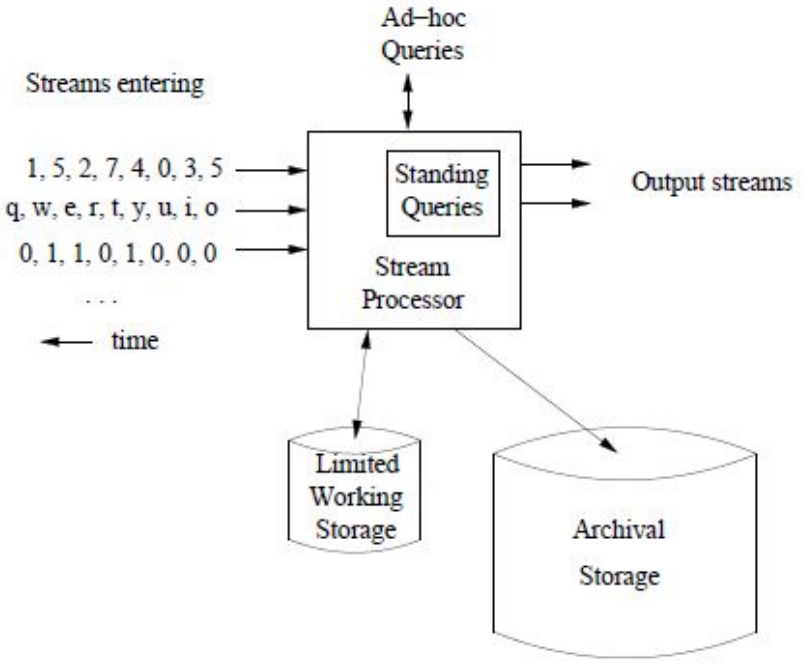


Figure 4.1: A data-stream-management system

Examples of Stream Queries

- **Standing queries**: produce outputs at appropriate time
  - Query is continuously running
  - Constantly reading new data
  - Query execution can be optimized
  - Example: maximum temperature ever recorded
- ***Ad hoc* query**: not predetermined, arbitrary query
  - Need to store stream
  - Approach: store sliding window in SQL DB
  - Do SQL query
  - Example: number of unique users over last 30 days
    - Store logins for last 30 days

BIG DATA

Exercise 1

Consider the queries on the right. Which among them are **STANDING QUERIES** and which are **AD HOC**?

- Alert when temperature > threshold
- Display average of last n temperature readings; n arbitrary
- List of countries from which visits have been received over last year
- Alert if website receives visit from a black-listed country

BIG DATA

Exercise 1 Solution

Consider the queries on the right. Which among them are **STANDING QUERIES** and which are **AD HOC**?

Solution

- Alert when temperature > threshold - standing
- Display average of last *n* temperature readings – *ad hoc*
- List of countries from which visits have been received over last year – *ad hoc*
- Alert if website receives visit from a black-listed country - standing

Note: Average of last n is ad-hoc because n is unknown in advance

Issues in Stream Processing

- Velocity
  - Streams can have high data rate
  - Need to process very fast
- Volume
  - Low data rate, but large number of streams
    - Ocean sensors, pollution sensors
- Need to store in memory
  - May not have huge memory
    - Approximate solutions(Eg: Bloom filter, Flajolet-Martin etc)

Need for a framework …

. . . for building such complex stream processing applications

But what are the requirements

from such a framework?



Streaming analysis framework requirements

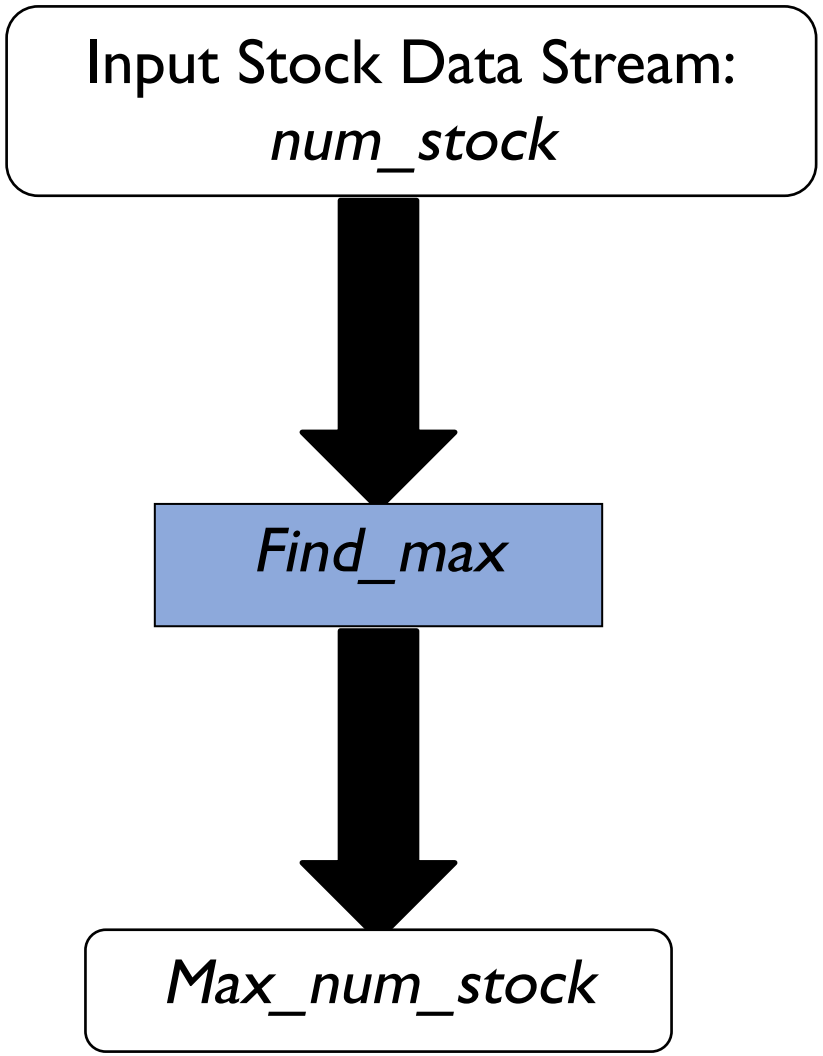
BIG DATA

Requirements

- **Scalable** to large clusters
- **Second**-scale latencies
- **Simple** programming model

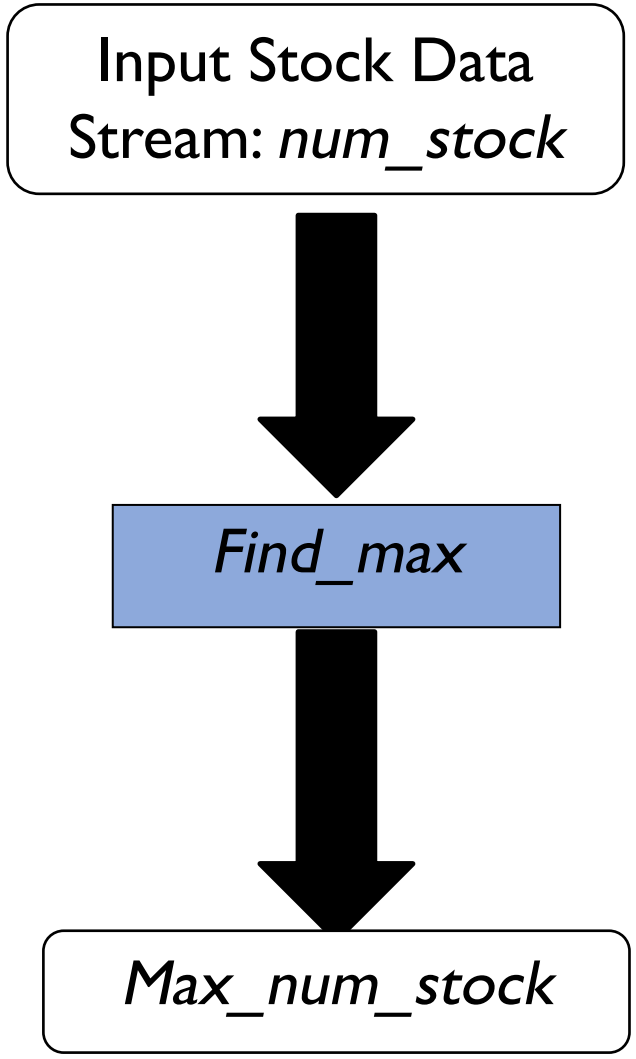
CAN WE USE HADOOP?

- Consider the simple program on the right.
- The input is a stream of records from the stock market.
  - Each time a stock is sold, a new record is created.
  - The record contains a field num_stock which is the number of stocks sold.
- Find_max is a program that updated a variable Max_num_stock which is the maximum of num_stock.



Exercise 2 - Solution CAN WE USE HADOOP?

- Write the pseudo-code for *Find_max*
  - *If num_stock > Max_num_stock*
    Max_num_stock = num_stock
- Can this be implemented in Hadoop?
  - We need to process one record at a time. Hadoop processes a full file in the Map
  - *Max_num_stock* is a global variable
- Does your solution assume that *Find_max* runs on a single node? Is this a reasonable assumption?
  - No, the number of transactions could be more than what a single node can handle



Similar problems can arise in Spark

Case study: Conviva, Inc.

- Real-time monitoring of online video metadata
  HBO, ESPN, ABC, SyFy, …

Since we can't use Hadoop

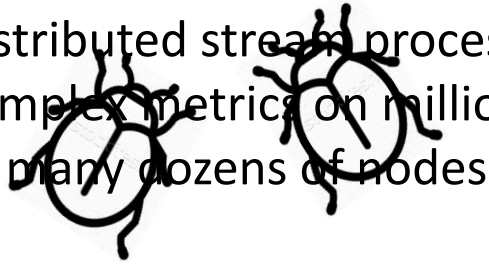
- Two processing stacks
  - Custom-built distributed stream processing system
    - 1000s complex metrics on millions of video sessions
    - Requires many dozens of nodes for processing
  - Hadoop backend for offline analysis
    - Generating daily and monthly reports
    - **Similar computation as the streaming system**

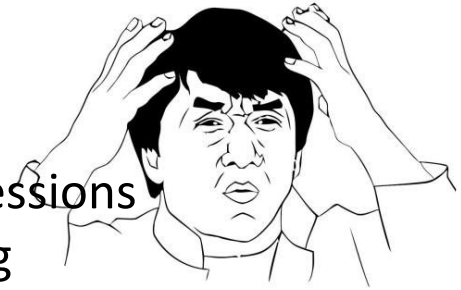
BIG DATA

Case study: XYZ, Inc.

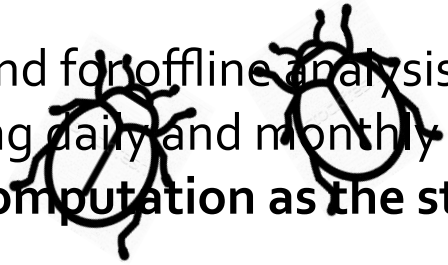
- ***Any company who wants to process live streaming data has this problem***
- **Twice** the effort to implement any new function
- **Twice** the number of bugs to solve
- **Twice** the headache

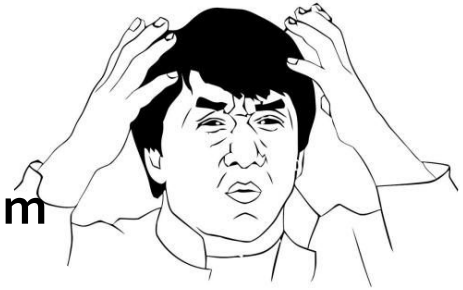
- Two processing stacks
  - Custom-built distributed stream processing system
    - 1000s complex metrics on millions of video sessions
    - Requires many dozens of nodes for processing





  - Hadoop backend for offline analysis
    - Generating daily and monthly reports
    - **Similar computation as the streaming system**





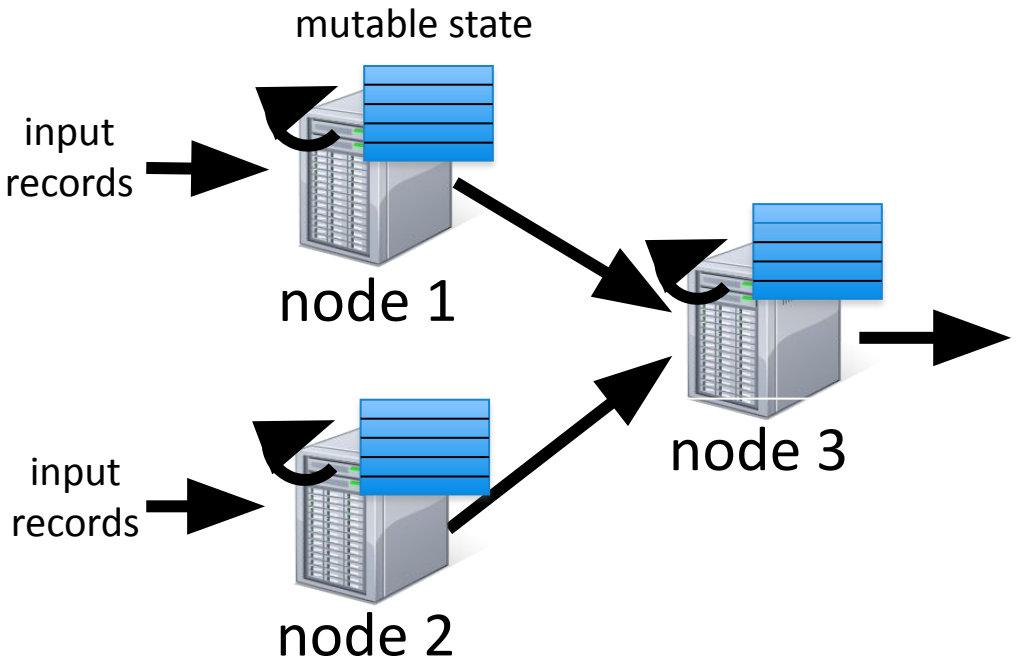
BIG DATA

Requirements

- **Scalable** to large clusters
- **Second-scale** latencies
- **Simple** programming model
- **Integrated** with batch & interactive processing

Stateful Stream Processing

- Traditional streaming systems have a event-driven **record-at-a-time** processing model
  - Each node has mutable state
  - For each record, update state & send new records
- State is lost if node dies!
- Making stateful stream processing be fault-tolerant is challenging



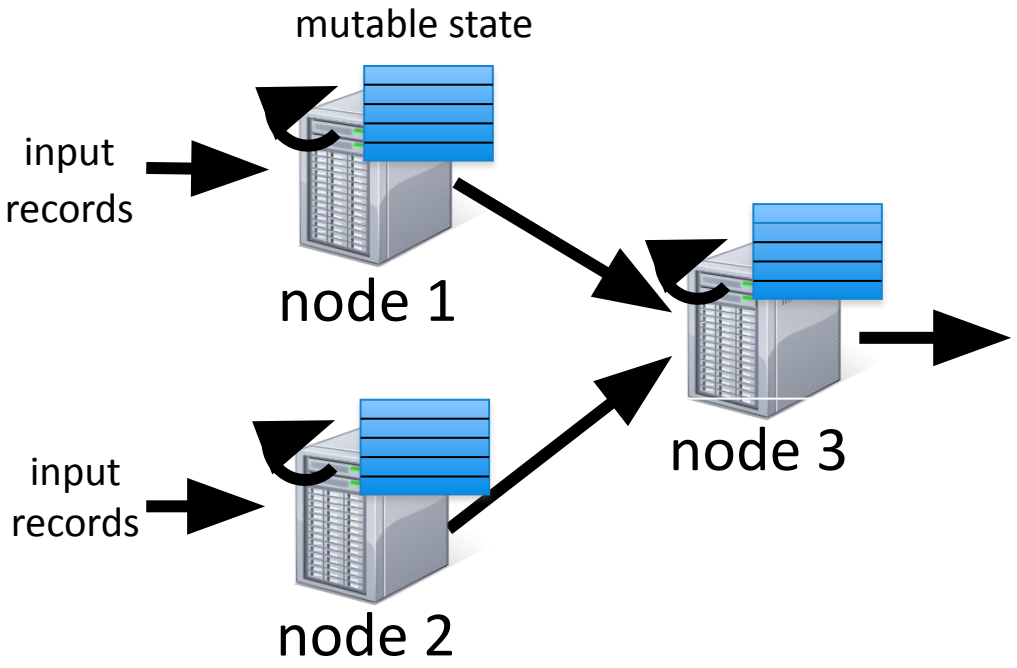
Exercise 3: Stateful Stream Processing

- Traditional streaming systems have a event-driven **record-at-a-time** processing model
  - Each node has mutable state
  - For each record, update
    records

In the stock market example, where is the state?

- State is lost if node dies!

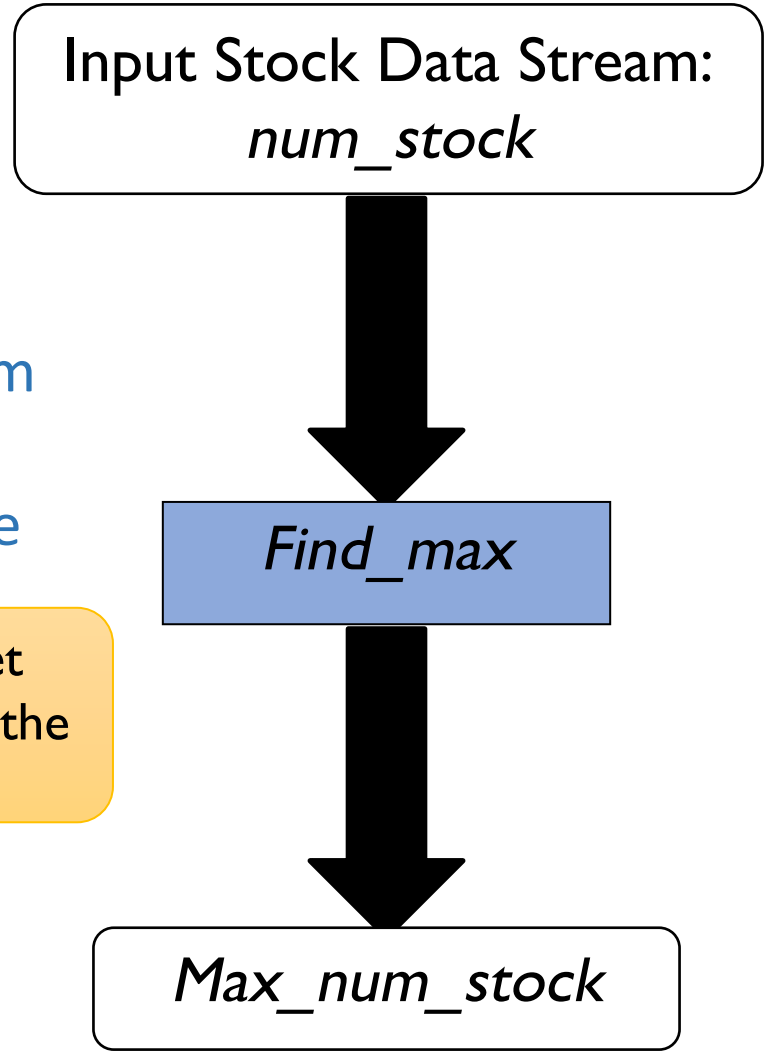
- Making stateful stream processing be fault-tolerant is challenging



Exercise 3: (solution): Stateful Stream Processing

- Write the pseudo-code for Find_max
  - If num_stock > Max_num_stock
    - Max_num_stock = num_stock
- Max_num_stock is a global state
  - Its value depends upon the entire stream sequence
  - The first num_stock could have been the largest

In the stock market example, where is the state?



Existing Streaming Systems

- Storm
  - Replays record if not processed by a node
  - Processes each record at least once
  - May update mutable state twice!
  - Mutable state can be lost due to failure!

- Trident – Use transactions to update state
  - Processes each record exactly once
  - Per state transaction updates slow

https://storm.apache.org/

https://storm.apache.org/releases/current/Trident-tutorial.html

BIG DATA

Requirements

- **Scalable** to large clusters
- **Second-scale** latencies
- **Simple** programming model
- **Integrated** with batch & interactive processing
- **Efficient fault-tolerance** in stateful computations

Spark Streaming

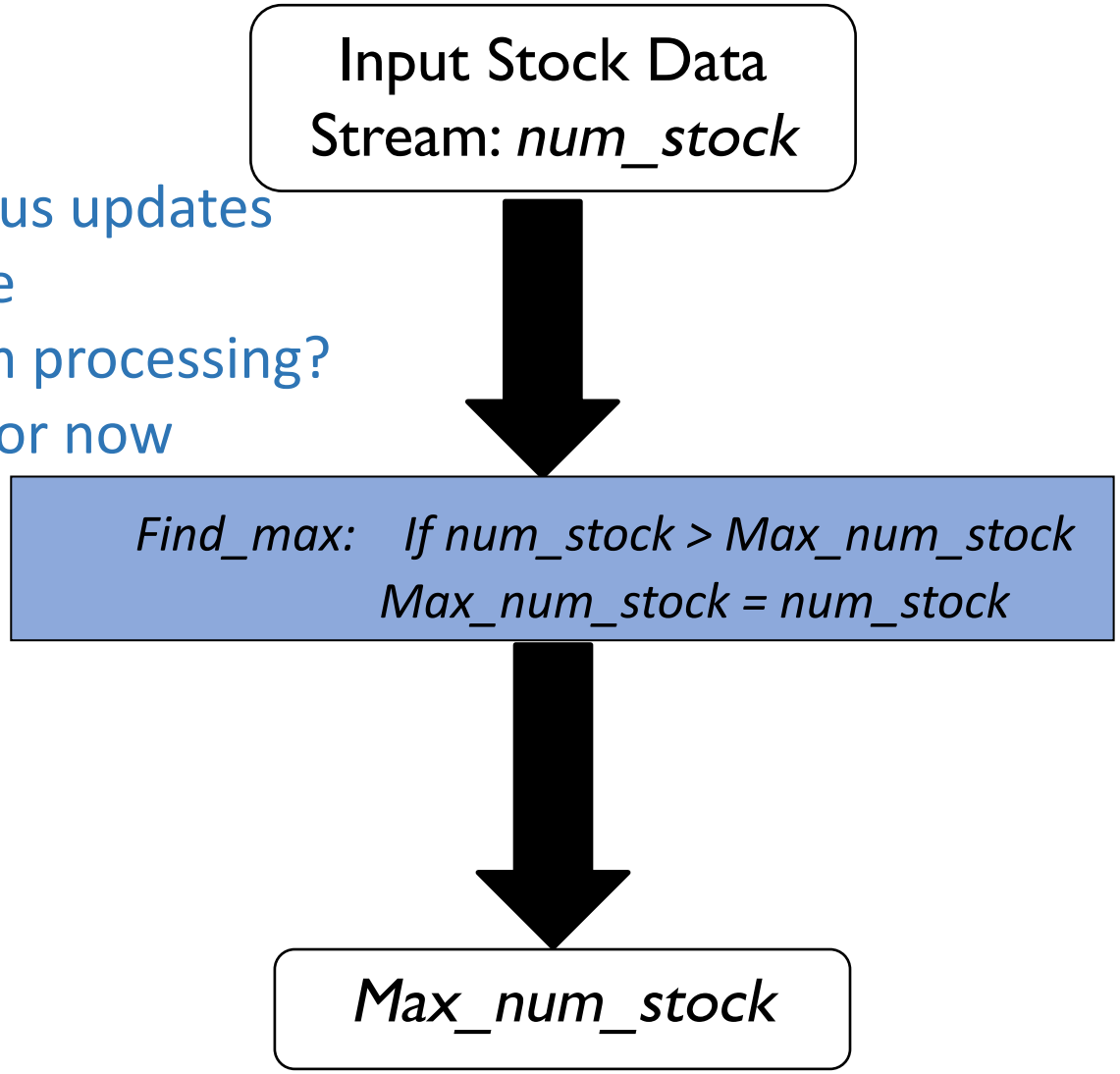
What is Spark Streaming?

- Framework for large scale stream processing
  - Scales to 100s of nodes
  - Can achieve second scale latencies
  - Integrates with Spark's batch and interactive processing
  - Provides a simple batch-like API for implementing complex algorithm
  - Can absorb live data streams from Kafka, Flume, ZeroMQ, etc.

Exercise 4:

Can we modify Hadoop?

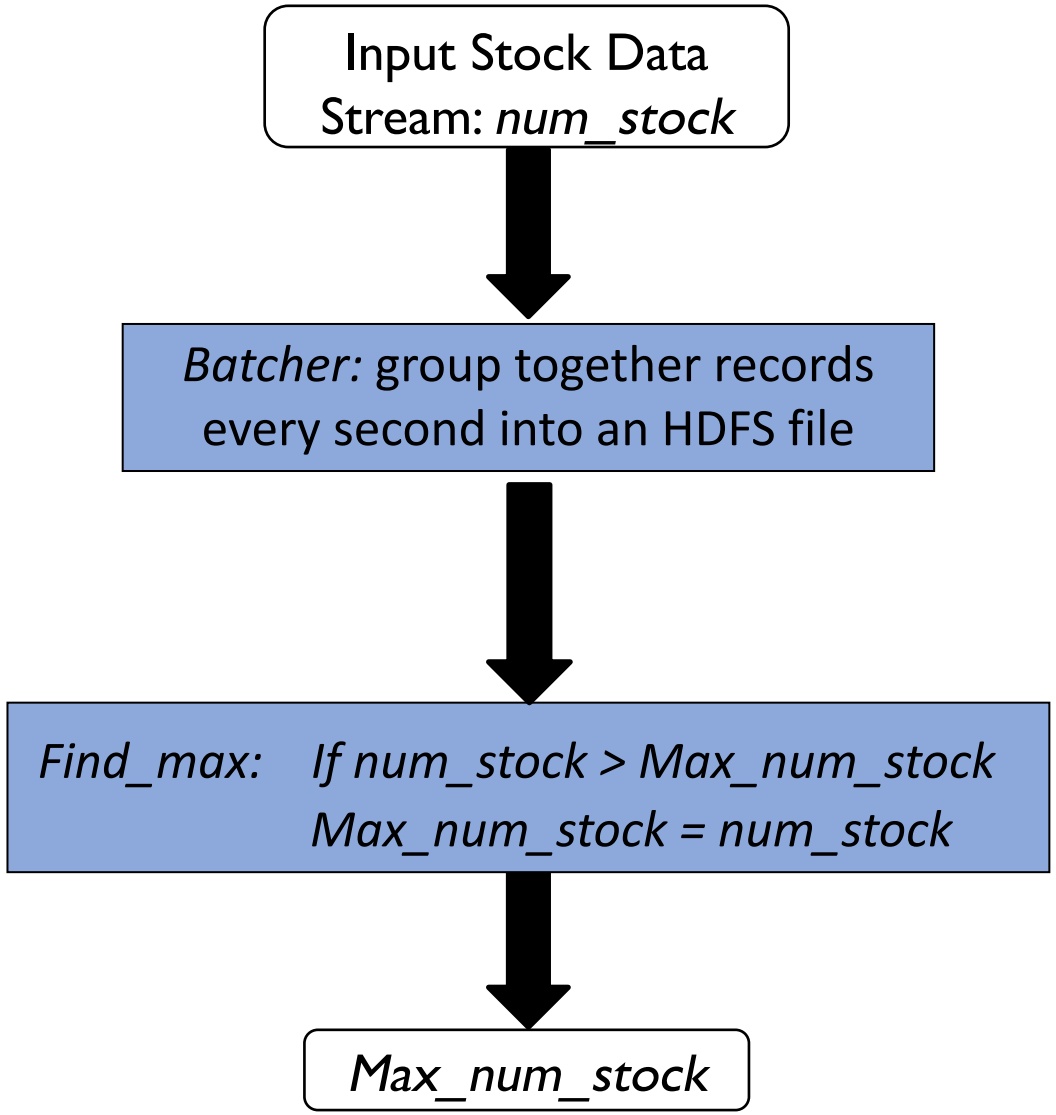
- Suppose we don't want instantaneous updates
- Say 1 second updates are acceptable
- Can we modify Hadoop to do stream processing?
- Ignore the global variable problem for now



BIG DATA

Exercise 4: Solution

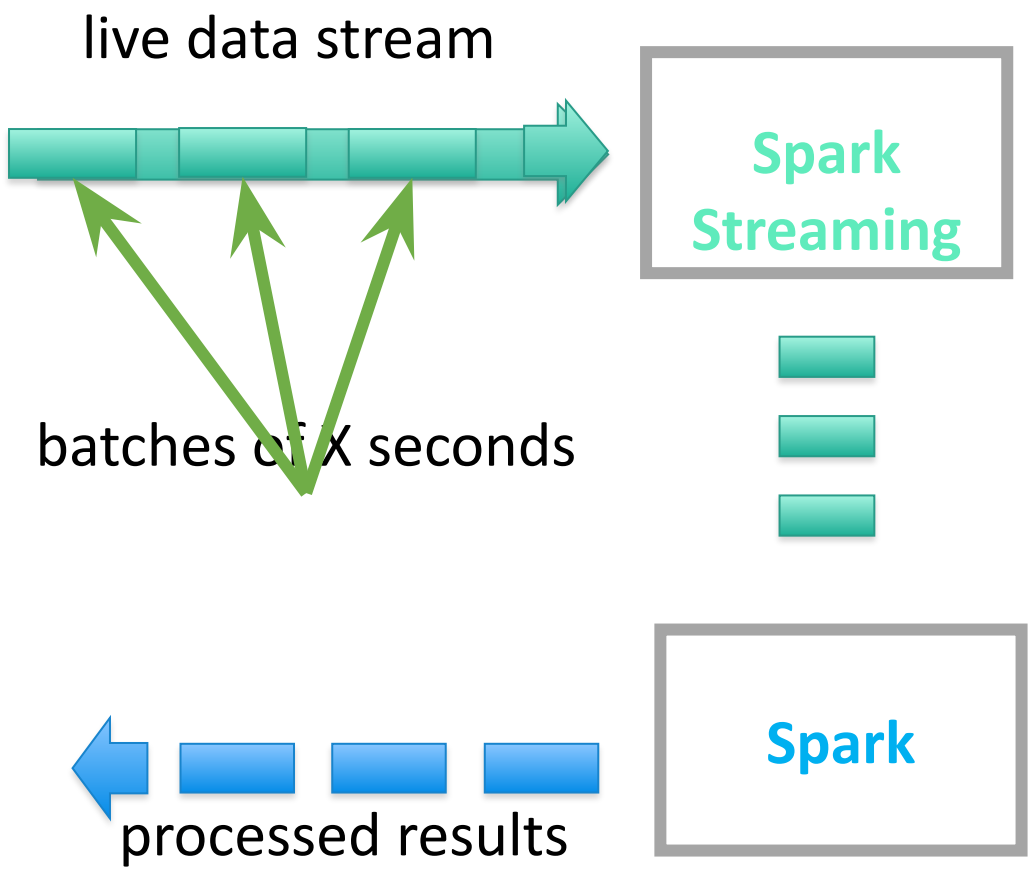
- We can *batch* input records into an HDFS file
- We can batch together the inputrecords every second
- This file can be processed using MapReduce
- We can produce an update every second
- We have ignored the global variable problem for now



Discretized Stream Processing

Run a streaming computation as a series of very small, deterministic batch jobs

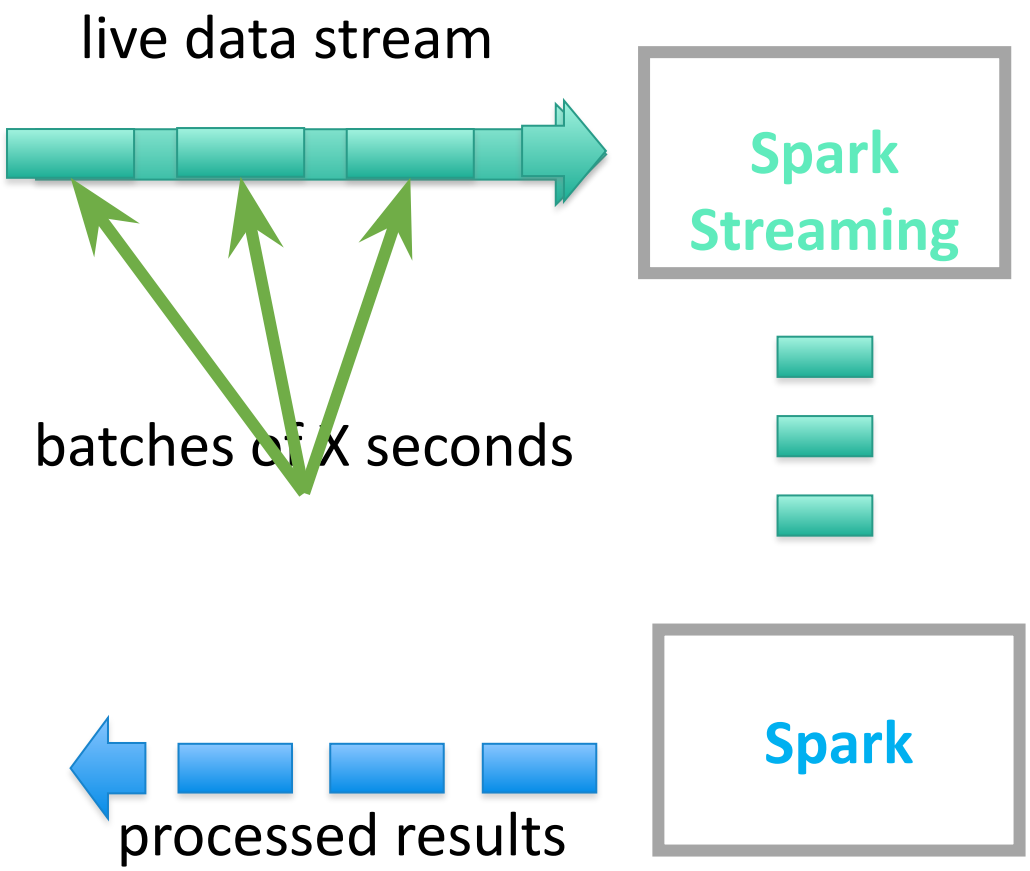
- Chop up the live stream into batches of X seconds
- Spark treats each batch of data as RDDs and processes them using RDD operations
- Finally, the processed results of the RDD operations are returned in batches



Discretized Stream Processing

Run a streaming computation as a series of very small, deterministic batch jobs

- Batch sizes as low as ½ second, latency ~ 1 second
- Potential for combining batch processing and streaming processing in the same system



Streaming Spark - DStreams

Remember!!!!

- **In Spark (not Streaming Spark)**
  - Every variable is an **RDD**
  - There are two kinds of **RDDs**
  - ***Pair RDDs*** are RDDs that consist of key-value pairs
  - **Special operations**, such as *reduceByKey* are defined on *pair RDDs*

Example 1 – Get hashtags from Twitter

```
val tweets = ssc.twitterStream(<Twitter username>, <Twitter password>)
```

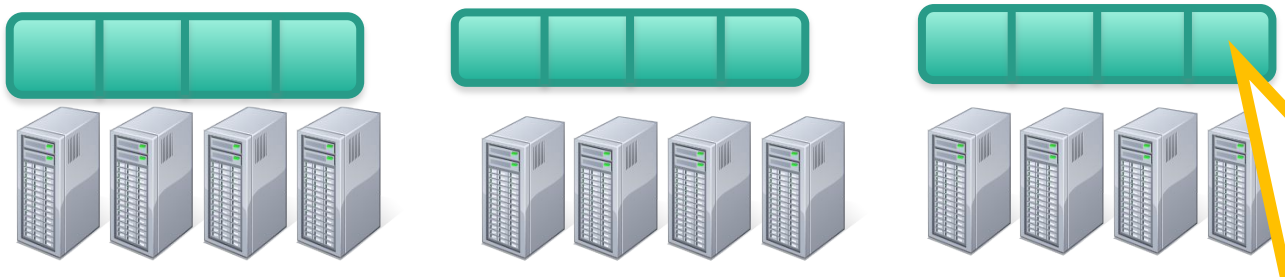
DStream: a sequence of RDD representing a stream of data

twitterstream returns a variable of type *DSTREAM*

Twitter Streaming API



tweets DStream



stored in memory as an RDD (immutable, distributed)

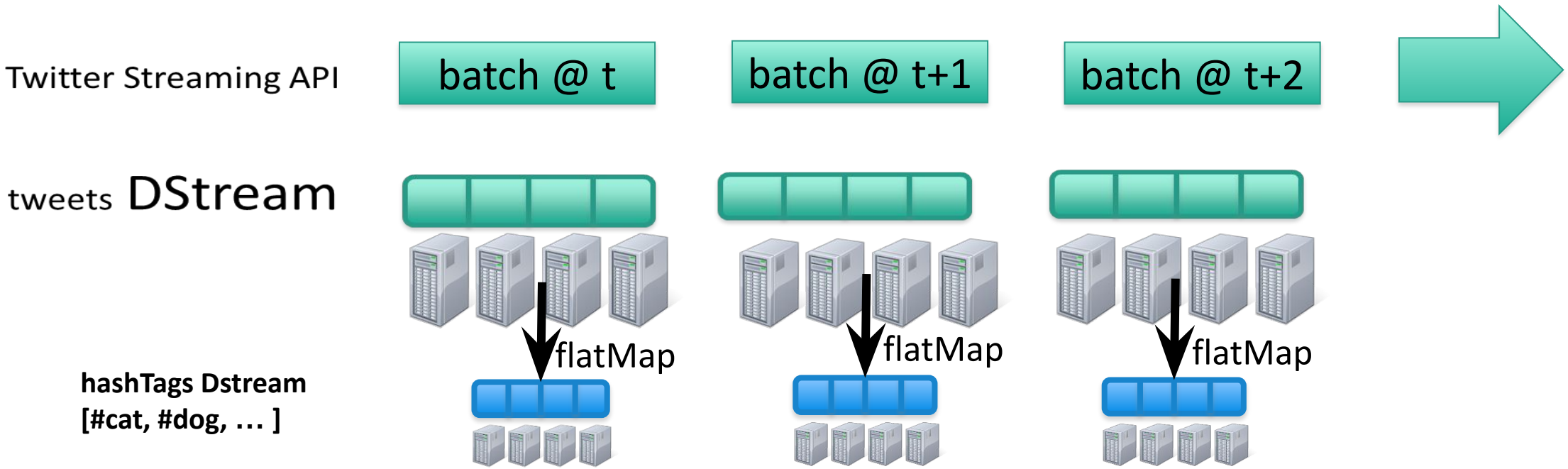
Example 1 – Get hashtags from Twitter

```
val tweets = ssc.twitterStream(<Twitter username>, <Twitter password>)
val hashTags = tweets.flatMap (status => getTags(status))
```

new DStream

transformation: modify data in one Dstream to create another DStream

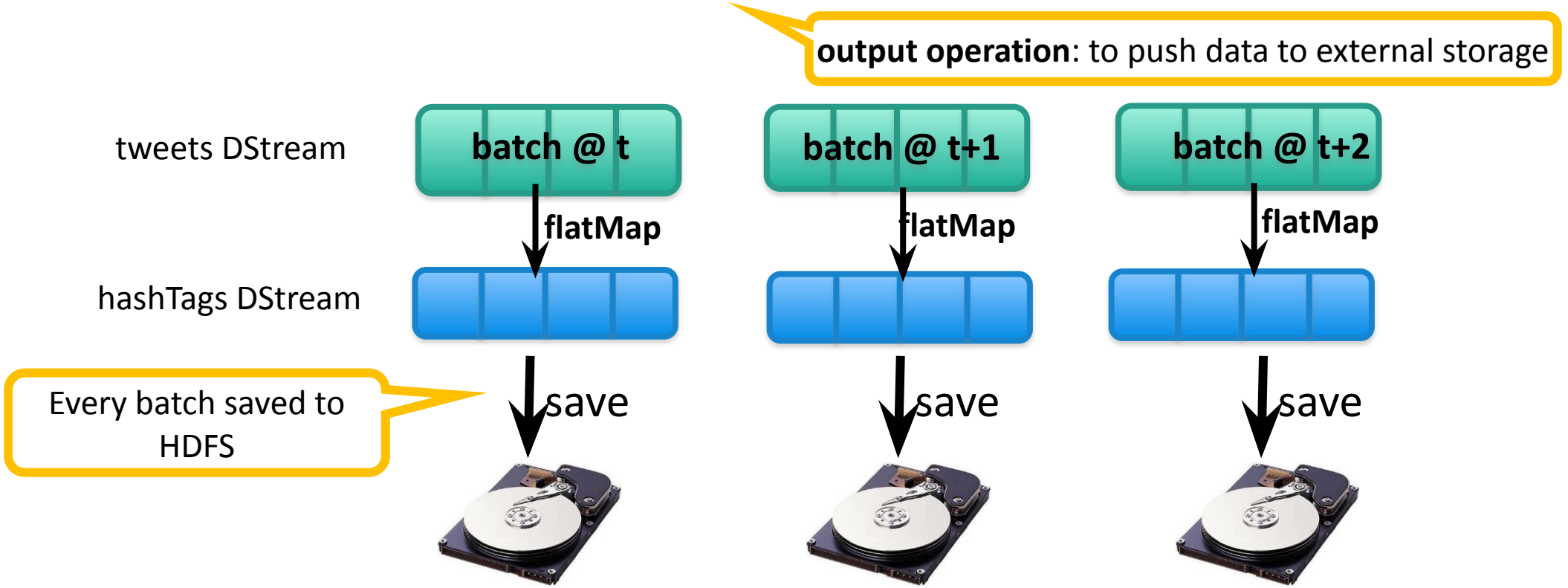
getTags is a function. A *DStream* is a sequence of RDDs. The function is applied to each RDD. The result is another *DStream*



Example 1 – Get hashtags from Twitter

```
val tweets = ssc.twitterStream(<Twitter username>, <Twitter password>)
val hashTags = tweets.flatMap (status => getTags(status))

hashTags.saveAsHadoopFiles("hdfs://...")
```



Java Example

PES UNIVERSITY
CELEBRATING 50 YEARS

Scala

```
val tweets = ssc.twitterStream(<Twitter username>, <Twitter password>)
val hashTags = tweets.flatMap (status => getTags(status))
hashTags.saveAsHadoopFiles("hdfs://...")
```

Java

```
JavaDStream<Status> tweets = ssc.twitterStream(<Twitter username>,
<Twitter password>)
JavaDstream<String> hashTags = tweets.flatMap(new Function<...> {  })
hashTags.saveAsHadoopFiles("hdfs://...")
```

Function object to define the transformation

Streaming Spark - execution

Streaming Spark Flow

Dstreams and Receivers

- Twitter, HDFS, Kafka, Flume

Transformations

- Standard RDD operations – map, countByValue, reduce, join, …
- Stateful operations – window, countByValueAndWindow, …

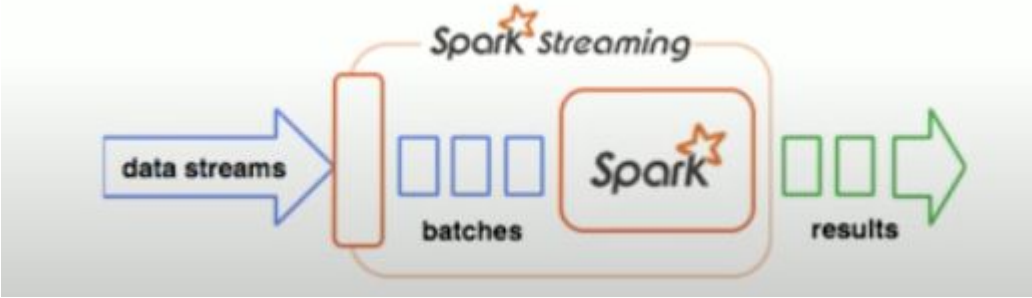
Output Operations on Dstreams

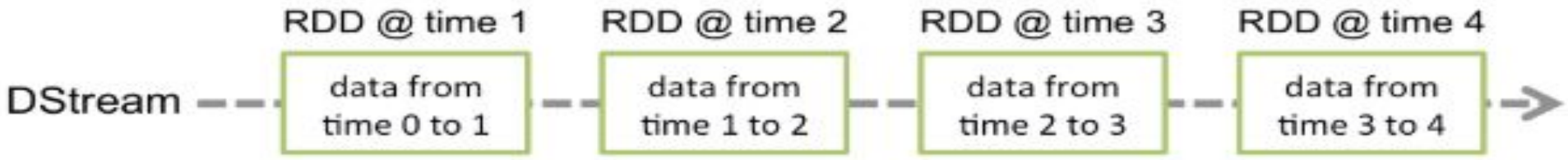
- saveAsHadoopFiles – saves to HDFS
- foreach – do anything with each batch of results



Dstreams and Receivers

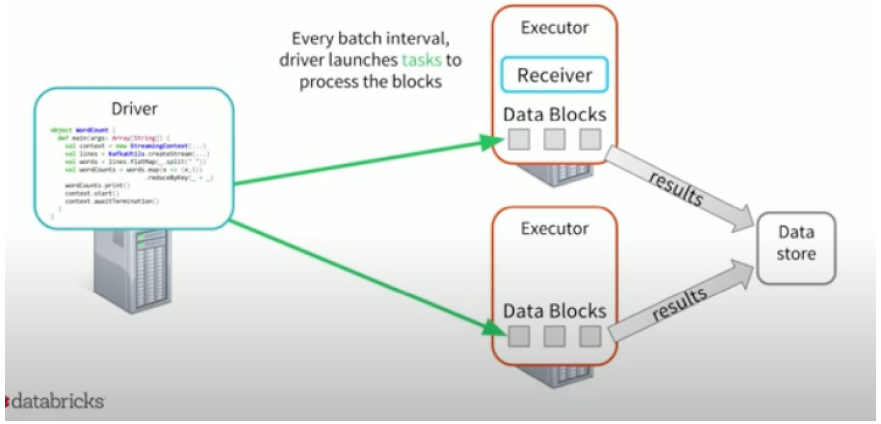
- **Streaming Spark processes data in batches**
  - Together termed the Dstream
- **Every Dstream is associated with a Receiver**
  - Receivers read data from a source and store into Spark Memory for processing
  - Types of sources
    - Basic – file systems and sockets
    - Advanced – Kafka, flume
- **Relationship between Dstream and RDD**





Dstreams and Receivers

- Streaming Spark processes job
- Starts a Receiver on an executor as a long running job
- Driver starts tasks to process blocks on every interval



Source: https://www.youtube.com/watch?v=NHfggxItokg

BIG DATA

Transformations in Spark

- Stateless transformations
- Stateful Transformations

Spark Streaming – stateless processing

Stateless transformations in Spark

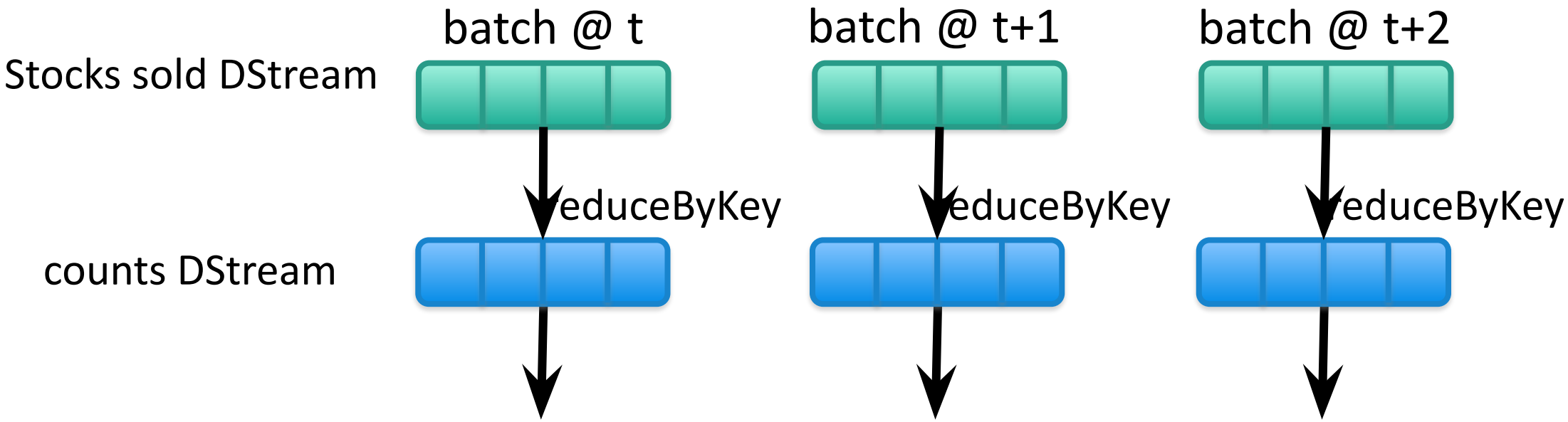
- Transformation is applied to every batch of data independently.
- No information is carried forward between one batch and the next batch
- **Examples**
  - Map()
  - FlatMap()
  - Filter()
  - Repartition()
  - reduceByKey()
  - groupByKey()

Class Exercise : Stateless stream processing (10 mins)

PES UNIVERSITY
CELEBRATING 50 YEARS

- Consider a Dstream on stock quotes generated similar to earlier that contains
  - A sequence of tuples that contain <company name, stock sold>
  - Need to find total shares sold per company in the last 1 minute
- Show Streaming spark design for the same.

Solution – count stock in every window



every batch processed independently, so no state across time

Stateful processing

Stateful transformations

- Sometimes we need to keep some state across different batches of data
- For example, what's the max amount of stock sold across the whole day for a company?
  - In this case we need to store max value for each company
- How do we store state across batches?
- First we ensure that data is in pairRDDs
  - Key, value format
  - Helps to ensure that we have a state per key

Stateful transformations

- Spark provides two options
  - Window operator – when we want a state to be maintained across short periods of time
  - Session based – where state is maintained for longer

PES UNIVERSITY
CELEBRATING 50 YEARS

Example 3 – Count the hashtags over last 10 mins

```
val tweets = ssc.twitterStream(<Twitter username>, <Twitter password>)
val hashTags = tweets.flatMap (status => getTags(status))
val tagCounts = hashTags.window(Minutes(10), Seconds(1)).countByValue()
```

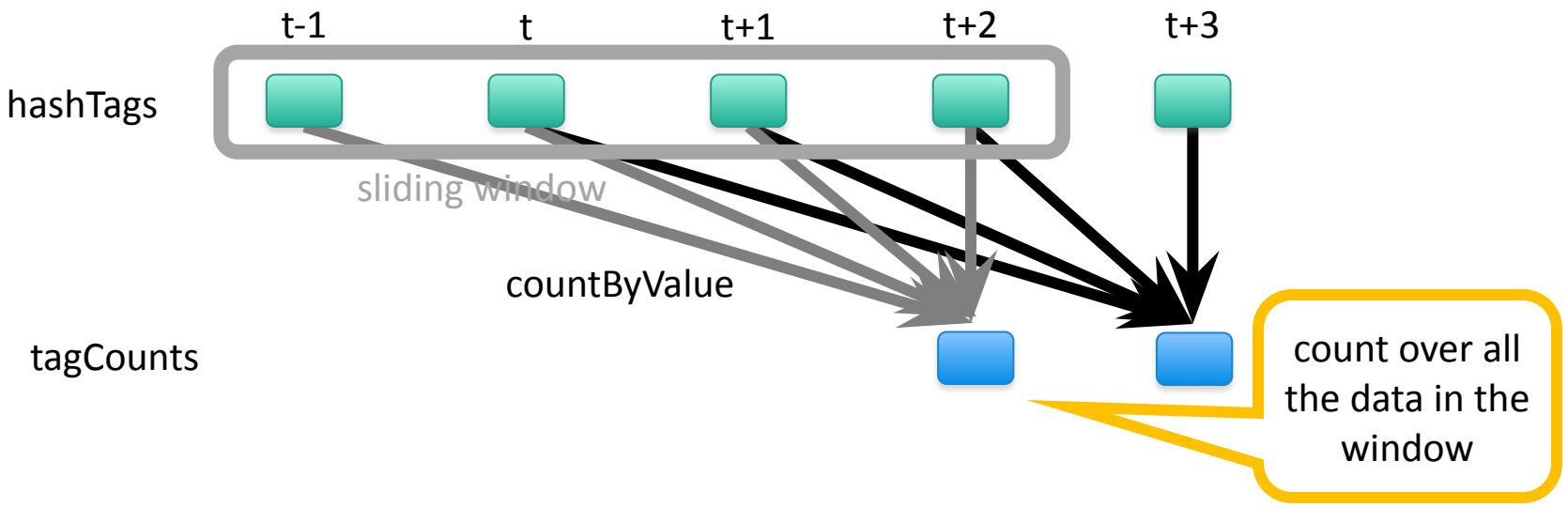
sliding window operation

window length

sliding interval

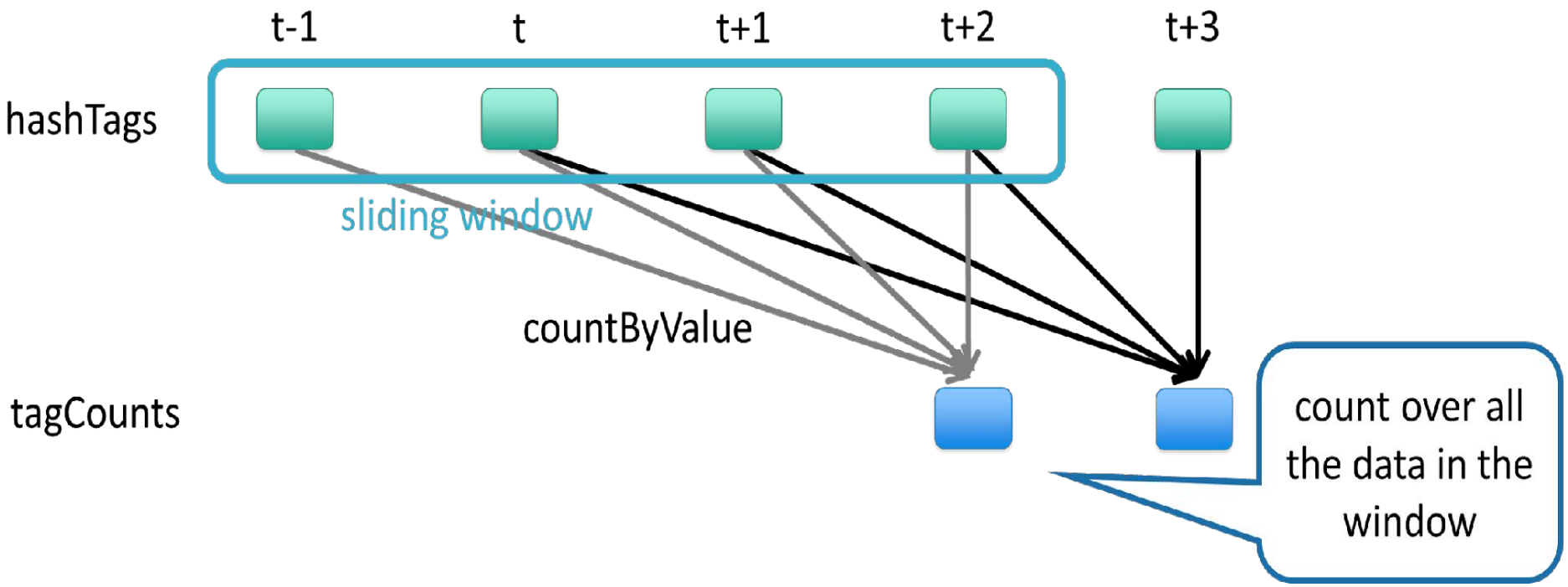
Example 3 – Counting the hashtags over last 10 mins

```
val tagCounts = hashTags.window(Minutes(10),Seconds(1)).countByValue()
```



Class Exercise: (5 mins)

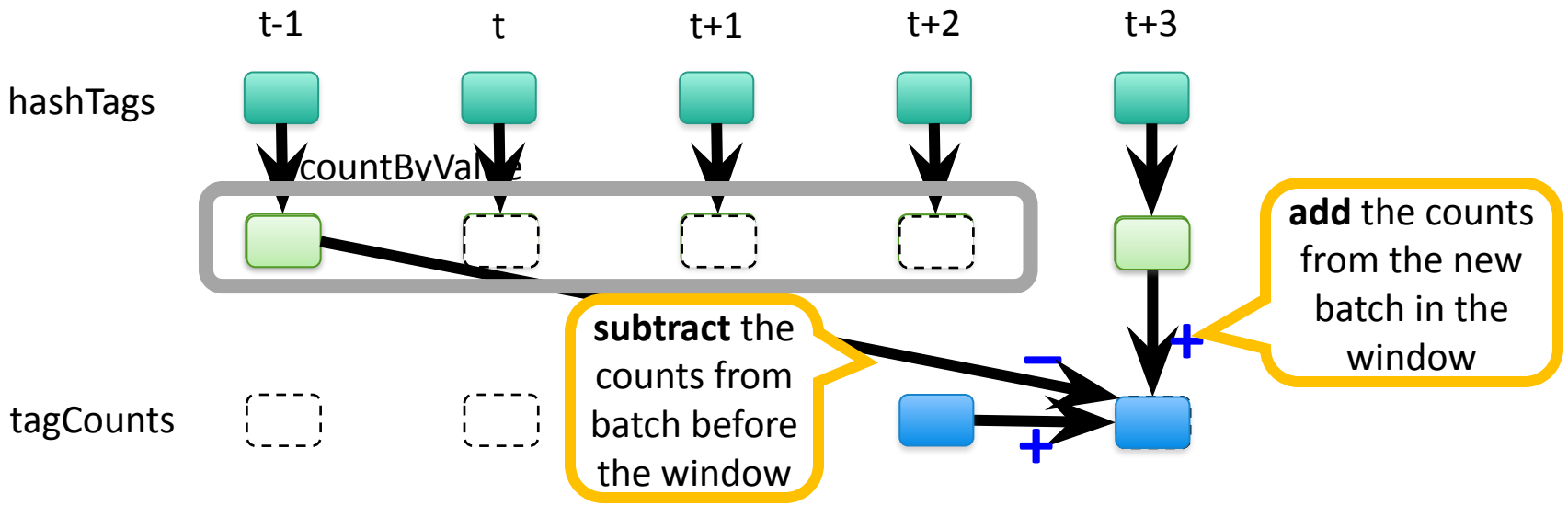
```
val tagCounts = hashTags.window(Minutes(10), Seconds(1)).countByValue()
```



How can we make this count operation more efficient?

Smart window-based countByValue

```
val tagCounts = hashtags.countByValueAndWindow(Minutes(10), Seconds(1))
```



Smart window-based reduce

- Technique to incrementally compute count generalizes to many reduce operations
  - Need a function to “inverse reduce” (“subtract” for counting)

- Could have implemented counting as:

  hashTags.reduceByKeyAndWindow(_ + _, _ - _, Minutes(1), …)

Session based state

- Maintaining arbitrary state, track sessions
  - Maintain per-user mood as state, and update it with his/her tweets

```
tweets.updateStateByKey(tweet => updateMood(tweet))
```

- `updateStateByKey` uses the current mood and the mood in the tweet to update the user's mood

Exercise 4 – Maintaining State (10 minutes)

- Consider the code to the right
  - What has to be the structure of the RDD *tweets*?
    - Hint – note that *updateStateByKey* needs a key
  - What does the function *updateMood* do?
    - Hint – note that it should update per-user mood

- Maintaining arbitrary state, track sessions
  - Maintain per-user mood as state, and update it with his/her tweets

```
tweets.updateStateByKey(tweet => updateMood(tweet))
```



Exercise 4 – Maintaining State: solution

- What has to be the structure of the RDD *tweets*?
  - Must consist of key value pairs with user as key and mood as value
    - Dinkar Happy
    - KVS VeryHappy
    - …
- What does the function *updateMood* do?
  - Compute the new mood based upon the old mood and tweet
- Suppose user KVS (key) tweets “Eating icecream” (value)

- Maintaining arbitrary state, track sessions
  - Maintain per-user mood as state, and update it with his/her tweets

```
tweets.updateStateByKey(tweet =>
updateMood(tweet))
```

Exercise 4 – Maintaining State: solution

- Consider the code to the right
  - What has to be the structure of the RDD *tweets*?
    - Must consist of key value pairs with user as key and mood as value
      - Dinkar Happy
      - KVS VeryHappy
      - …
  - Suppose user Dinkar (key) tweets "Eating icecream" (value)
    - *updateStateByKey* finds the current mood – Happy
    - current mood (Happy) and tweet (Eating icecream) is passed to *updateMood*
    - *updateMood* calculates new mood as VeryHappy
    - *updateStateByKey* stores the new mood for Dinkar as VeryHappy

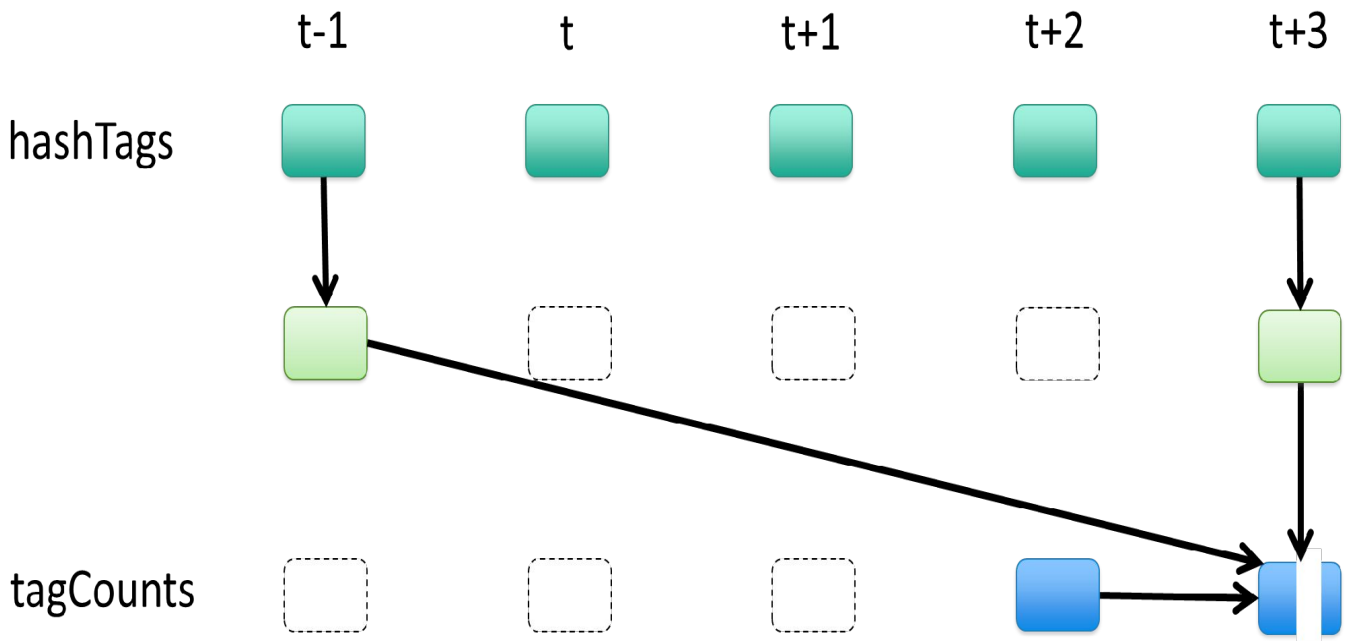
- Maintaining arbitrary state, track sessions
  - Maintain per-user mood as state, and update it with his/her tweets

```
tweets.updateStateByKey(tweet => updateMood(tweet))
```

Fault Tolerant stateful processing

Fault-tolerant Stateful Processing

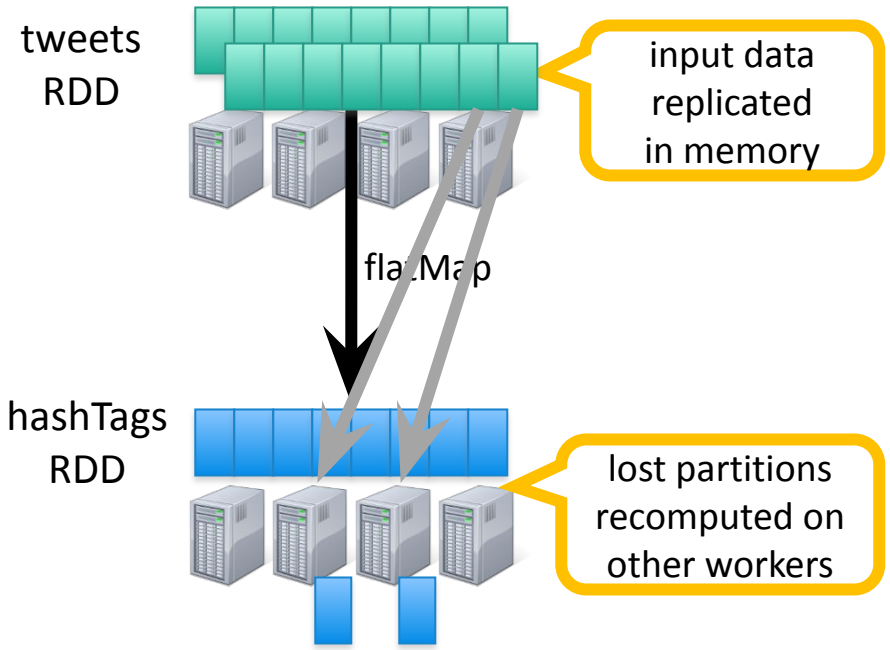
- All intermediate data are RDDs, hence can be recomputed if lost.



BIG DATA

Fault-tolerance

- RDDs remember the sequence of operations that created it from the original fault-tolerant input data
- Batches of input data are replicated in memory of multiple worker nodes, therefore fault-tolerant
- Data lost due to worker failure, can be recomputed from input data



Fault Tolerance

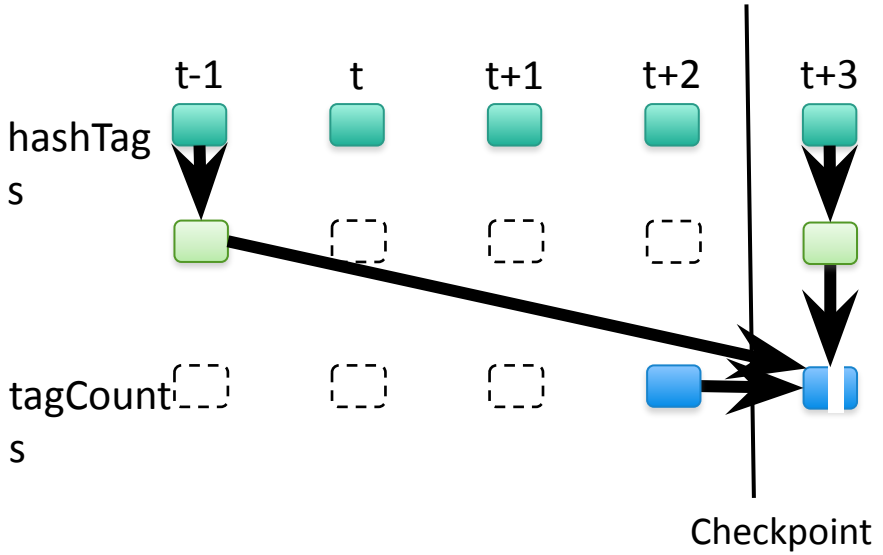
- What happens if there is a failure with
  - Stateless processing
  - Stateful processing

- Stateless
  - Previous history is not required.
  - Processing can just be recomputed
- Stateful
  - State from previous batches required for computation.
  - How much state to retain?

BIG DATA

Checkpointing

- Sometimes there may be too much data to be stored
  - For a streaming algorithm, may have to store all the streams
- Checkpointing
  - Stores an RDD
  - Forgets the lineage
  - A checkpoint at *t+2* will
    - store the hashTags and tagCounts at t+2
    - Forget the rest of the lineage

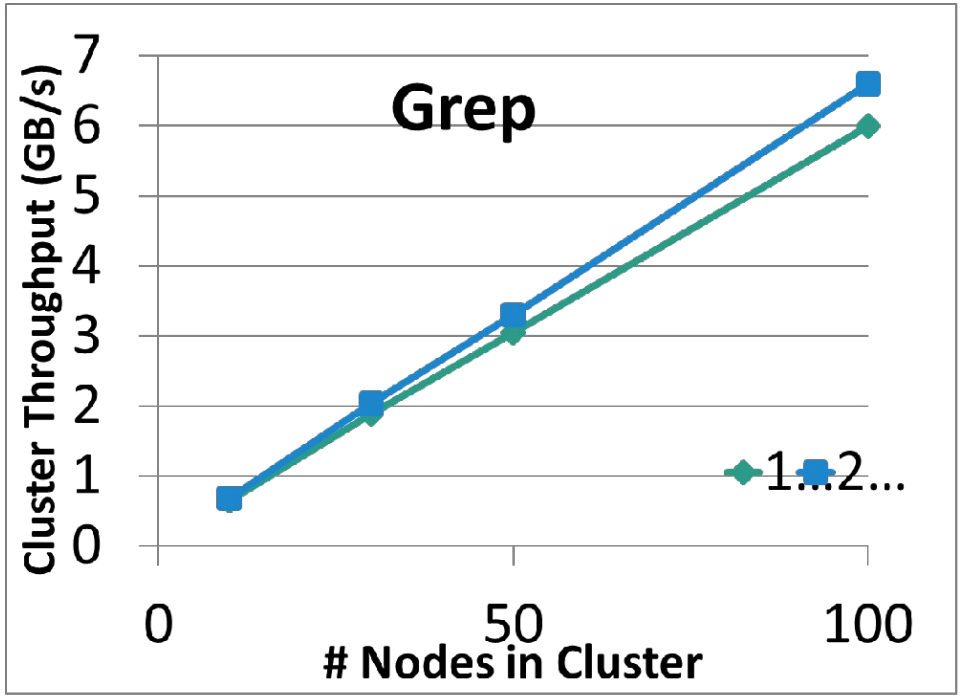


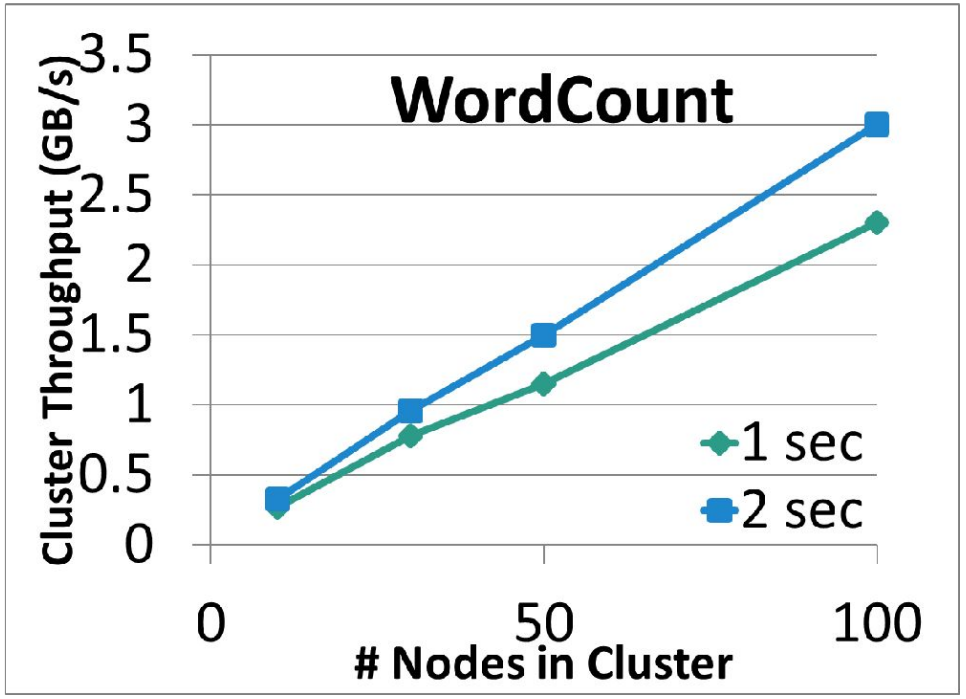
Performance

BIG DATA

Performance

- Can process **6 GB/sec (60M records/sec)** of data on 100 nodes at **sub-second** latency
  - Tested with 100 streams of data on 100 EC2 instances with 4 cores each

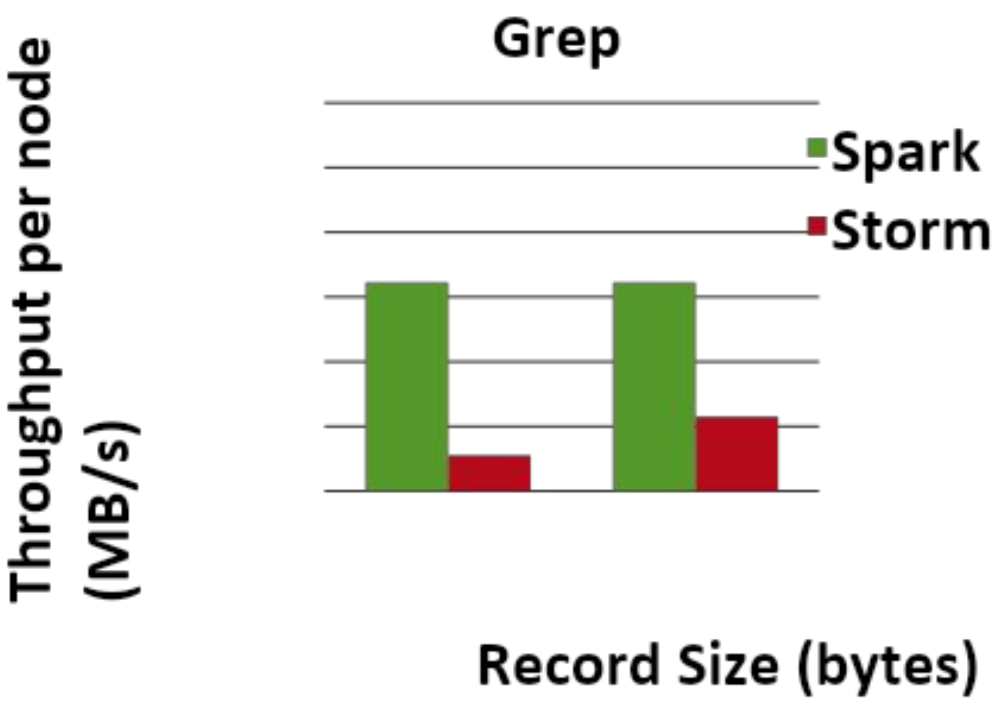


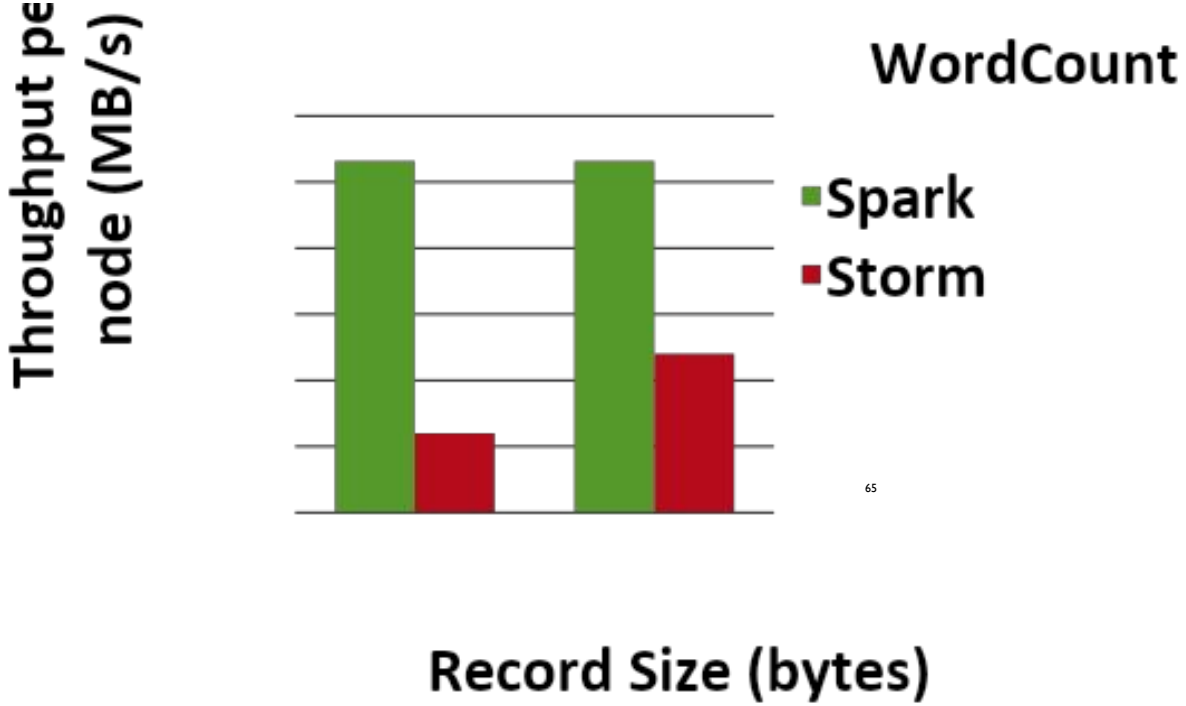


Comparison with Storm and S4

Higher throughput than Storm

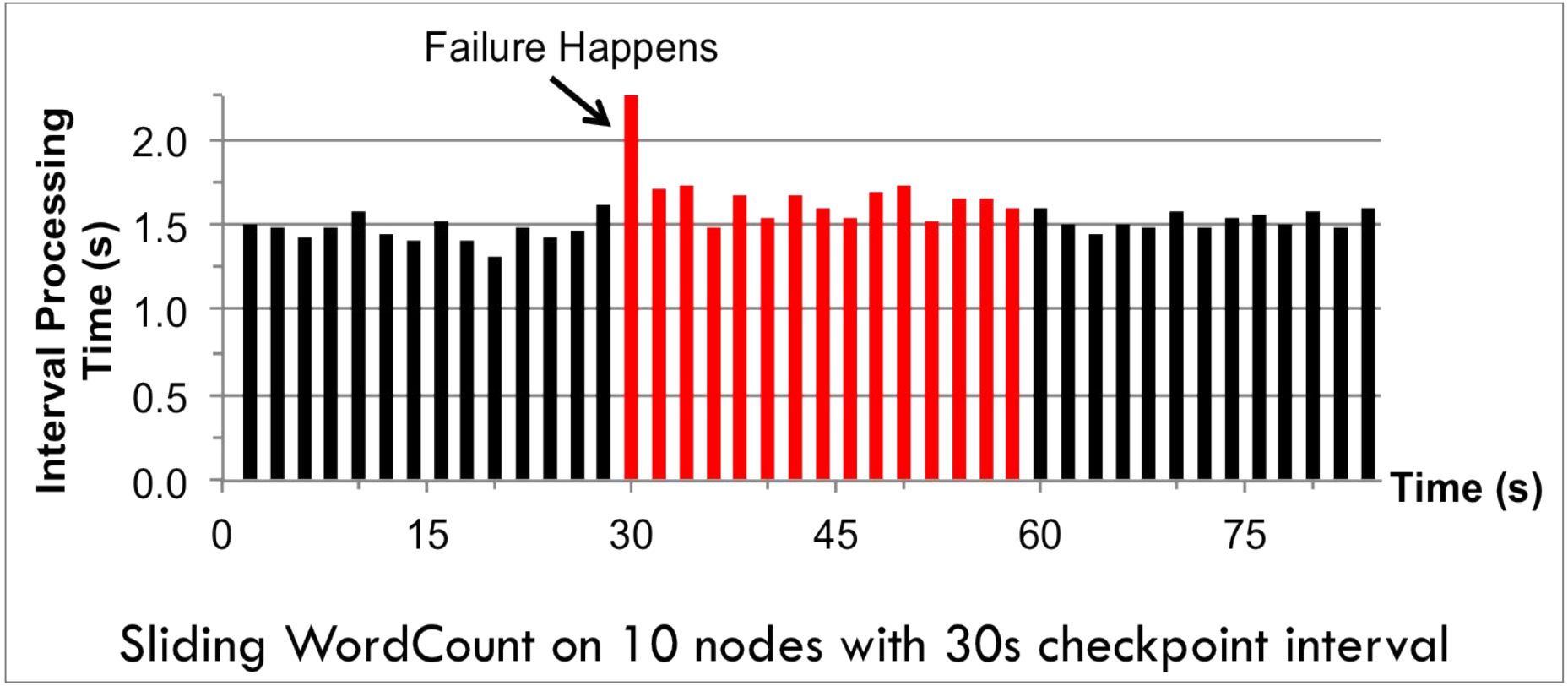
- Spark Streaming: **670k** records/second/node
- Storm: **115k** records/second/node
- Apache S4: 7.5k records/second/node





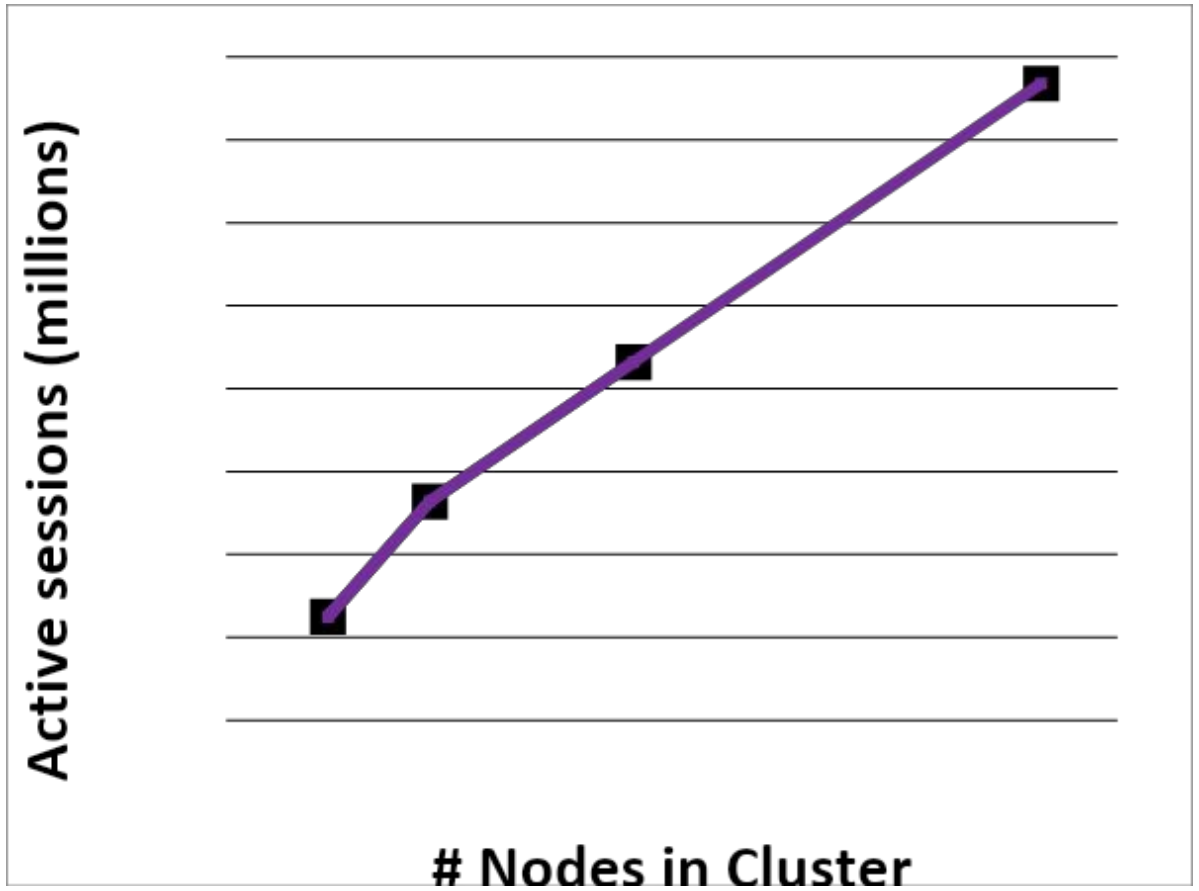
Fast Fault Recovery

- Recovers from faults/stragglers within **1 sec**



Real Applications: Conviva

- Real-time monitoring of video metadata

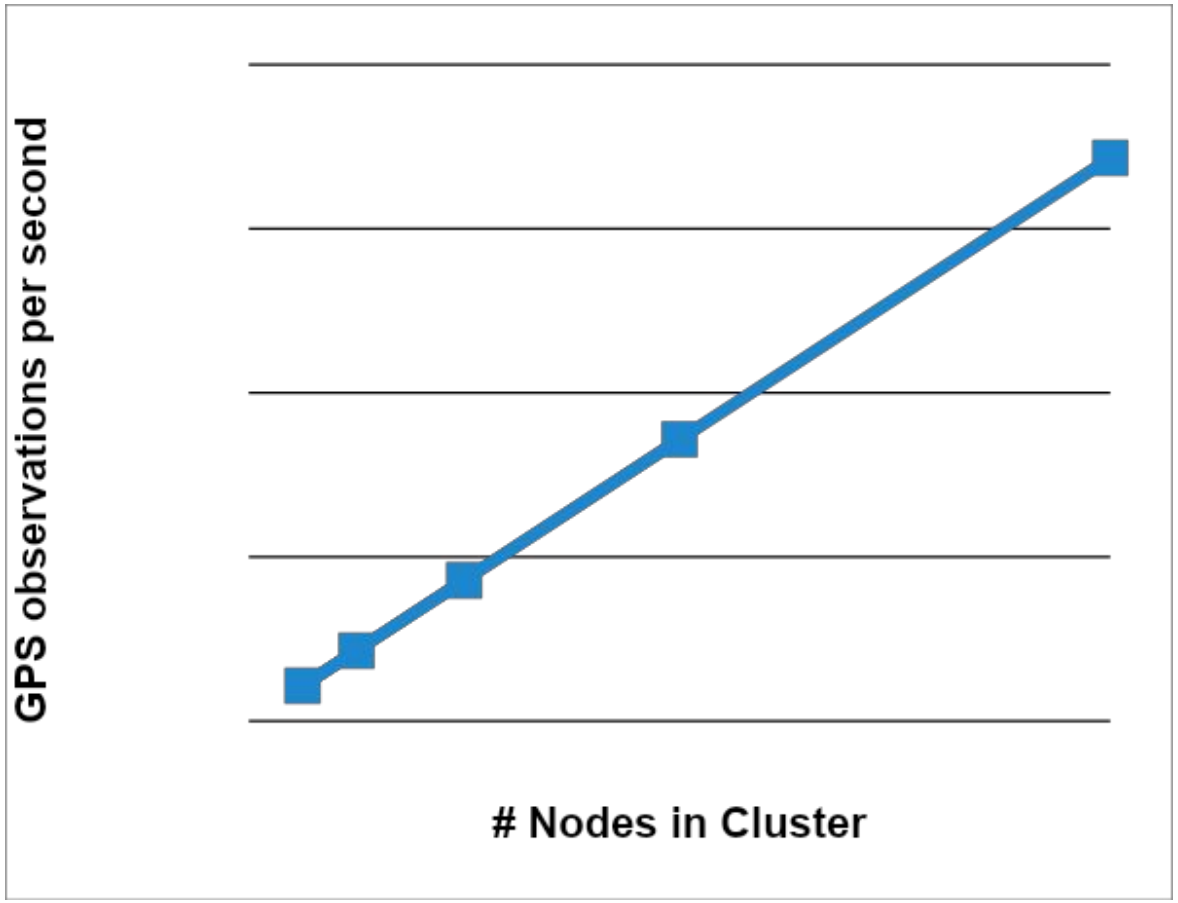


- Achieved 1-2 second latency
- Millions of video sessions processed
- Scales linearly with cluster size

BIG DATA

Real Applications: Mobile Millennium Project

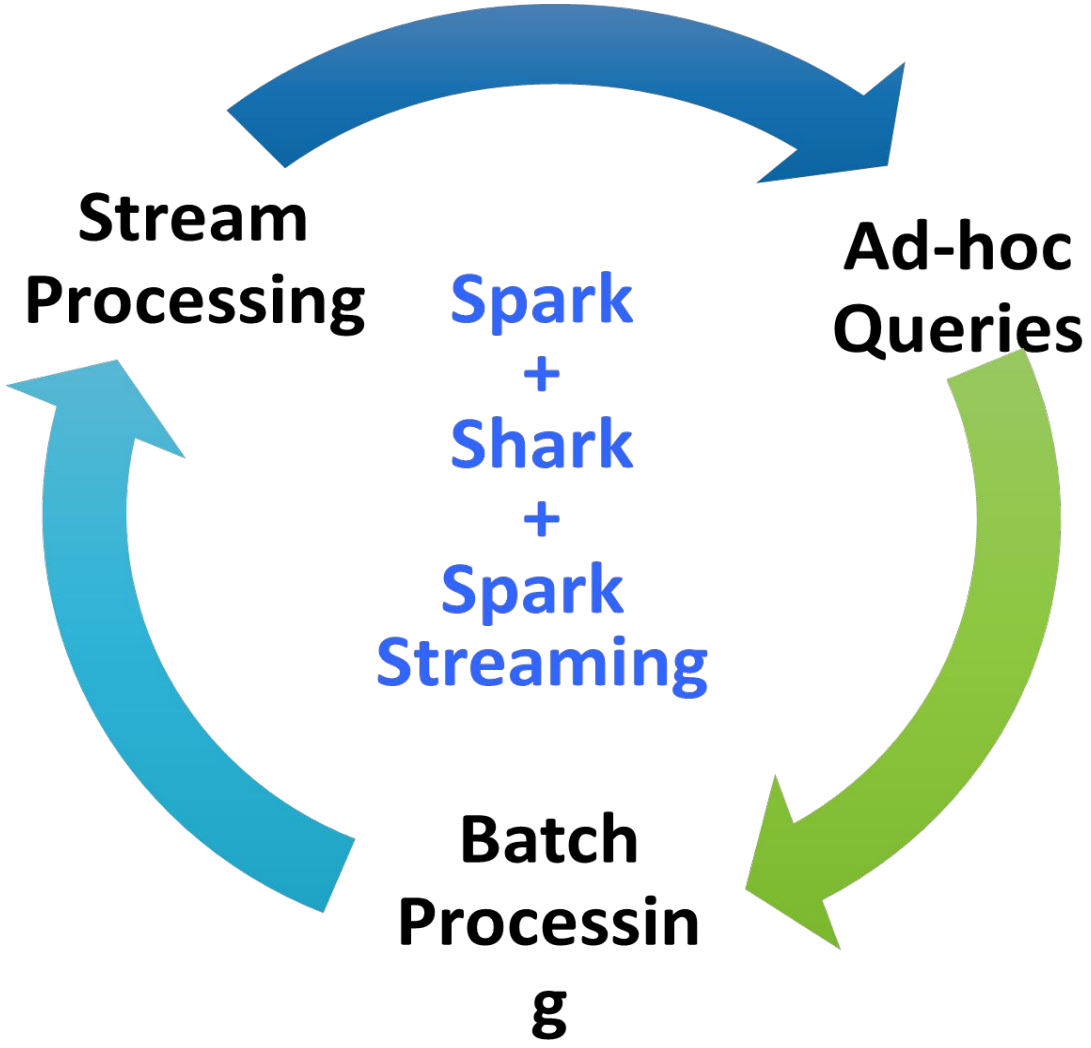
- Traffic transit time estimation using online machine learning on GPS observations



- Markov chain Monte Carlo simulations on GPS observations
- Very CPU intensive, requires dozens of machines for useful computation
- Scales linearly with cluster size

Putting it all together

Vision - one stack to rule them all



Spark program vs Spark Streaming program

PES UNIVERSITY
CELEBRATING 50 YEARS

Spark Streaming program on Twitter stream

```
val tweets = ssc.twitterStream(<Twitter username>,
<Twitter password>)
val hashTags = tweets.flatMap (status => getTags(status))
hashTags.saveAsHadoopFiles("hdfs://...")
```

Spark program on Twitter log file

```
val tweets = sc.hadoopFile("hdfs://...")
val hashTags = tweets.flatMap (status => getTags(status))
hashTags.saveAsHadoopFile("hdfs://...")
```

BIG DATA

Vision - one stack to rule them all

- Explore data interactively using Spark Shell / PySpark to identify problems
- Use same code in Spark stand-alone programs to identify problems in production logs
- Use similar code in Spark Streaming to identify problems in live log streams

```
$ ./spark-shell
scala> val file = sc.hadoopFile("smallLogs")
...
scala> val filtered = file.filter(_.contains("ERROR"))
...
```

```
object ProcessProductionData {
  def main(args: Array[String]) {
    val sc = new SparkContext(...)
    val file = sc.hadoopFile("productionLogs")
    val filtered = file.filter(_.contains("ERROR"))
    val mapped = file.map(...)
```

```
object ProcessLiveStream {
  def main(args: Array[String]) {
    val sc = new StreamingContext(...)
    val stream = sc.kafkaStream(...)
    val filtered = file.filter(_.contains("ERROR"))
    val mapped = file.map(...)
    ...
  }
}
```

BIG DATA

Vision - one stack to rule them all

- Explore data interactively using Spark Shell / PySpark to identify problems
- Use same code in Spark stand-alone programs to identify problems in production logs
- Use similar code in Spark Streaming to identify problems in live log streams

```
$ ./spark-shell
scala> val file = sc.hadoopFile("smallLogs")
...
scala> val filtered = file.filter(_.contains("ERROR"))
...
```

```
object ProcessProductionData {
  def main(args: Array[String]) {
    val sc = new SparkContext(...)
    val file = sc.hadoopFile("productionLogs")
    val filtered = file.filter(_.contains("ERROR"))
    val mapped = file.map(...)
```

```
object ProcessLiveStream {
  def main(args: Array[String]) {
    val sc = new StreamingContext(...)
    val stream = sc.kafkaStream(...)
    val filtered = file.filter(_.contains("ERROR"))
    val mapped = file.map(...)
    ...
  }
}
```

BIG DATA

Alpha Release with Spark 0.7

- Integrated with Spark 0.7
  - Import **spark.streaming** to get all the functionality
- Both Java and Scala API
- Give it a spin!
  - Run locally or in a cluster
- Try it out in the hands-on

https://spark.apache.org/docs/latest/streaming-programming-guide.html

BIG DATA

Limitations

- Streaming Spark processes data in batches
  - Near Real Time
  - Not necessarily acceptable for certain scenarios

Summary

- Stream processing framework that is ...
  - Scalable to large clusters
  - Achieves second-scale latencies
  - Has simple programming model
  - Integrates with batch & interactive workloads
  - Ensures efficient fault-tolerance in stateful computations

- For more information, checkout the paper:

https://www.usenix.org/system/files/conference/hotcloud12/hotcloud12-final28.pdf

References

- https://spark.apache.org/docs/latest/streaming-programming-guide.html
- https://spark.apache.org/streaming/
- Mining of Massive Datasets, Anand Rajaraman, Jure Leskovec, Jeffrey D. Ullman
- Big Data Analytics Beyond Hadoop: Real-Time Applications with Storm, Spark, and More Hadoop Alternatives, Vijay Srinivasa Agneeswaran



THANK YOU

Prafullata Kiran Auradkar
Department of CSE
prafullatak@pes.edu



BIG DATA

Hands On Session - 4

STREAMING SPARK

K V Subramaniam

Usha Devi B G

Dept of Computer Science and Engineering

BIG DATA

STREAMING SPARK Hands-on



SPECIFICATIONS

1. Hadoop: 3.2
2. Java: 1.8
3. Apache Spark 3.0
4. Dataset: Please download the dataset from the forum.

BIG DATA

STREAMING SPARK Hands-on

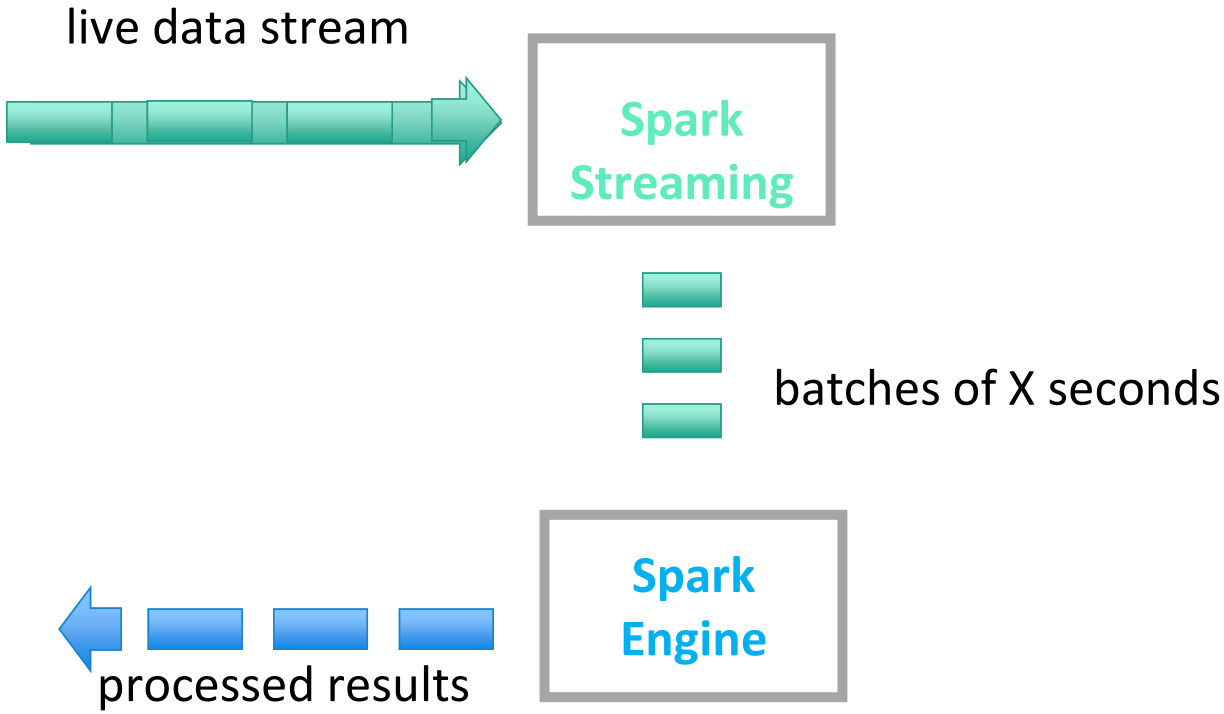


Configurations

- **Download Apache Spark using the below link**: (choose the version compatible for your Hadoop version)
  https://spark.apache.org/downloads.html
- **Untar and move the file to /usr/local/**
  $ tar –zxvf spark-3.0.0-bin-hadoop2.7.tgz
  $ mv spark-3.0.0-bin-hadoop2.7.tgz /usr/local/spark
- **Edit the .bashrc file to add the spark variables**
  export SPARK_HOME=/usr/local/spark
- **Run the below command to check whether spark has been installed successfully**.
  $ spark-submit

BIG DATA

Discretized Stream Processing



BIG DATA

Problem Statement



- Calculate the total number of tweets in the given data set using streaming spark.

- Calculate the number of tweets tweeted using Twitter for Android.

"ID","language","Date","source","len","likes","RTs","Hashtags","Usernames","Userid","name","Place","followers","friends"

01E+18;en;02-07-18 1:35;Twitter for Android;140;0;477;WorldCup POR ENG;Squawka Football;Squawka;Cayleb;Accra;861;828

01E+18;en;02-07-18 1:35;Twitter for Android;139;0;1031;WorldCup;FC Barcelona Ivan Rakitic HNS | CFF;FCBarcelona ivanrakitic HNS_CFF;Febri Aditya;Bogor;667;686

01E+18;en;02-07-18 1:35;Twitter for Android;140;0;477;WorldCup POR ENG;Squawka Football;Squawka;tar;;137;216

01E+18;en;02-07-18 1:35;Twitter Web Client;138;0;1;PowerByEXO WorldCupFIFAStadiumDJ XiuminLeague;Frida Carrillo;FridaCarrillo05;STAN LEGENDS STAN EXO;Lima Peru;7;9

BIG DATA

Steps to run spark streaming program



- Create a directory “checkpoint_BIGDATA” with write permission to store the intermediate values while computing the streaming data.

  $mkdir checkpoint_BIGDATA
  $ chmod 777 checkpoint_BIGDATA

- Use the “app.py” to stream data to spark using sockets

  $python3 app.py

- Run “spark.py” to accept streaming data and process it using spark engine

  $spark-submit spark.py

- Stderr and Stdout outputs are jumbled, redirect Stdout to a file to see the actual output

  $spark-submit spark.py > output.txt



THANK YOU

K V Subramaniam
Prafullata Auradkar
Usha Devi B G
Department of Computer Science and Engineering



BIG DATA

Streaming Algorithms - 2

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Significant information in the slide deck presented through the Unit 1 of the course have been created by **Dr. K V Subramaniam's** and would like to acknowledge and thank him for the same. There have been some information which I might have leveraged from **Dr. H L Phalachandra's** slide contents too. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

BIG DATA

Overview: Streaming Algorithms

PES UNIVERSITY
CELEBRATING 50 YEARS

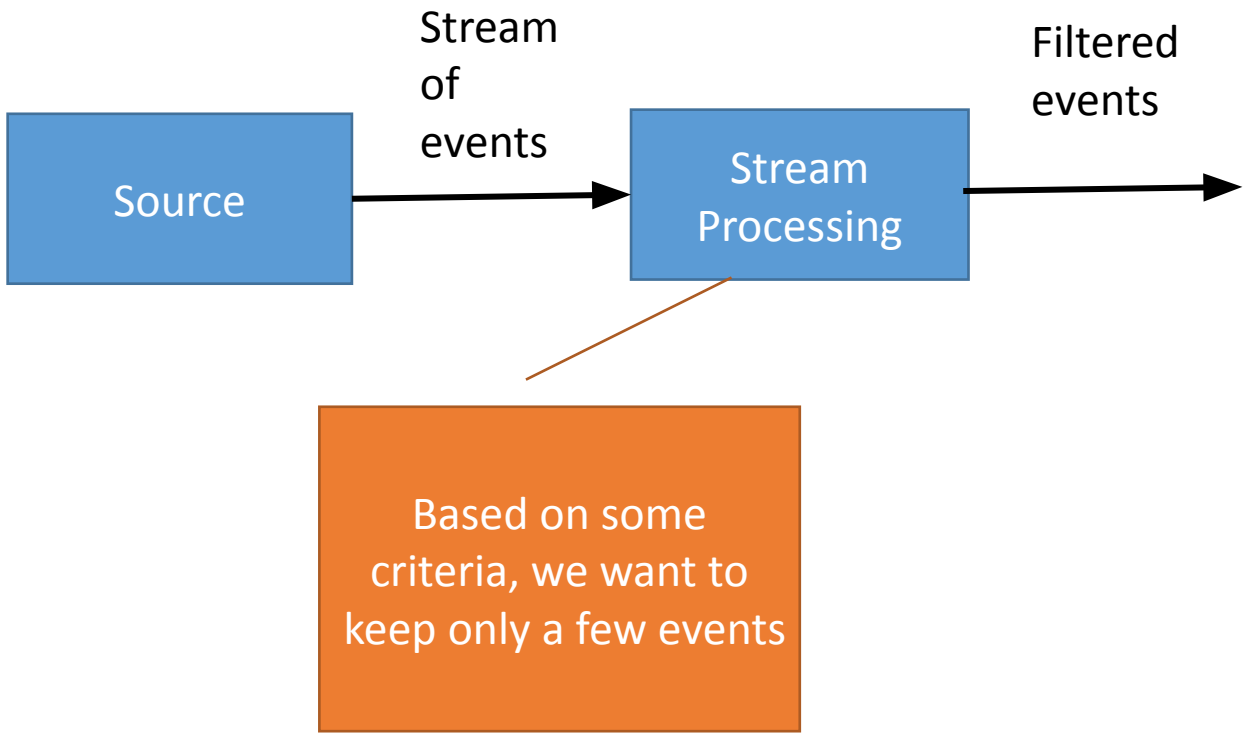
- Filtering algorithms – Bloom Filter
  - Motivation
  - General bloom filters
  - Extensions
- Counting unique elements
  - Motivation
  - Flajolet Martin Algorithm and working
  - Practical considerations

Bloom Filters

BIG DATA

Filtering Data

- Sometimes we need to take a decision
  - To filter out certain events
  - The decision has to be taken instanteneously
  - Large number of events need to be processed.

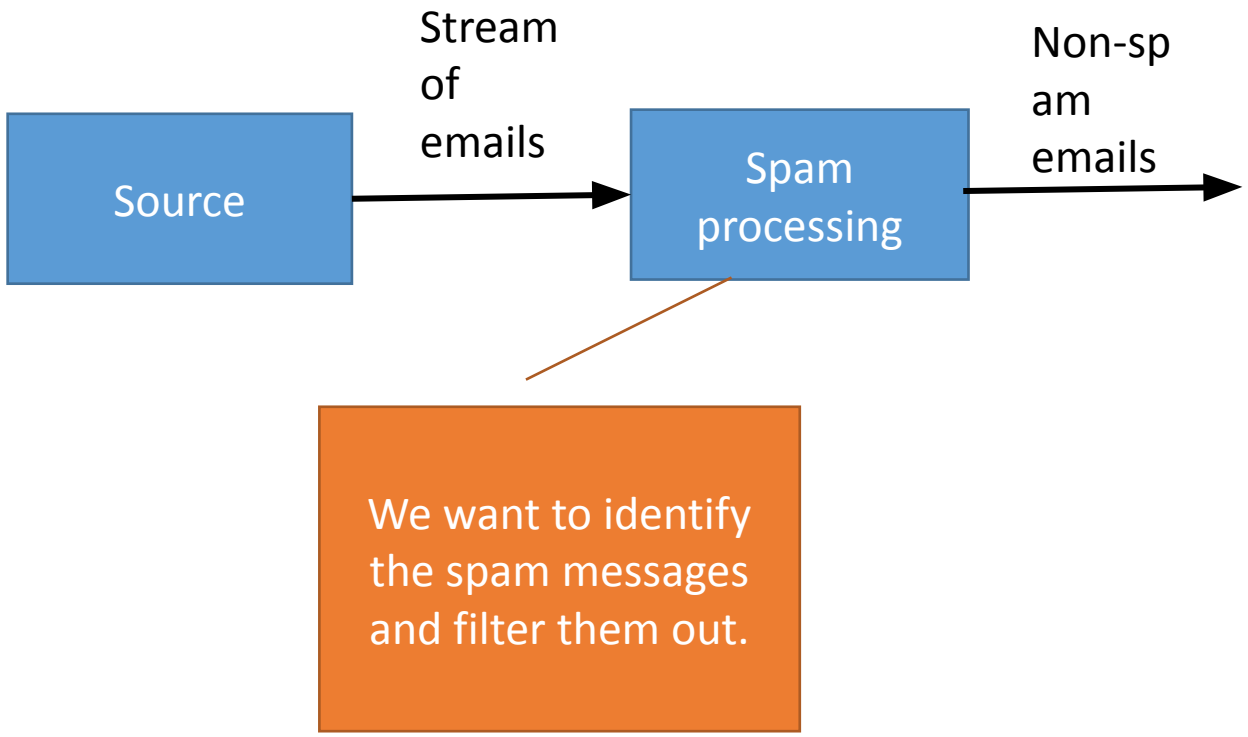


Filtering Data – Motivational example

- Incoming email
  - Remove all spam emails
- Constraints
  - 1GB of main memory
  - 1 billion non-spam email ids (well known)
  - 20 bytes/email address

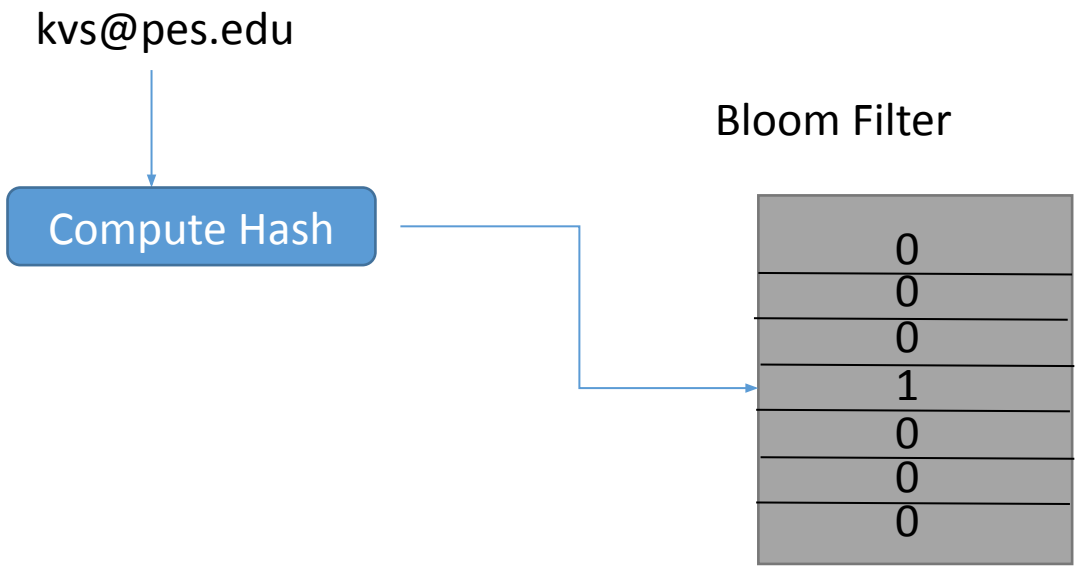
(Almost 18.63 GB)

- Can’t store all email ids in memory
  - Disk is a slow store



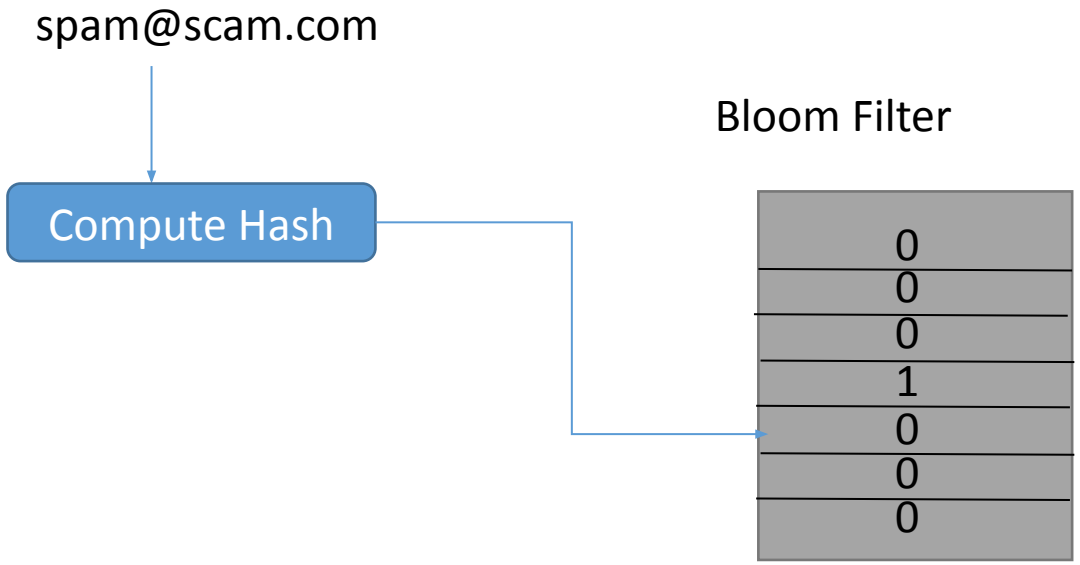
Bloom filter basic: initialization

- 1 GB memory => 8 billion bit string
- Bloom filter initialization
  - Hash non-spam email ids to 0..8 billion-1
  - Set corresponding bit to 1

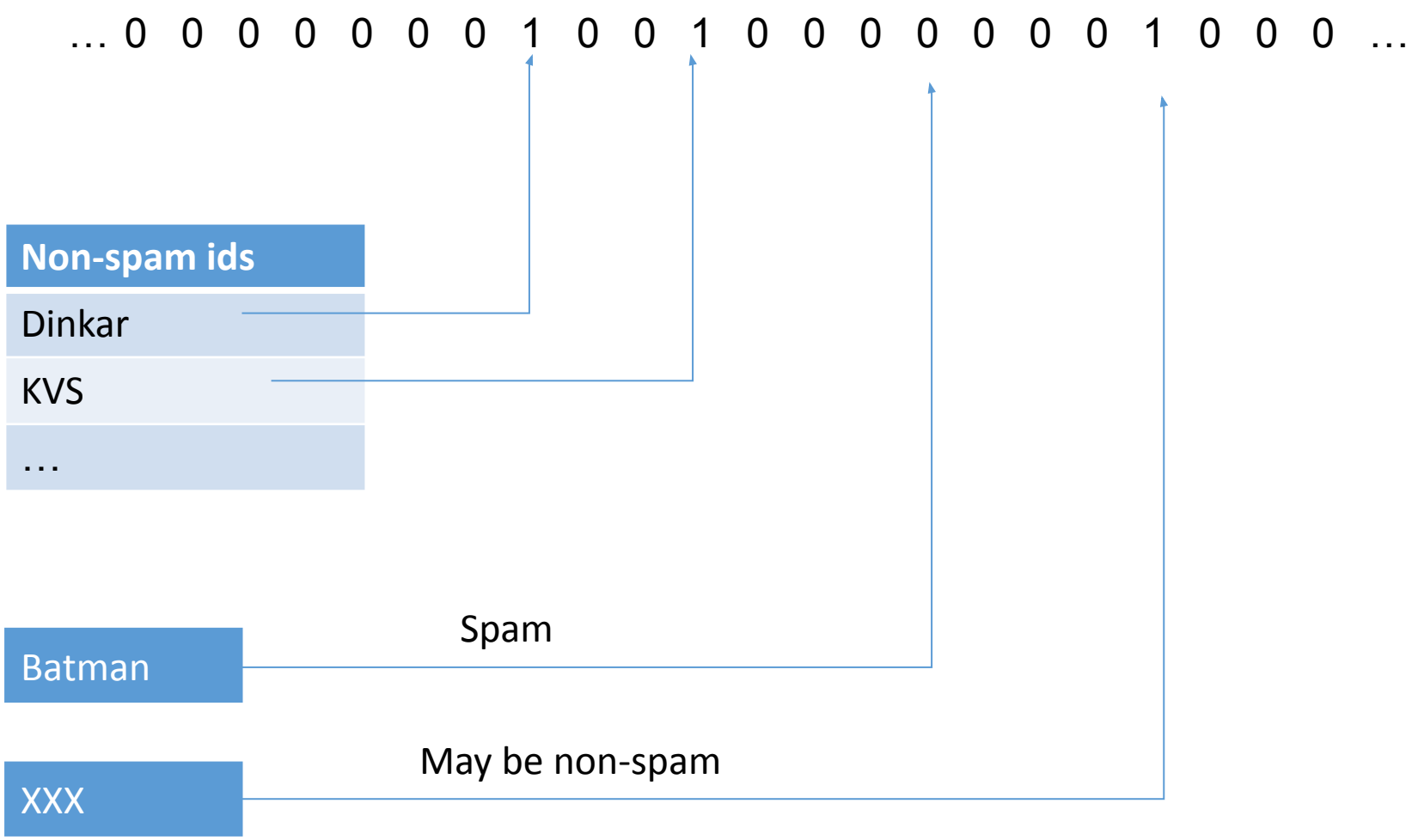


Bloom filter basic: initialization

- Usage
  - Hash incoming email id
  - Check bloom filter entry
    - If (0)
      - Definitely has not been seen before it is a spam
    - If (1)
      - Not sure if it has been seen before



Bloom filter basic: illustration



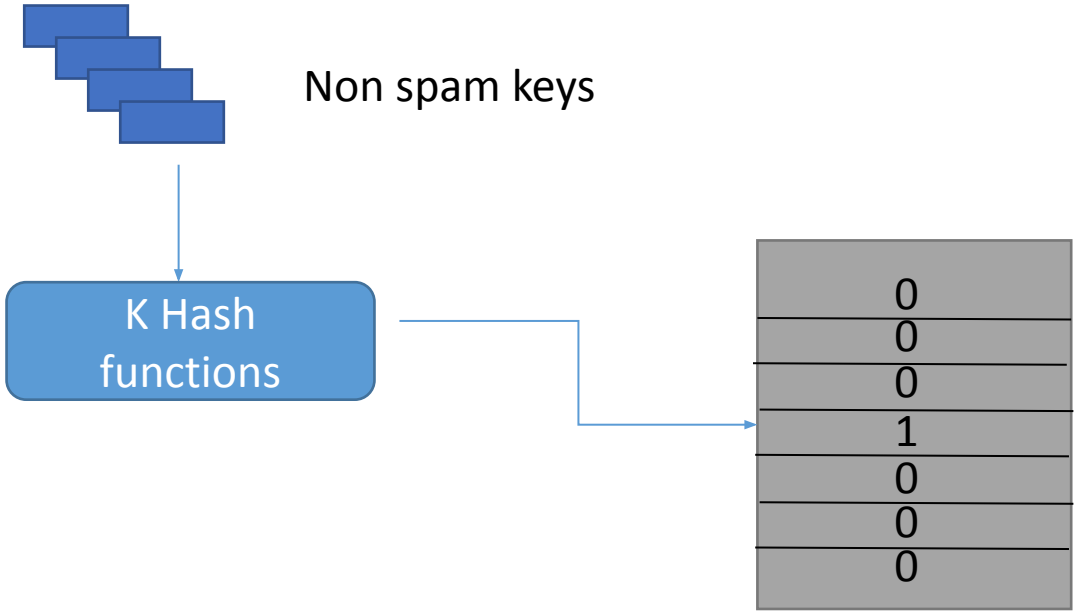
General Bloom Filters

General Bloom Filters

Bloom filter consists of

- Array of n bits (size of memory in example)
- A collection of k hash functions h1, ..., hk
- A set S of keys with m elements (non-spam email ids in example)
- Purpose: given a key a, determine if it is in S (in example, given email id, determine if non-spam)

- Initialization: for all keys in S,
  - Compute the k hash functions
  - Set corresponding bits to 1

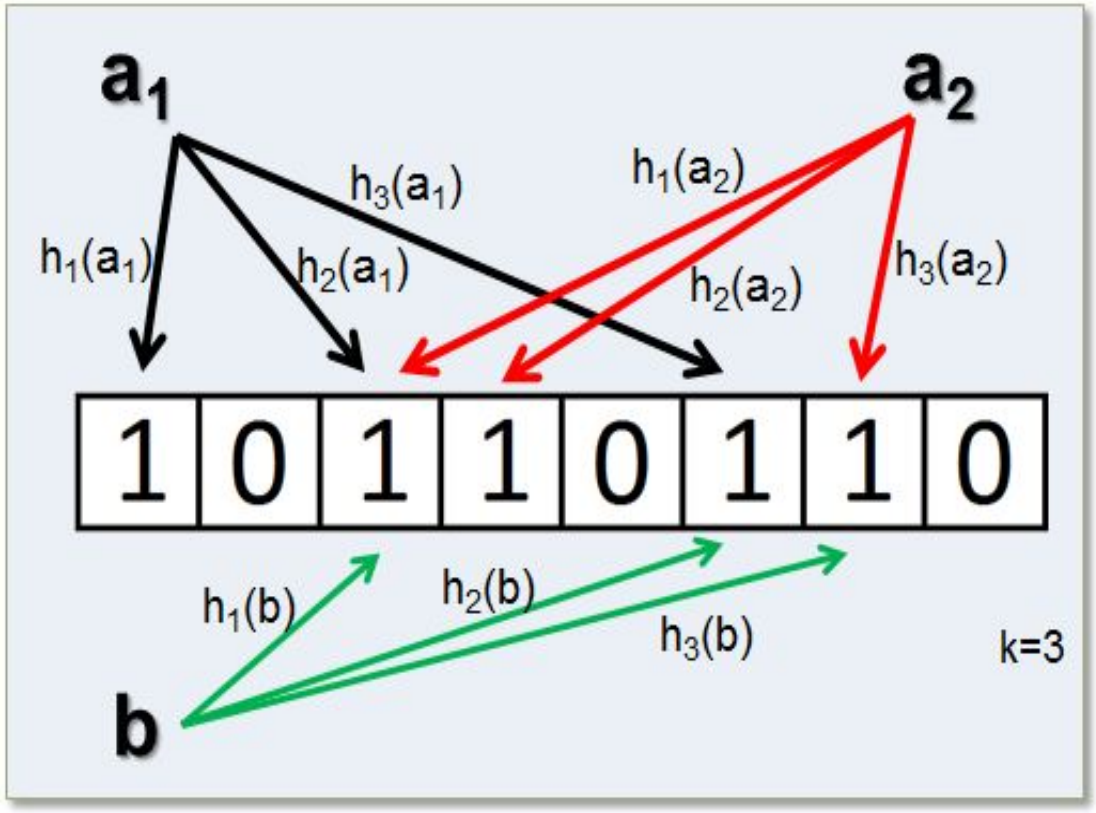


General Bloom Filters

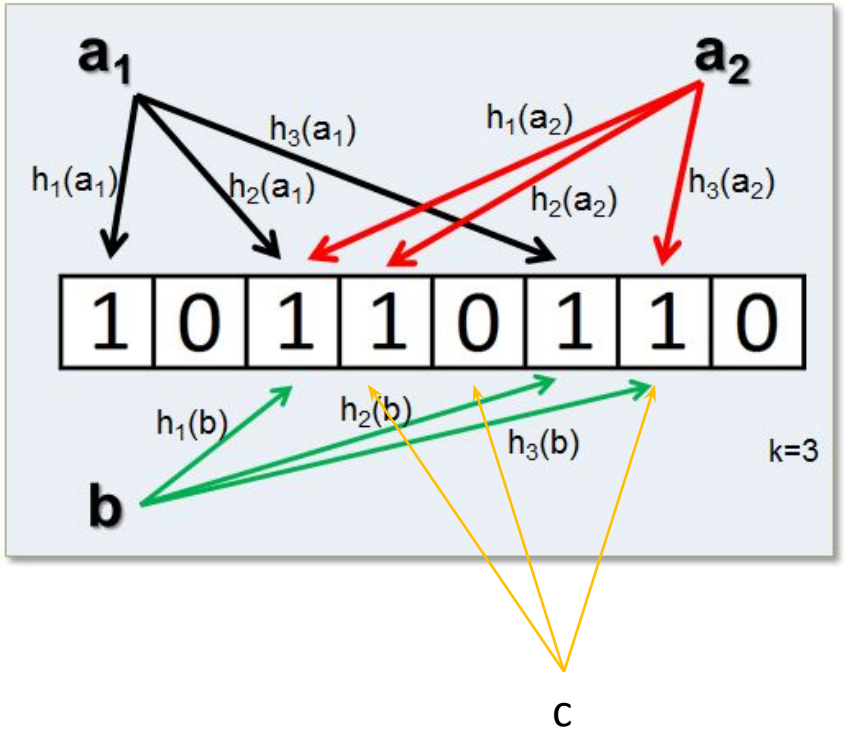
- Usage
  - Hash a using the k hash functions
  - If all the corresponding k bits are 1, a €S
- Probability of false positive $(1 - e^{-km/n})^k$
  - Read derivation in book

General Bloom Filters

- Top
  - Shows the insertion
- Bottom
  - Shows the check for set membership
  - B is checked to see if it is part of the bloom filter.

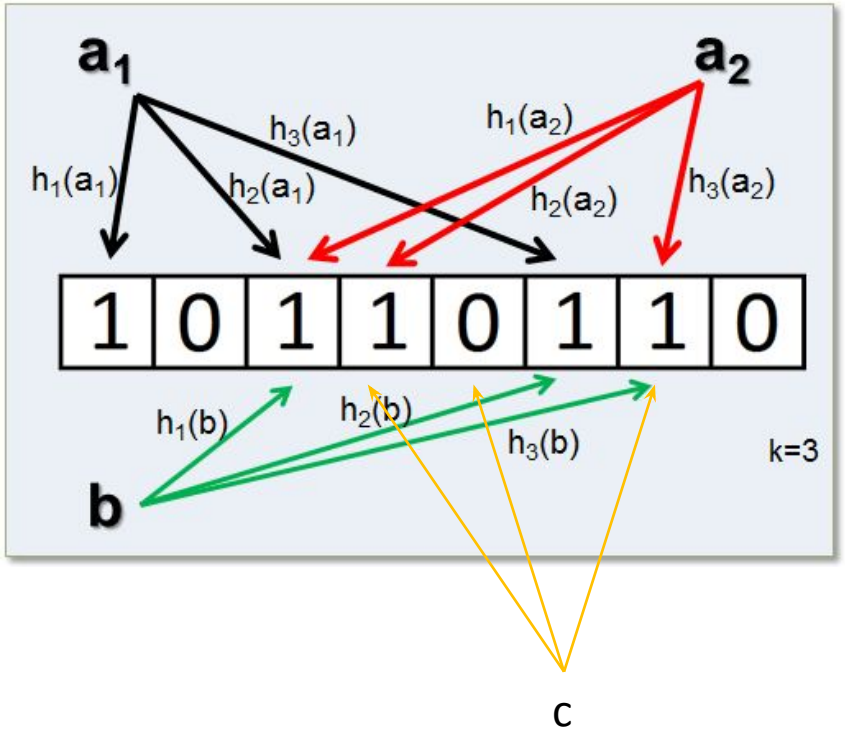


Exercise



- Suppose c hashes as shown
- Is c spam, not spam, or possibly spam?

Exercise - solution

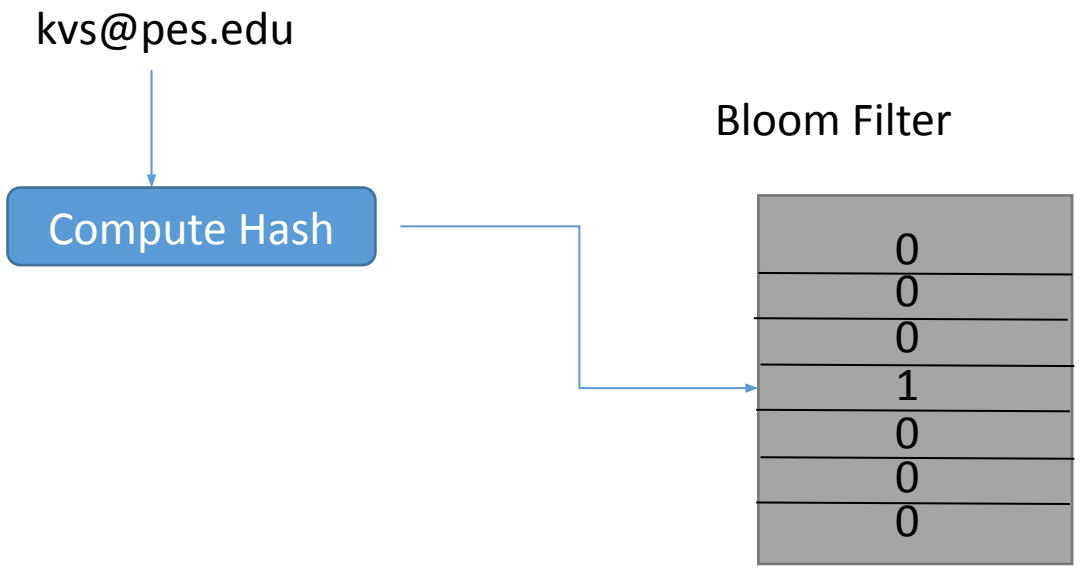


- Suppose c hashes as shown
- Is c spam, not spam, or possibly spam?
- C – is a spam email as it hashes to one bucket that contains a 0

Bloom filter extensions

Bloom filter basic: extensions

- Use secondary storage
  - 7/8 of the time, we can filter from memory
    - 7/8 of the time, a spam id will hash to 0
    - Because, there are 8B bits, 1B are 1, 7B are 0
  - 1/8 of the time, verify non-spam by disk lookup



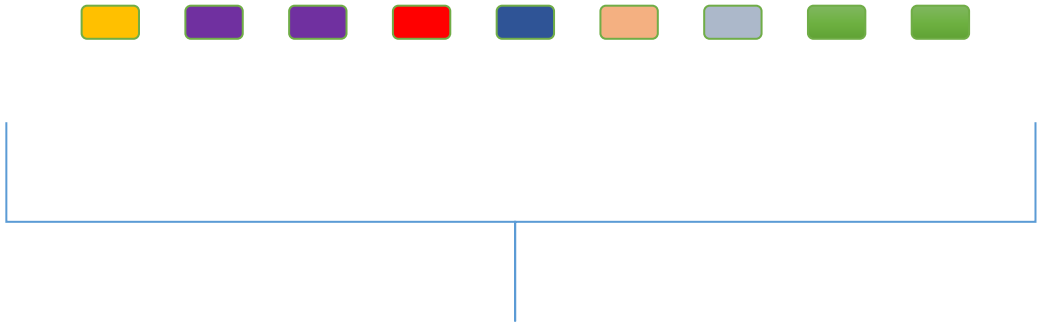
Cascaded Bloom filters

- We can have 2 Bloom filters in series with different hashes
- If bit is 1, use second Bloom filter
- We will reject much more of the spam

Counting Distinct Elements

Motivation

- Website that wants to know how many distinct users visit
  - Amazon: have userid
  - Google: have to use IP address
  - 4 billion IP addresses
- A more complex problem
  - How many different users visit each web page?
  - Users x Web pages combinations



How many distinct users are accessing the website?

Flajolet Martin Algorithm

- Pick hash function that is bigger than set to be hashed

- To count IP addresses: hash > 4 billion
- To count URLs: use 64 bits

Flajolet Martin Algorithm: basic property

- Tail length for hash function: number of 0's at the end of the hash for a given hash function
- Hash each element in stream
- Let R be the maximum tail length
- 2^R is approximately the number of distinct elements seen

11110100 has tail length 2

Exercise

Tail length for hash function: number of 0's at the end of the hash for a given hash function

11110100 has tail length 2

Hash each element in stream

Let R be the maximum tail length

2^R is approximately the number of distinct elements seen

- Suppose we want to count the number of userids that visit a Web page
  - Suppose userid is 0..15
- Mid square hash
  - Cube userid, make 12 bits, take middle 6 bits
  - Hint: powers of 2: 0 1 2 4 8 16 32 64 128 256 512 1024 2048 4096
- Suppose the userid sequence is 10 10 7 10 6 14 14 12 6 5 7

Exercise

Tail length for hash function: number of 0's at the end of the hash for a given hash function

11110100 has tail length 2

Let r be the tail length

What is the probability that r is the tail length?

PES UNIVERSITY
CELEBRATING 50 YEARS

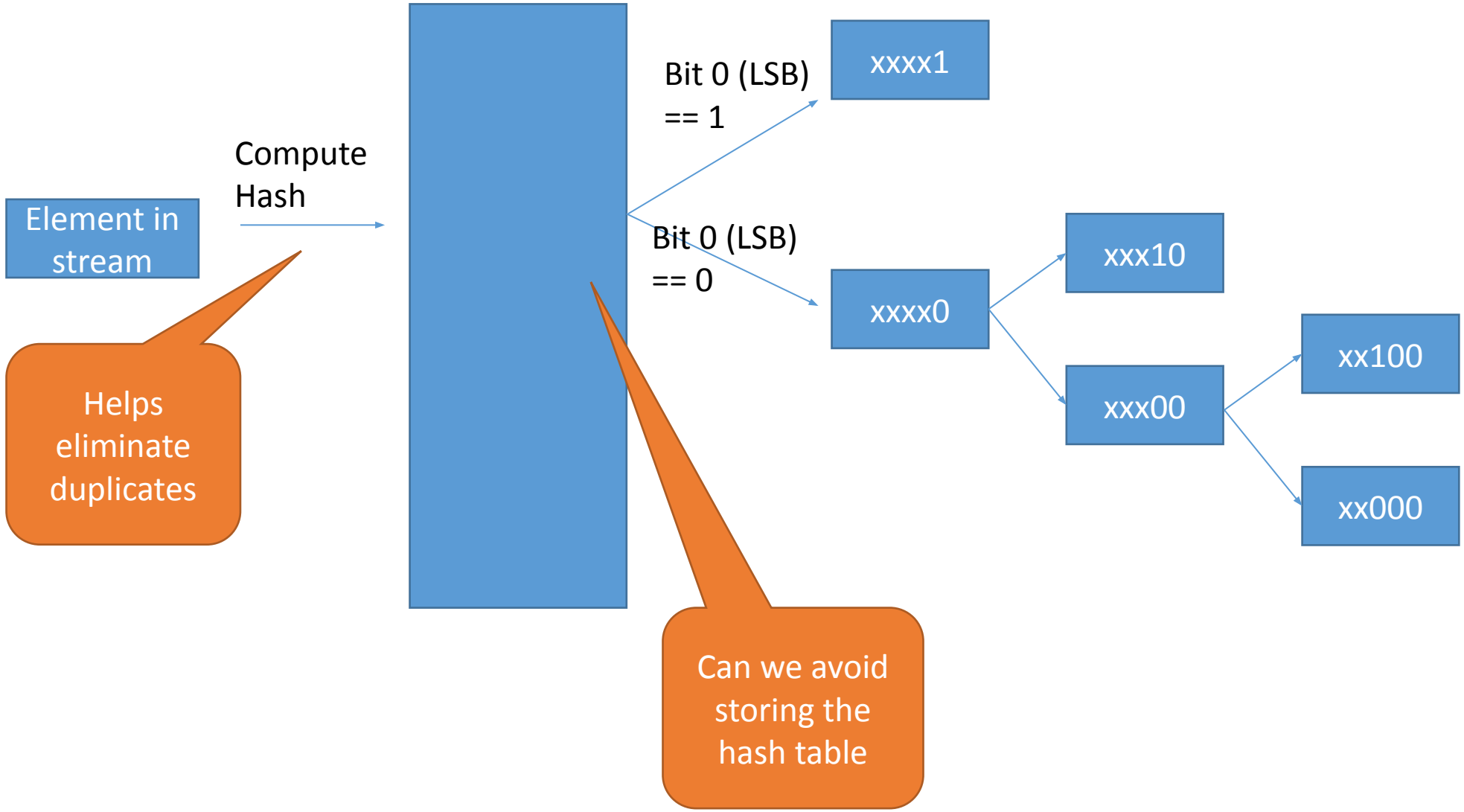
Flajolet Martin - intuitive

Working of the algorithm

- If we want to count all the distinct elements
  - We need to store all previous distinct elements
  - Compare the new incoming with all of them to see if it is new.
  - □ huge number of comparisons
    - Want to avoid this.

  - Is there a simpler way to compute? 2 step process
    - Check if unique
    - Estimate the #unique

Working of the algorithm



Flajolet Martin - working

Why does Flajolet Martin Algorithm work

- p ($h(a)$ ends in at least r 0's) = 2^{-r}
  - Suppose the hash is $h_1 h_2 \ldots h_n$
  - Probability any bit is 0 is ½
  - Probability h_n is 0 is ½ = 2^{-1}
  - Probability last two bits $h_{n-1} h_n$ are both 0 is (1/2) x (1/2) = 2^{-2}
  - Similarly, probability last r bits are all 0 is 2^{-r}

Exercise

p (*tail length is* r) = 2^{-r}

If there are m elements in the stream, what is the probability that none of them have tail length r?

Generalization of the algorithm

Suppose there are m distinct elements in the stream

p(no element has tail length r) = $(1 - 2^{-r})^m$

Suppose the elements (e.g., userids) are $u_1, u_2, \ldots, u_m$

p (u_1 has tail length r) = 2^{-r}

p (u_1 doesn't have tail length r) = $1-2^{-r}$

Similarly, p (u_2 doesn't have tail length r) = $1-2^{-r}$

p (u_1 and u_2 don't have tail length r) = $(1-2^{-r})(1-2^{-r})$

p ($u_1 \ldots u_m$ all don't have tail length r) = $(1-2^{-r})^m$

Recall

p ($h(a)$ ends in at least r 0's) = 2^{-r}

Suppose the hash is $h_1 h_2 \ldots h_n$

Probability any bit is 0 is ½

Probability h_n is o is ½ = 2^{-1}

Probability last two bits $h_{n-1} h_n$ are both 0 is (1/2) x (1/2) = 2^{-2}

Similarly, probability last r bits are all 0 is 2^{-r}

Exercise

p ($h(a)$ ends in at least r 0's) = 2^{-r}

Probability any bit is 0 is 1/2

Suppose there are m distinct elements in the stream

p(no element has tail length r) = $(1 - 2^{-r})^m$

p(no element has tail length r) = $(1 - 2^{-r})^m \sim e^{-mx}$
where $x=2^{-r}$ (See textbook)

p(at least one element has tail length r) = $1 - e^{-mx}$

Exercise

p(at least one element has tail length r) = $1 - e^{-mx}$

$mx = m2^{-r} = m/2^r$

What will p be if

$m >> 2^r$

$m \sim 2^r$

$m << 2^r$

Flajolet Martin – why the algorithm works

p(at least one element has tail length r) = $1 - e^{-mx}$

$mx = m2^{-r} = m/2^r$

There are 3 cases: $m>>2^r$, $m<<2^r$, $m\sim2^r$

$m>>2^r$

$mx = m2^{-r} = m/2^r$ will be very large

Therefore, $e^{-mx} \sim 0$

Therefore, p(at least one element has tail length r) = $1 - e^{-mx} \sim 1$

So we are likely to find tail lengths r where $m>>2^r$

$m\sim2^r$

$mx = m2^{-r} = m/2^r$ will a small number

Therefore, $e^{-mx} \sim$ will be some fraction

Therefore, p(at least one element has tail length r) = $1 - e^{-mx} \sim$ some fraction

So there is some probabilty to find tail lengths r where $m\sim2^r$

Flajolet Martin – why the algorithm works

p(at least one element has tail length r) = $1 - e^{-mx}$

$mx = m2^{-r} = m/2^r$

There are 3 cases: $m>>2^r$, $m\sim2^r$, $m<<2^r$

Suppose, $m>>2^r$

So we are likely to find tail lengths r where $m>>2^r$

Suppose, $m\sim2^r$

So there is some probabilty to find tail lengths r where $m\sim2^r$

Suppose, $m<<2^r$

$mx = m2^{-r} = m/2^r$ will be very small

Therefore, $e^{-mx} \sim 1$ since $e^0 = 1$

Therefore, p(at least one element has tail length r) = $1 - e^{-mx} \sim 0$

So we are not likely to find tail lengths r where $m<<2^r$

Flajolet Martin – summary

p(at least one element has tail length r) = $1 - e^{-mx}$

$mx = m2^{-r} = m/2^r$

There are 3 cases: $m>>2^r$, $m\sim2^r$, $m<<2^r$

Suppose, $m>>2^r$

We are likely to find tail lengths r where $m>>2^r$

Suppose, $m\sim2^r$

There is some probabilty to find tail lengths r where $m\sim2^r$

Suppose, $m<<2^r$

We are not likely to find tail lengths r where $m<<2^r$

- We are likely to find tail lengths r where $m>>2^r$ or $m\sim2^r$
- If we take the largest r, we must have $m\sim2^r$

Flajolet Martin – practical considerations

Flajolet Martin – in practise

- Simple approach
  - If we have only one hash function, *m* will always be a power of 2
  - We can pick *k* hash functions, estimate $m=2^R$ for each, take average or median
  - Will also give better estimate
- Problems with simple approach
  - Average will be pulled towards max (maybe outlier)
  - Median: estimate will always be power of 2

Flajolet Martin – in practise

Combined approach

- Divide *k* hashes into groups
  - E.g., if we have 6 hash functions $(h_1 \ldots h_6)$, we might have 3 groups where $g_1=(h_1\ h_2)$ and so on
- Compute average of each group
  - E.g.,estimate m_1 as average calculated from $g_1=(h_1\ h_2)$ and so on
- Then median of averages

References

- Big Data Analytics Beyond Hadoop: Real-Time Applications with Storm, Spark, and More Hadoop Alternatives, Vijay Srinivasa Agneeswaran
- **Refer chapter 4.1 and 4.2 from Mining of Massive datasets book**



THANK YOU

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu



Big Data

Apache Flink

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Significant information in the slide deck presented through the Unit 4 of the course have been created by **Dr. K V Subramaniam's** and would like to acknowledge and thank him for the same. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

BIG DATA

Lecture Overview

- Introduction to Apache Flink : What it is and why it's useful for big data.
- Flink vs. Other Big Data Tools :How it differs from Hadoop and Spark.
- Core Concepts in Flink : Streaming, batch processing, and APIs.
- Flink Architecture : Job Manager, Task Managers, and their roles.
- Key Features : Event time vs. processing time, windowing.
- Practical Examples and Use Cases : Real-world applications in IoT, finance, and more.
- Challenges and Best Practices : Common challenges in streaming and how to address them.
- Case Study ; Real-time sentiment analysis example.

What is Apache Flink ?



- Apache Flink is a framework for processing big data in real-time and batch modes.
- Why use Flink?

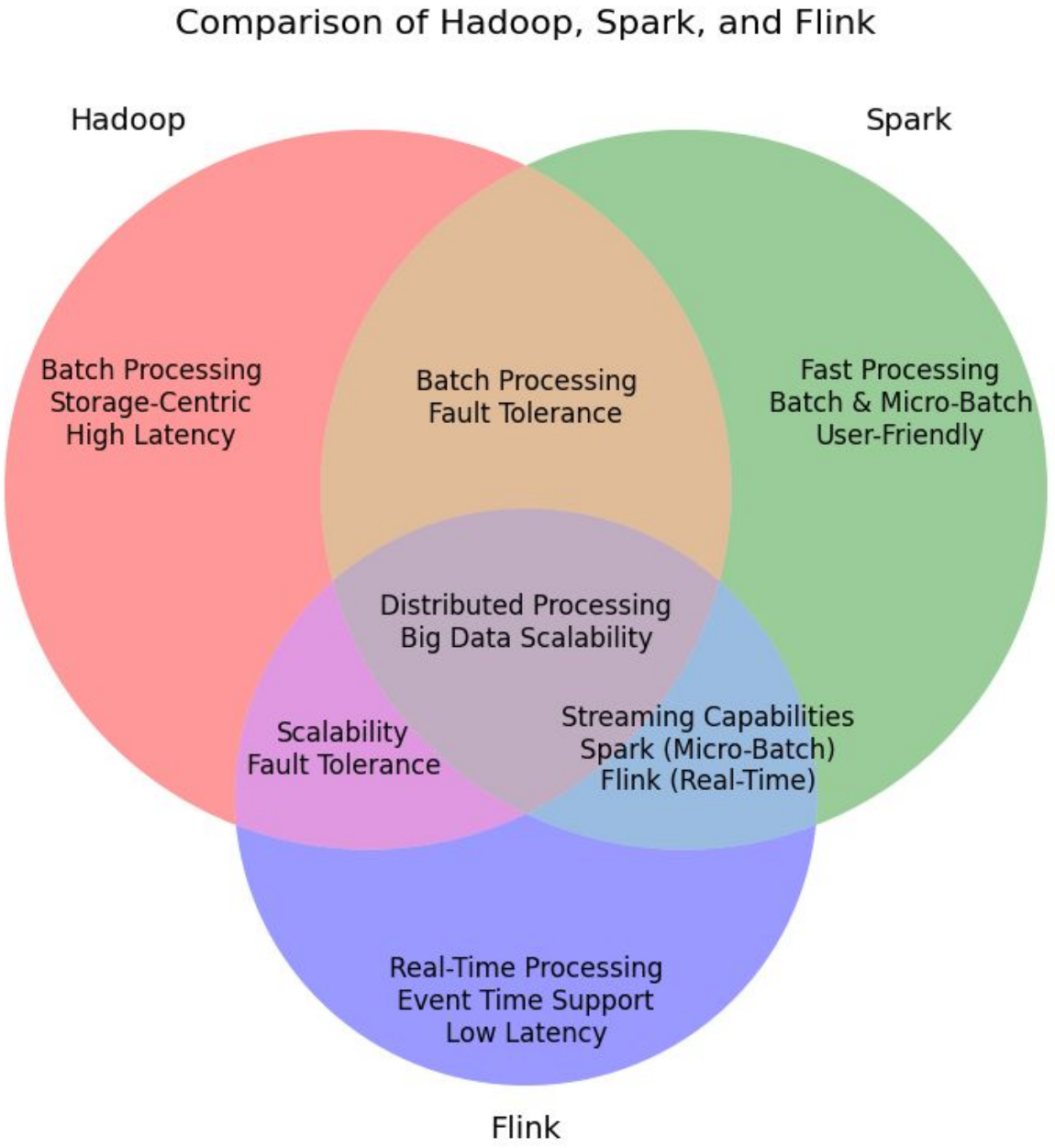
  Designed for low-latency, real-time data streaming.

  Handles both streaming and batch processing.
- Simple Use Case: Imagine a stock market app giving live price updates every second.

BIG DATA

Flink vs. Other Big Data Tools

- Flink vs. Other Tools
  - Hadoop: For batch processing.
  - Spark: For batch and micro-batch.
  - Flink: For low-latency real-time streaming.
- Main Takeaway: Use Flink when real-time processing is needed.



Key concepts in Flink

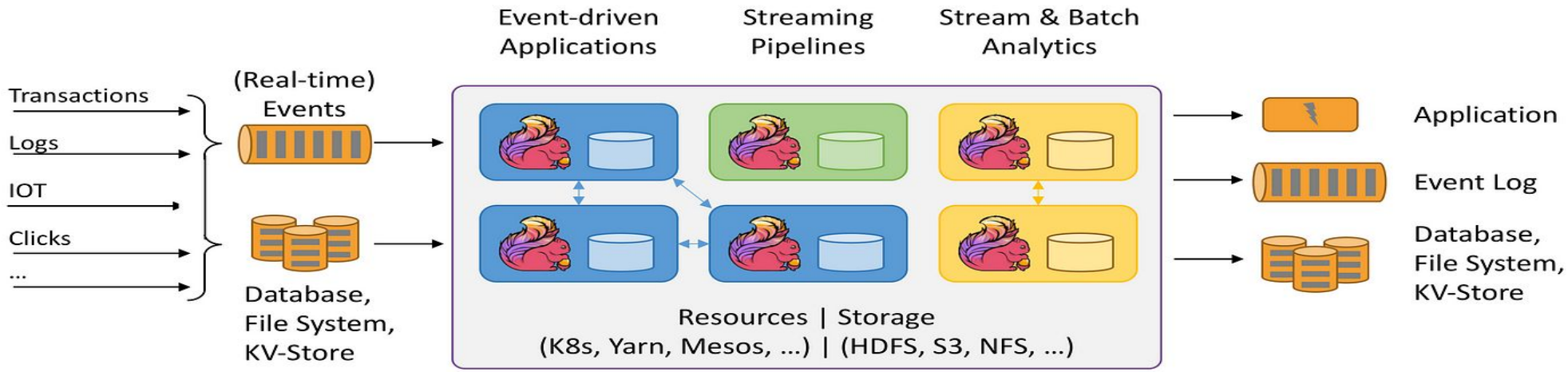
- Streaming: Continuous data flow (e.g., live tweets, sensor data).
- Batch Processing: Process static data (e.g., analyzing daily sales).
- APIs in Flink:
  - DataStream API: For streaming data.
  - DataSet API: For batch data.
- Example: Social media feed (streaming) vs. daily sales (batch).

- **Example Image**: Streaming Data Flow

BIG DATA

Flink Architecture (Simplified)

- Job Manager: The “planner” that organizes tasks.
- Task Managers: The “workers” that execute tasks.
- How It Works:
  - Job Manager breaks down the job.
  - Task Managers process data.
- Analogy: Job Manager as project manager; Task Managers as team members.



Important aspects of Flink's architecture

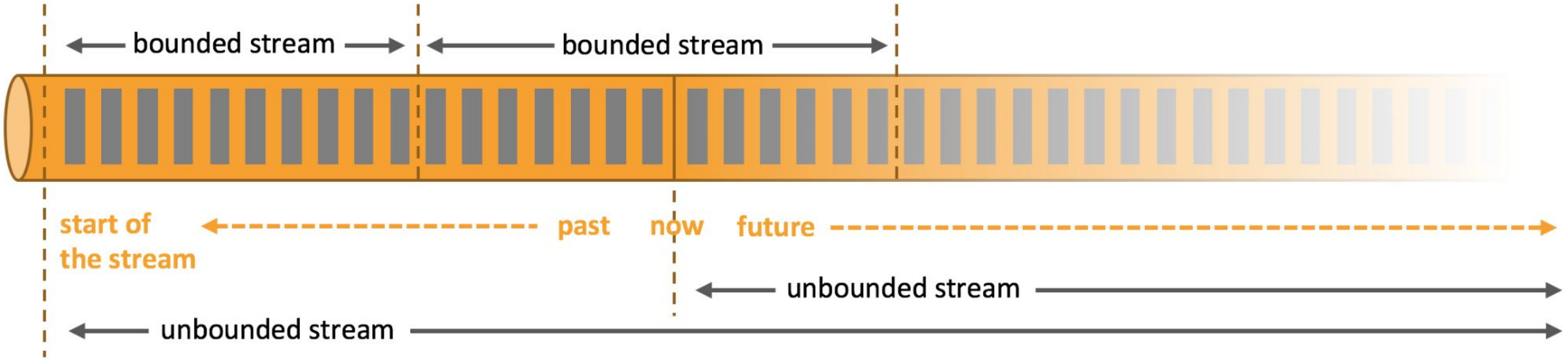
Some of the most important aspects of Flink's architecture are as follows:

1. Process Unbounded and Bounded Data
2. Deploy applications anywhere
3. Run applications at any scale
4. Leverage In-Memory performance

Process Unbounded and Bounded Data

Any kind of data is produced as a stream of events. Credit card transactions, sensor measurements, machine logs, or user interactions on a website or mobile application, all of these data are generated as a stream.

Data can be processed as unbounded or bounded streams.



Process Unbounded and Bounded Data

- **Unbounded streams** have a start but no defined end. They do not terminate and provide data as it is generated. Unbounded streams must be continuously processed, i.e., events must be promptly handled after they have been ingested. It is not possible to wait for all input data to arrive because the input is unbounded and will not be complete at any point in time. Processing unbounded data often requires that events are ingested in a specific order, such as the order in which events occurred, to be able to reason about result completeness.

- **Bounded streams** have a defined start and end. Bounded streams can be processed by ingesting all data before performing any computations. Ordered ingestion is not required to process bounded streams because a bounded data set can always be sorted. Processing of bounded streams is also known as batch processing.

Deploy applications anywhere



- Apache Flink is a distributed system and requires compute resources in order to execute applications. Flink integrates with all common cluster resource managers such as Hadoop YARN and Kubernetes but can also be setup to run as a stand-alone cluster.

- Flink is designed to work well with each of the previously listed resource managers. This is achieved by resource-manager-specific deployment modes that allow Flink to interact with each resource manager in its idiomatic way.

- When deploying a Flink application, Flink automatically identifies the required resources based on the application's configured parallelism and requests them from the resource manager. In case of a failure, Flink replaces the failed container by requesting new resources. All communication to submit or control an application happens via REST calls. This eases the integration of Flink in many environments.

Run applications at any scale



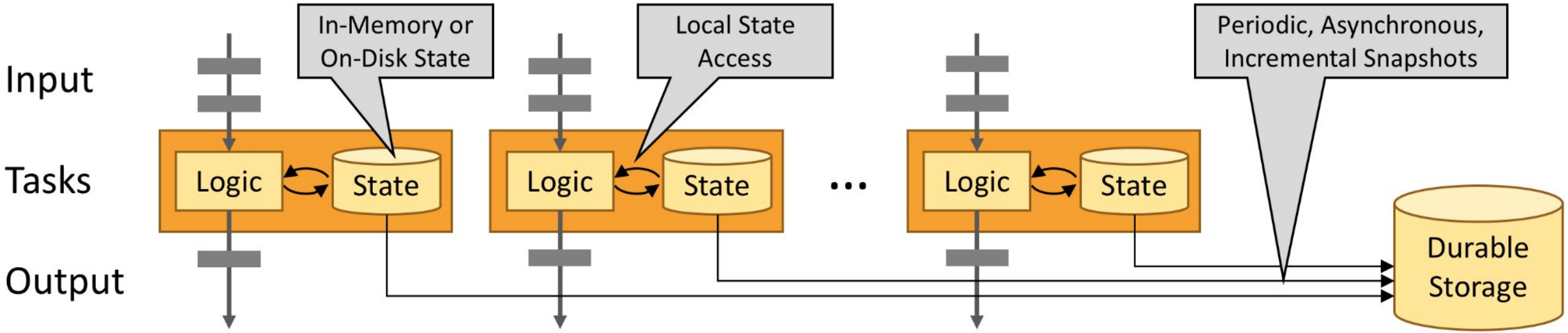
Flink is designed to run stateful streaming applications at any scale. Applications are parallelized into possibly thousands of tasks that are distributed and concurrently executed in a cluster. Therefore, an application can leverage virtually unlimited amounts of CPUs, main memory, disk and network IO. Moreover, Flink easily maintains very large application state. Its asynchronous and incremental checkpointing algorithm ensures minimal impact on processing latencies while guaranteeing exactly-once state consistency.

Users reported impressive scalability numbers for Flink applications running in their production environments, such as:

- applications processing **multiple trillions of events per day,**
- applications maintaining **multiple terabytes of state**, and
- applications **running on thousands of cores.**

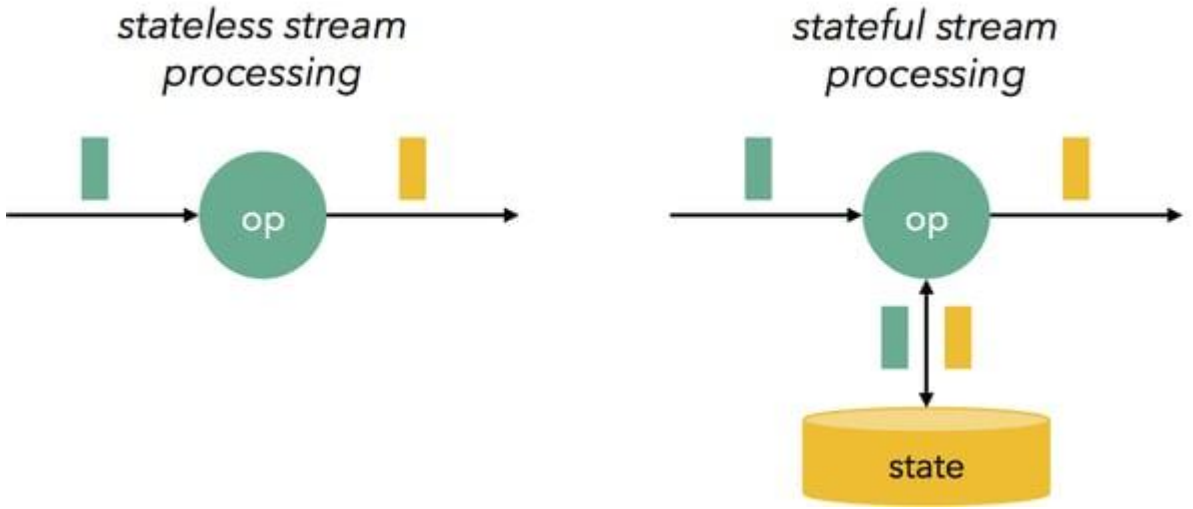
Leverage In-Memory Performance

- Stateful Flink applications are optimized for local state access. Task state is always maintained in memory or, if the state size exceeds the available memory, in access-efficient on-disk data structures. Hence, tasks perform all computations by accessing local, often in-memory, state yielding very low processing latencies. Flink guarantees exactly-once state consistency in case of failures by periodically and asynchronously checkpointing the local state to durable storage.



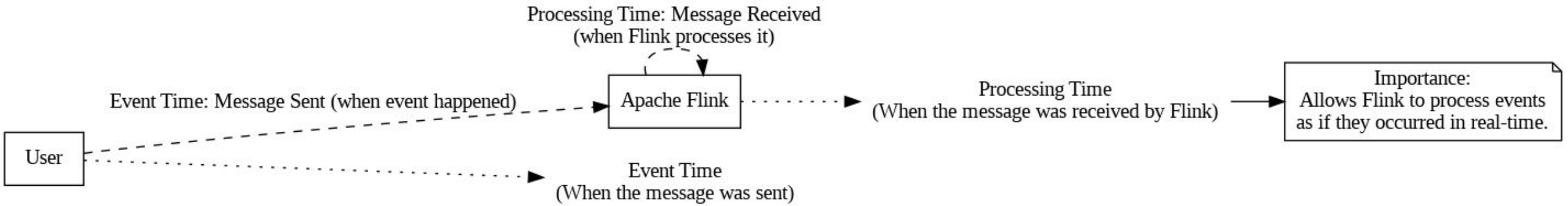
Stateful and Stateless Processing

- Stateless Processing: Processes each event independently (e.g., filtering tweets).
- Stateful Processing: Keeps track of past events (e.g., tracking user sessions).
- Why Stateful Matters: Helps make smarter decisions by remembering previous data.



Event Time vs. Processing Time

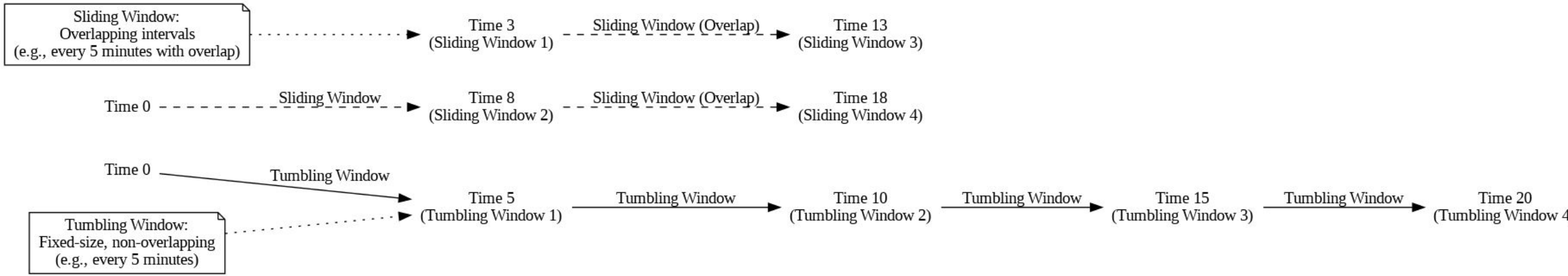
- Event Time: When the event happened.
- Processing Time: When Flink received and processed it.
- Example: Sending a message in chat (event time) vs. when it's received (processing time).
- Why Important?: Allows Flink to work as if events happened in real-time.



BIG DATA

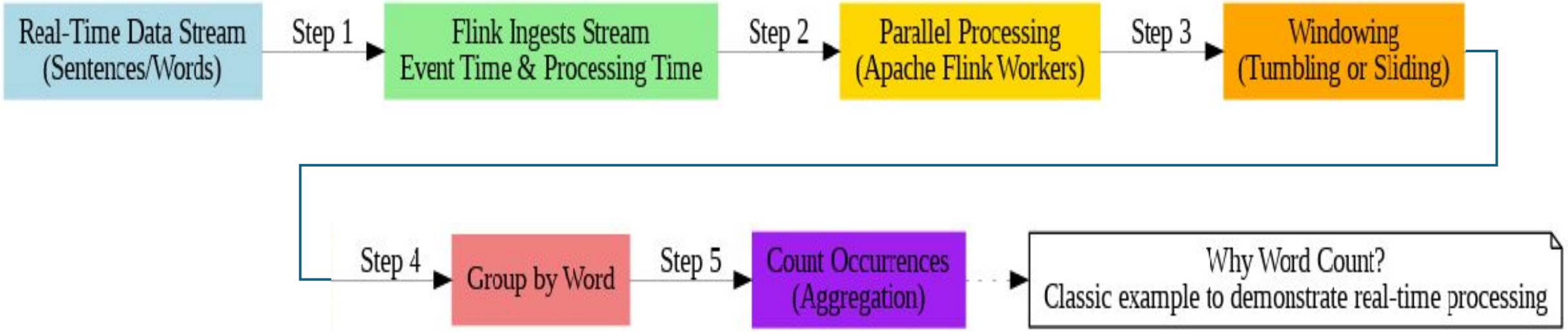
Windowing in Flink

- Windowing: Group data by time.
- Tumbling Window: Fixed-size (e.g., every 5 minutes).
- Sliding Window: Overlapping (e.g., every 5 minutes with overlap).
- Visual Example: A timeline to illustrate tumbling and sliding windows.



Simple Example - Word Count

- Goal: Count words in a continuous stream.
- Code Walkthrough:
  1. Read words from the stream.
  2. Group by word.
  3. Count occurrences.
- Why Word Count?: Classic example for demonstrating real-time processing.



Real-World Use Cases for Apache Flink

- Applications:
  1. IoT: Real-time sensor monitoring.
  2. Finance: Fraud detection.
  3. E-commerce: Product recommendations.
- Why Real-Time?: Immediate reaction to data changes.

Challenges and Best Practices

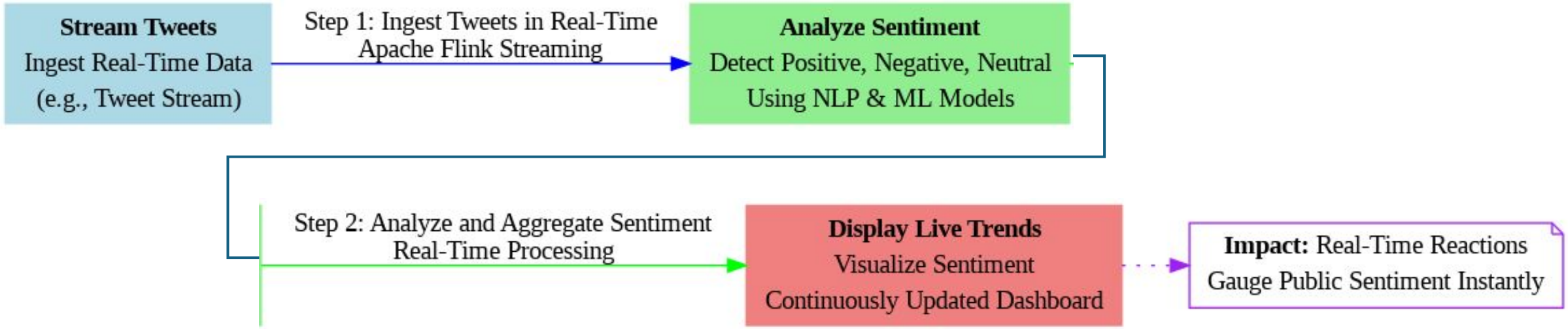
- Challenges:
  - Handling high volumes.
  - Ensuring low-latency.
  - Managing stateful operations.
- Best Practices:
  - Start with small datasets.
  - Test for scalability.
  - Keep code modular.

Challenges and Best Practices

- Challenges:
  - Handling high volumes.
  - Ensuring low-latency.
  - Managing stateful operations.
- Best Practices:
  - Start with small datasets.
  - Test for scalability.
  - Keep code modular.

Case Study - Real-Time Sentiment Analysis

- Use Case: Analyzing tweet sentiment in real-time.
- Steps:
  - Stream tweets.
  - Analyze sentiment.
  - Display live sentiment trends.
- Impact: Companies can gauge public reactions instantly.



Summary

- Apache Flink is ideal for low-latency, real-time data streaming.
- Key Concepts: Streaming vs. Batch, Event Time vs. Processing Time, Windowing.
- Use Cases: IoT, finance, e-commerce.
- Next Steps: Explore simple projects with Flink to understand its applications.

PES UNIVERSITY ONLINE

References

- Apache Flink Official Documentation
- Flink Streaming Concepts and APIs
- Real-Time Data Processing with Apache Flink
- Flink Architecture and Internals
- Machine Learning in Apache Flink
- Windowing and Time Semantics
- Sentiment Analysis with Apache Flink

PES UNIVERSITY ONLINE



THANK YOU

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu



Big Data

Spark GraphX

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

This slide deck on Spark GraphX. I have drawn on the prescribed textbook, the Apache Spark documentation and the research papers listed in the References, and would like to sincerely thank, acknowledge and reiterate that the credit/rights for that material remain with the original authors/publishers only. These are intended for classroom presentation only.

Lecture Overview

- Why graphs need special treatment in Big Data
- What is Spark GraphX and where it fits in the Spark stack
- The property graph: vertices, edges and triplets
- Graph operators and aggregateMessages
- Pregel: iterative graph computation, with a shortest-path example
- PageRank in GraphX, compared with the MapReduce version
- Built-in graph algorithms, and GraphX vs GraphFrames
- **Additional information (optional):** join operators, partitioning, internals, practical tips, GraphFrames in PySpark

Graphs are Everywhere in Big Data

Web graph

Vertices: web pages

Edges: hyperlinks

Question: which pages matter most?

Social network

Vertices: users

Edges: follows, friendships

Question: who is influential?

Payments

Vertices: accounts, merchants

Edges: transactions

Question: any suspicious cycles?

Road network

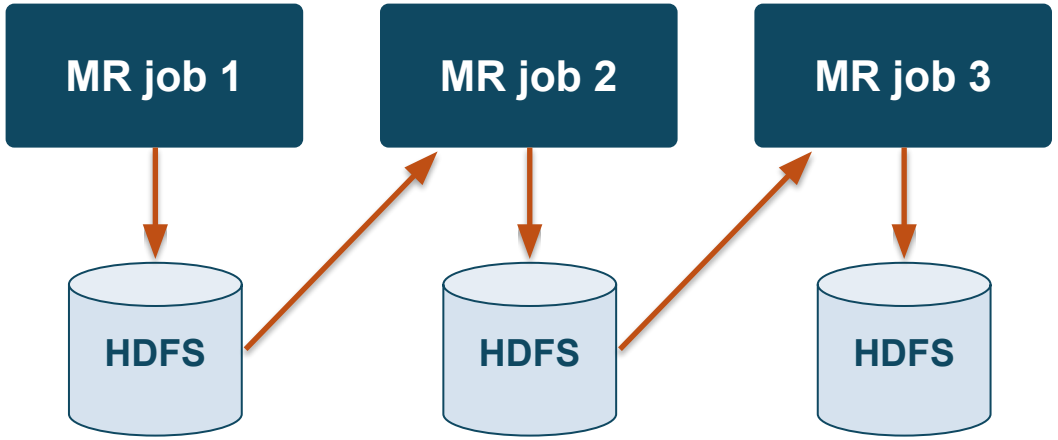
Vertices: junctions

Edges: roads with distances

Question: what is the shortest route?

The common thread: the answer for one entity depends on its neighbours, and on their neighbours, and so on. These problems are iterative and relationship-driven, which row-at-a-time processing handles poorly.

Recall: PageRank with MapReduce



write v(i) → read v(i) + the whole graph → next job

Graph algorithms are iterative, and each vertex only needs its neighbours' latest values.

What hurts

Every iteration is a full job: the whole graph is read from HDFS, shuffled and written back, even though the links never change.

The loop that compares v(i) with v(i+1) and decides when to stop is written by hand.

Only the importance values change between iterations. Ideally they are the only thing that moves.

Data-Parallel vs Graph-Parallel Processing

	Data-parallel	Graph-parallel
Unit of work	A record, or a partition of records	A vertex, using its neighbours' values
Typical systems	MapReduce, Spark RDDs and SQL	Pregel, GraphLab, Spark GraphX
Good at	ETL, joins, aggregation	PageRank, components, shortest paths
Iteration	Each pass is a new job over all the data	Supersteps over a graph kept in memory

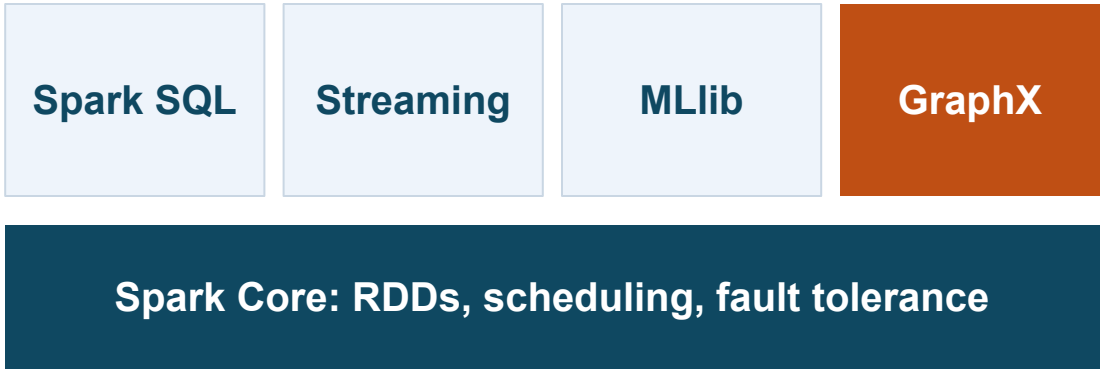
The gap GraphX fills

Real pipelines mix both: clean tables, build a graph, run an algorithm, then join results back to tables. GraphX keeps both views of the same data inside Spark, so nothing is copied between systems.

Source: Gonzalez et al., GraphX, OSDI 2014

What is Spark GraphX?

Spark's library for graphs and graph-parallel computation.



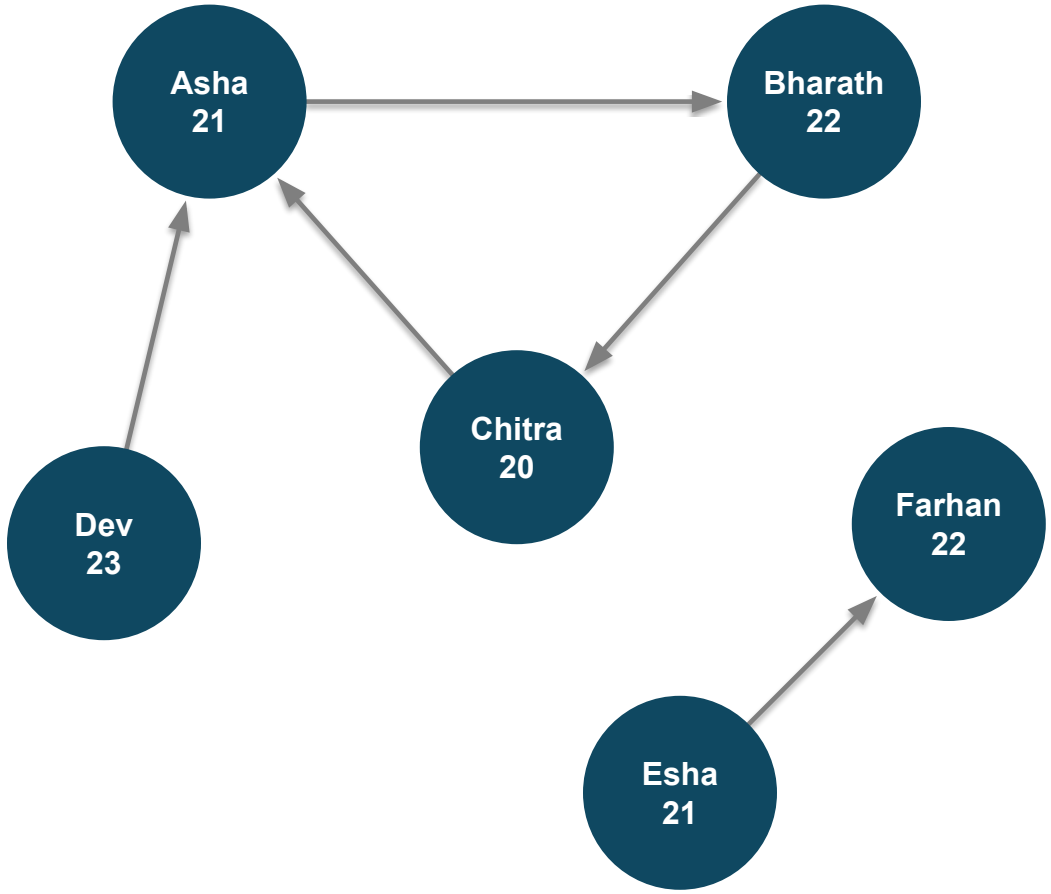
Cluster managers: Standalone, YARN, Kubernetes

- A graph is stored as two RDDs: one for vertices, one for edges.
- The same data can be used as a graph or as ordinary collections, without copying.
- Graph operations are built from normal Spark operations (join, map, group-by), so GraphX gets Spark's fault tolerance for free.
- API in Scala and Java. For Python, use GraphFrames (covered later).

Source: Apache Spark GraphX Programming Guide

The Property Graph Model

- **Directed multigraph** with user-defined properties on every vertex and every edge.
- Each vertex has a unique `VertexId` (a 64-bit Long).
- Two vertices can be joined by more than one edge, e.g. *friend* and *co-worker*.
- **Immutable and distributed,** like RDDs. Changing a graph creates a new graph.



Every edge property: "follows"

Type of this graph: `Graph[(String, Int), String]`

Source: Raj Kamal & Saxena, Big Data Analytics, Ch. 8; Apache Spark GraphX Programming Guide

One Graph, Three Views

`graph.vertices`

id	name	age
1	Asha	21
2	Bharath	22
3	Chitra	20
4	Dev	23
5	Esha	21
6	Farhan	22

`graph.edges`

srcId	dstId	attr
1	2	follows
2	3	follows
3	1	follows
4	1	follows
5	6	follows

`graph.triplets`

srcAttr	attr	dstAttr
(Asha, 21)	follows	(Bharath, 22)
(Bharath, 22)	follows	(Chitra, 20)
(Chitra, 20)	follows	(Asha, 21)
(Dev, 23)	follows	(Asha, 21)
(Esha, 21)	follows	(Farhan, 22)

A triplet is an edge joined with both of its end vertices. Logically, it is this SQL join:

```
SELECT src.id, dst.id, src.attr, e.attr, dst.attr
FROM edges AS e JOIN vertices AS src, vertices AS dst
ON e.srcId = src.id AND e.dstId = dst.id
```

Source: Apache Spark GraphX Programming Guide

Building a Graph from RDDs

Hands-on: Demo 1

```
import org.apache.spark.graphx._
import org.apache.spark.rdd.RDD

val users: RDD[(VertexId, (String, Int))] = sc.parallelize(Seq(
  (1L, ("Asha", 21)), (2L, ("Bharath", 22)), (3L, ("Chitra", 20)),
  (4L, ("Dev", 23)), (5L, ("Esha", 21)), (6L, ("Farhan", 22))))

val follows: RDD[Edge[String]] = sc.parallelize(Seq(
  Edge(1L, 2L, "follows"), Edge(2L, 3L, "follows"),
  Edge(3L, 1L, "follows"), Edge(4L, 1L, "follows"),
  Edge(5L, 6L, "follows")))

val g = Graph(users, follows)

g.triplets.map(t =>
  s"${t.srcAttr._1} ${t.attr} ${t.dstAttr._1}").collect()
```

Output

```
Asha follows Bharath
Bharath follows Chitra
Chitra follows Asha
Dev follows Asha
Esha follows Farhan
```

Other ways to build a graph

Graph.fromEdges(edges, default)

GraphLoader.edgeListFile(sc, path) reads a text file of "src dst" lines (used in Demo 5)

Graph Operators at a Glance

Group	Operators	What they do
Information	`numVertices, numEdges, inDegrees, outDegrees`	Basic statistics about the graph
Structural	`subgraph, reverse`	Choose which vertices and edges to keep
Neighbourhood	`aggregateMessages`	Collect information from neighbours
Iterative	`pregel`	Repeat a computation until nothing changes
Algorithms	`staticPageRank, connectedComponents, ...`	Ready-made graph analytics

Property operators (mapVertices, mapEdges) and **join operators** (joinVertices, outerJoinVertices) are covered in Additional Information.

Degrees, Filtering and Subgraphs

Hands-on: Demo 2

Who follows whom, and filtering by a property

```
g.inDegrees.collect()

g.vertices.filter { case (id, (name, age)) =>
  age >= 22 }.map(_._2._1).collect()
```

Output

```
In-degrees: (1,2) (2,1) (3,1) (6,1)
Age 22+: Bharath, Dev, Farhan
```

Asha has 2 followers. Dev and Esha have none, so they do not appear in inDegrees.

Keep only users aged 21 or more

```
val adults = g.subgraph(
  vpred = (id, attr) => attr._2 >= 21)

adults.triplets.map(t =>
  s"${t.srcAttr._1} ${t.attr}
${t.dstAttr._1}").collect()
```

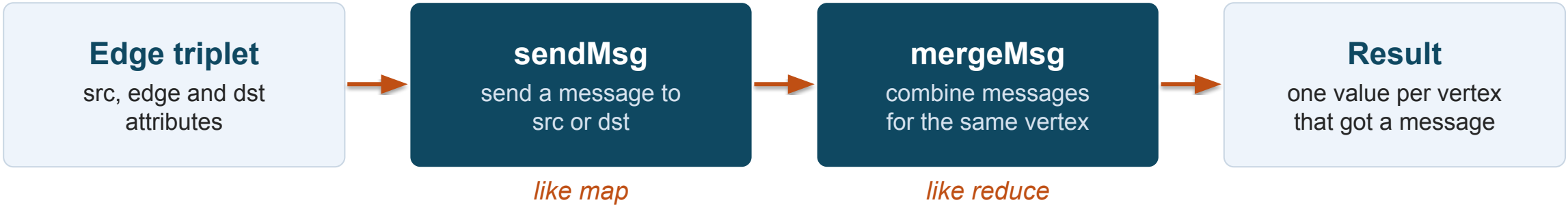
Output

```
5 vertices, 3 edges
Asha follows Bharath
Dev follows Asha
Esha follows Farhan
```

Chitra (20) is removed, and both edges touching her are removed with her.

aggregateMessages: Map-Reduce Along the Edges

Hands-on: Demo 3



Example: how many followers does each user have?

```
val followers = g.aggregateMessages[Int](
  ctx => ctx.sendToDst(1),
  (a, b) => a + b)

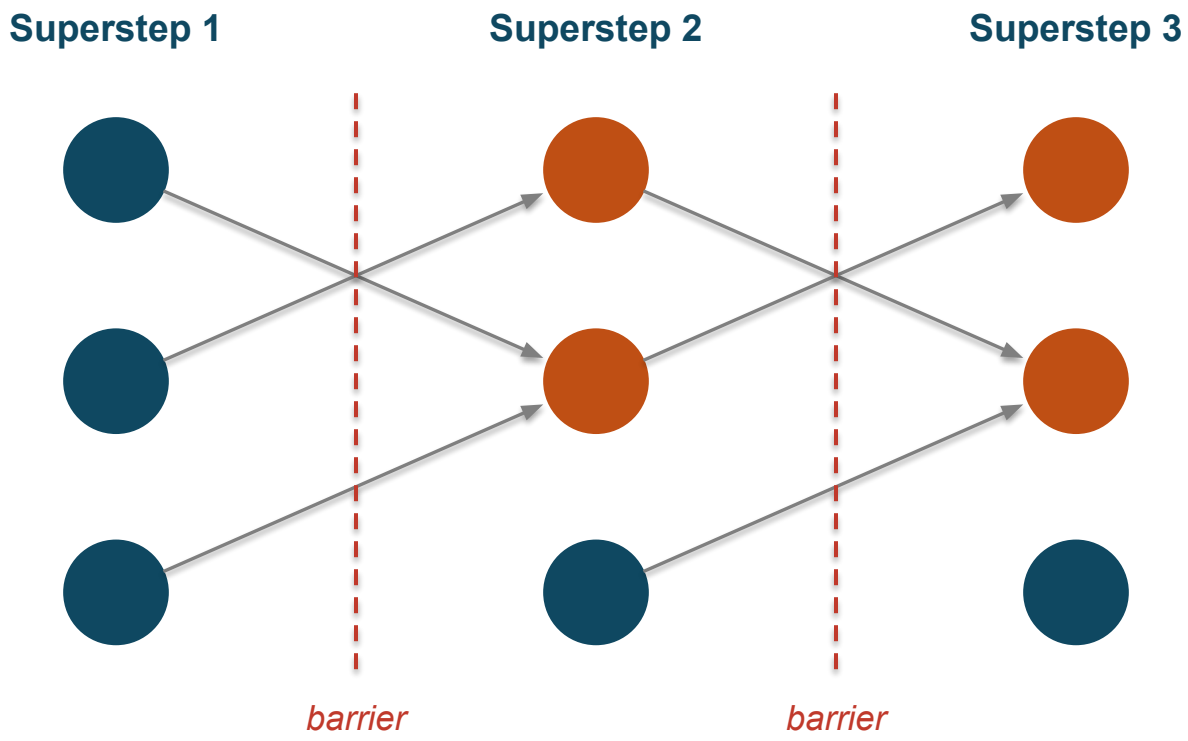
followers.collect()
```

Output

```
(1,2)  (2,1)  (3,1)  (6,1)
```

- Every edge sends 1 to the user being followed; the 1s are added up.
- Dev and Esha receive no message, so they have no row in the result.

Pregel: "Think Like a Vertex"



Arrows are messages. Orange vertices received messages and updated their value.

- Google's Pregel (2010) runs a graph algorithm as a series of supersteps.
- In each superstep, every vertex that received messages updates its value and sends new messages to its neighbours.
- A barrier separates supersteps: everyone finishes before the next step starts.
- The computation stops when no messages are left.

Source: Malewicz et al., Pregel, SIGMOD 2010

BIG DATA

The Pregel Operator in GraphX

```
graph.pregel(initialMsg, maxIterations, activeDirection)(vprog, sendMsg, mergeMsg)
```

Function	Type	What it does
vprog	`(VertexId, VD, A) => VD`	Update a vertex using its current value and the incoming message
sendMsg	`EdgeTriplet[VD, ED] => Iterator[(VertexId, A)]`	Look at one edge and decide which neighbour gets what message
mergeMsg	`(A, A) => A`	Combine two messages going to the same vertex

Difference from Google's Pregel: in GraphX, messages are computed per edge (both end vertices are visible) and can only go to neighbours. The loop ends when no messages remain or maxIterations is reached.

Source: Apache Spark GraphX Programming Guide

Example: Single-Source Shortest Path

Hands-on: Demo 4

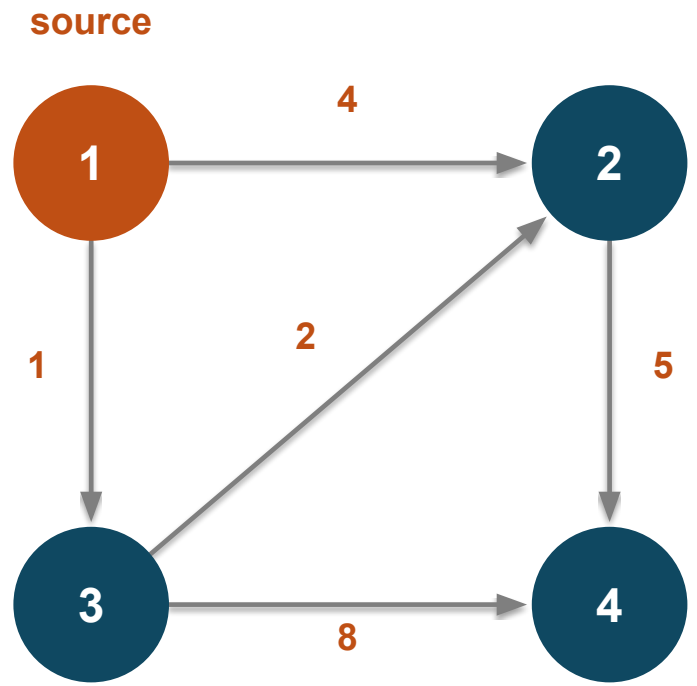
```
val roads = sc.parallelize(Seq(
  Edge(1L, 2L, 4.0), Edge(1L, 3L, 1.0), Edge(3L, 2L, 2.0),
  Edge(2L, 4L, 5.0), Edge(3L, 4L, 8.0)))
val init = Graph.fromEdges(roads, 0.0).mapVertices(
  (id, _) => if (id == 1L) 0.0 else Double.PositiveInfinity)

def vprog(id: VertexId, dist: Double, newDist: Double): Double =
  math.min(dist, newDist)
def sendMsg(t: EdgeTriplet[Double, Double]): Iterator[(VertexId, Double)] = {
  if (t.srcAttr + t.attr < t.dstAttr) Iterator((t.dstId, t.srcAttr + t.attr))
  else Iterator.empty
}
def mergeMsg(a: Double, b: Double): Double = math.min(a, b)

val sssp = init.pregel(Double.PositiveInfinity)(vprog, sendMsg, mergeMsg)
```

Output

```
(1,0.0) (2,3.0) (3,1.0) (4,8.0)
```



Best route to 4 is 1 → 3 → 2 → 4 = 1 + 2 + 5 = 8, not 1 → 3 → 4 = 9.

Shortest Path, Superstep by Superstep

Superstep	Messages delivered	d(1)	d(2)	d(3)	d(4)
Start	none	0	∞	∞	∞
1	2 gets 4, 3 gets 1	0	4	1	∞
2	2 gets 3 (via vertex 3), 4 gets 9	0	3	1	9
3	4 gets 3 + 5 = 8	0	3	1	8
4	no message improves any distance: stop	0	3	1	8

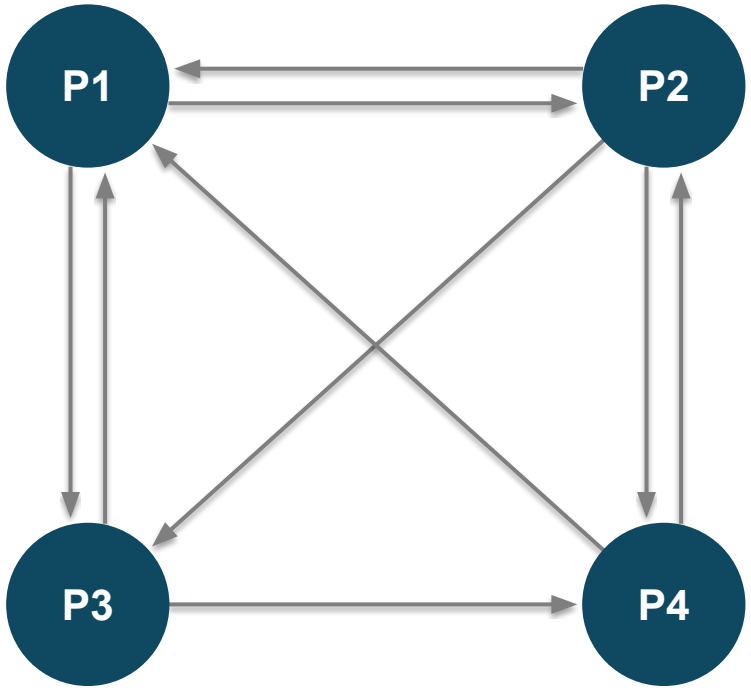
Why vertex 2 changes twice

First it hears 4 from vertex 1. One superstep later, vertex 3 (now at 1) offers 1 + 2 = 3, which is smaller.

Why it stops

Vertex 4 has no outgoing edges, so after superstep 3 nobody has a better distance to send.

PageRank Recap: The Random Surfer



Each page shares its importance equally among its out-links.

Transition matrix M (column = from, row = to)

	P1	P2	P3	P4
P1	0	1/3	1/2	1/2
P2	1/2	0	0	1/2
P3	1/2	1/3	0	0
P4	0	1/3	1/2	0

Start: `v0 = [1/4, 1/4, 1/4, 1/4]`

One move: `v1 = M · v0 = [1/3, 1/4, 5/24, 5/24]`

Repeat `v(i+1) = M · v(i)` until v stops changing, i.e. v = M · v. That v is the PageRank.

Source: PageRank lecture, Big Data course (PES University)

BIG DATA

PageRank in GraphX

Hands-on: Demo 5

```
val web = GraphLoader.edgeListFile(
  sc, "web_graph.txt").cache()

web.staticPageRank(1, 0.0).vertices.collect()
web.staticPageRank(2, 0.0).vertices.collect()
web.staticPageRank(50, 0.0).vertices.collect()
```

```
1 2
1 3
2 1
2 3
2 4
3 1
3 4
4 1
4 2
```

web_graph.txt

One link per line: source page, then destination page. Same 4-page graph as the lecture.

Page	1 iteration	2 iterations	50 iterations
P1	0.3333 (1/3)	0.2917 (7/24)	0.3051
P2	0.2500 (1/4)	0.2708 (13/48)	0.2542
P3	0.2083 (5/24)	0.2500 (1/4)	0.2373
P4	0.2083 (5/24)	0.1875 (3/16)	0.2034

Reading the code

staticPageRank(n, 0.0) runs n iterations of $v(i+1) = M \cdot v(i)$.

GraphX starts every page at 1 instead of 1/4, so its values are 4 times larger. The table shows them divided by 4.

PageRank: MapReduce vs GraphX

	Hadoop MapReduce	Spark GraphX
One iteration	One full MapReduce job	One step in memory
Graph structure	Re-read from HDFS every iteration	Loaded and cached once; only values change
Stopping rule	Hand-written loop compares v(i) and v(i+1)	Built in: staticPageRank(n, ...)
Code size	Mapper + reducer + driver script	A few lines (previous slide)
Fault tolerance	Files in HDFS between jobs	RDD lineage, plus optional checkpoints

The maths is identical. What changes is where the data lives between iterations, and who writes the loop.

Built-in Algorithms and Connected Components

Hands-on: Demo 6

In org.apache.spark.graphx.lib

- **PageRank:** importance of each vertex
- **Connected Components:** which vertices are linked at all
- **Strongly Connected Components:** groups reachable both ways
- **Triangle Count:** how tightly knit a neighbourhood is
- **Label Propagation:** fast community detection
- **Shortest Paths:** hop distance to landmark vertices
- **SVD++:** recommendations

Example: find groups of connected users

```
g.connectedComponents().vertices.collect()
```

Output

```
(1,1) (2,1) (3,1) (4,1) (5,5) (6,5)
Group 1: Asha, Bharath, Chitra, Dev
Group 5: Esha, Farhan
```

- Each user is labelled with the smallest id in their group.
- Edge direction is ignored: Dev follows Asha, so Dev joins her group.

GraphX vs GraphFrames

	GraphX	GraphFrames
Built on	RDDs	DataFrames
Languages	Scala, Java	Python, Scala, Java
Vertex ids	64-bit Long only	Any type, in an id column
Extra queries	Operators, Pregel	Motif (pattern) search, SQL
Comes with	Spark itself	Separate package

Status today

In late 2024, Spark developers voted to deprecate GraphX. Whether GraphFrames will replace it is still being discussed.

GraphX still ships with Spark 4.2.

Learn GraphX for the concepts. Use GraphFrames when working in PySpark (see the last Additional Information slide).

Source: Apache Spark developer mailing list (2024); graphframes.io

Summary

- GraphX keeps graph and table views of the same data inside Spark.
- A property graph is a vertex RDD plus an edge RDD; a triplet is an edge joined with its two end vertices.
- aggregateMessages is map-reduce along edges. Pregel repeats it in supersteps until no messages remain.
- PageRank from the lecture takes a few lines in GraphX, and the graph stays in memory instead of going back and forth to HDFS.
- GraphX includes ready-made algorithms: PageRank, connected components, triangle count and more.
- For PySpark, GraphFrames offers the same ideas on DataFrames.

BIG DATA

Practice Questions

1. Explain the property graph model in GraphX. What are the vertex, edge and triplet views?
2. What does subgraph do with an edge when one of its end vertices fails the vertex predicate? Use the follows graph as an example.
3. Using aggregateMessages, count how many users each user follows, without using outDegrees.
4. For the road graph, trace the Pregel supersteps if the edge 3 → 2 had weight 4 instead of 2.
5. For the four-page web graph, compute v2 = M · v1 by hand and check it against staticPageRank(2, 0.0).
6. Compare the MapReduce and GraphX implementations of PageRank in terms of disk I/O per iteration and how stopping is decided.
7. When would you choose GraphFrames over GraphX?

Additional Information

Optional slides

These go deeper into GraphX than the prescribed syllabus.

- Property and join operators
- Computing averages with aggregateMessages
- How GraphX splits a graph across machines (vertex-cut)
- How a graph is stored inside GraphX
- Practical tips for running GraphX
- GraphFrames in PySpark (Demo 7)

Property and Join Operators

Property operators (mapVertices, mapEdges, mapTriplets) change attributes but keep the structure, so GraphX can reuse its internal indices. Prefer graph.mapVertices(...) over building a new Graph from graph.vertices.map(...).

```
val withOutDeg = g.outerJoinVertices(g.outDegrees) {
  (id, attr, degOpt) => (attr._1, degOpt.getOrElse(0))
}
withOutDeg.vertices.collect()
```

Output

```
(1,(Asha,1)) (2,(Bharath,1)) (3,(Chitra,1))
(4,(Dev,1)) (5,(Esha,1)) (6,(Farhan,0))
```

Farhan follows nobody, so outDegrees has no entry for him. degOpt is None and getOrElse(0) fills the gap.

	joinVertices	outerJoinVertices
Applied to	Only vertices with a match	Every vertex
No match	Vertex keeps its old value	Function gets None
Property type	Must stay the same	Can change

Computing Averages with aggregateMessages

Average age of each user's followers

```
val ageSums = g.aggregateMessages[(Int, Int)](
  ctx => ctx.sendToDst((1, ctx.srcAttr._2)),
  (a, b) => (a._1 + b._1, a._2 + b._2))

val avgAge = ageSums.mapValues(v => v._2.toDouble / v._1)
```

Output

```
(1,21.5) (2,21.0) (3,22.0) (6,21.0)
```

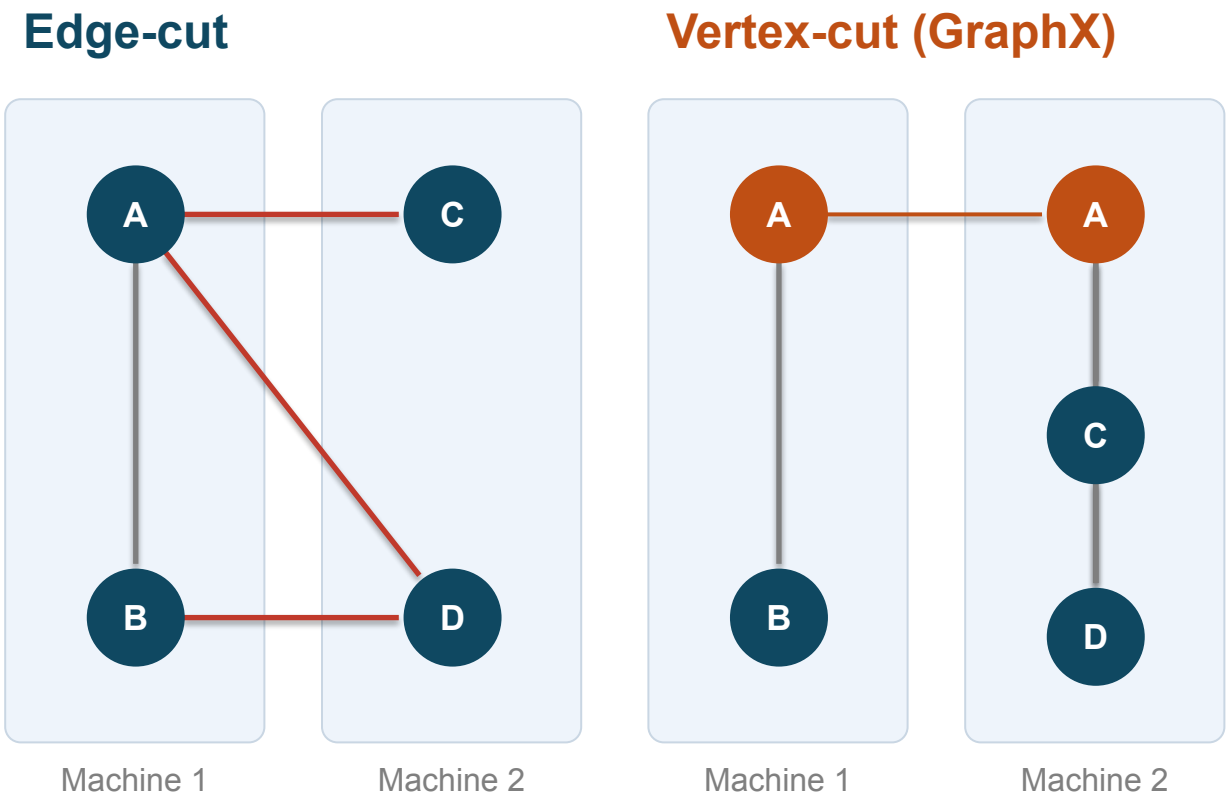
Asha's followers are Chitra (20) and Dev (23): (20 + 23) / 2 = 21.5.

The pattern

Each message is a pair (count, sum). mergeMsg adds pairs. Divide once at the end.

Messages stay small and fixed-size, which is what makes aggregateMessages fast. Avoid sending growing lists.

How GraphX Splits a Graph: Vertex-Cut



Red: edges cut across machines. Orange: vertex A copied to both machines; only A's value needs syncing.

- Real graphs have a few huge vertices (celebrities, popular pages) with millions of edges.
- Edge-cut puts each vertex on one machine. A huge vertex's edges then cross the network all the time.
- Vertex-cut assigns edges to machines and lets a vertex be copied to several machines.
- Choose a strategy with graph.partitionBy(...): RandomVertexCut, EdgePartition1D, EdgePartition2D and others.

Source: Gonzalez et al., PowerGraph (OSDI 2012) and GraphX (OSDI 2014)

How a Graph is Stored Inside GraphX



- To build triplets, GraphX sends each vertex's value only to the edge partitions that need it. The routing table says where.
- Graphs have far more edges than vertices, so moving vertex values to the edges is the cheaper direction.
- If a vertex's value did not change in the last step, it is not sent again.
- Key idea of the GraphX paper: graph computation can be written as joins, maps and group-bys inside a normal dataflow system.

Source: Gonzalez et al., GraphX, OSDI 2014

BIG DATA

Practical Tips

Cache what you reuse

Graphs are not cached by default. Call graph.cache() before running several operations on the same graph.

Use Pregel for loops

It cleans up old iterations from memory. Hand-written loops tend to fill memory with old graphs.

Checkpoint long runs

Set spark.graphx.pregel.checkpointInterval and a checkpoint directory so lineage does not grow forever.

Keep messages small

Send numbers or pairs, not lists. Pass tripletFields to aggregateMessages when you only need one side.

Ids must be Long

String keys (emails, URLs) need mapping to Long first, for example with zipWithUniqueId and a join.

spark-shell line breaks

A line starting with .method or else is read as a new statement. Break lines inside brackets.

Source: Apache Spark GraphX Programming Guide; tested on Spark 4.0.1

BIG DATA

GraphFrames in PySpark

Hands-on: Demo 7

```
from graphframes import GraphFrame

v = spark.createDataFrame([(1, "Asha", 21), (2, "Bharath", 22), (3, "Chitra", 20),
    (4, "Dev", 23), (5, "Esha", 21), (6, "Farhan", 22)], ["id", "name", "age"])
e = spark.createDataFrame([(1, 2, "follows"), (2, 3, "follows"), (3, 1, "follows"),
    (4, 1, "follows"), (5, 6, "follows")], ["src", "dst", "relationship"])
g = GraphFrame(v, e)

g.inDegrees.show()
g.find("(a)-[]->(b); (b)-[]->(c); (c)-[]->(a)").select("a.name", "b.name", "c.name").show()
```

inDegrees

```
id inDegree
 1        2
 2        1
 3        1
 6        1
```

3-cycle pattern (motif)

```
Chitra   Asha     Bharath
Bharath  Chitra   Asha
Asha     Bharath  Chitra
```

The motif query finds the follow cycle Asha → Bharath → Chitra → Asha. It appears three times, once per starting point.

References

Suggested reading for students

1. Raj Kamal and Preeti Saxena, Big Data Analytics: Introduction to Hadoop, Spark, and Machine-Learning, McGraw Hill Education, 2019. Chapter 8 (Spark GraphX).
2. Apache Spark, GraphX Programming Guide. https://spark.apache.org/docs/latest/graphx-programming-guide.html

Further references (optional)

3. J. E. Gonzalez, R. S. Xin, A. Dave, D. Crankshaw, M. J. Franklin and I. Stoica, "GraphX: Graph Processing in a Distributed Dataflow Framework," OSDI 2014.
4. G. Malewicz et al., "Pregel: A System for Large-Scale Graph Processing," ACM SIGMOD 2010.
5. J. E. Gonzalez, Y. Low, H. Gu, D. Bickson and C. Guestrin, "PowerGraph: Distributed Graph-Parallel Computation on Natural Graphs," OSDI 2012.
6. A. Dave et al., "GraphFrames: An Integrated API for Mixing Graph and Relational Queries," GRADES 2016. https://graphframes.io
7. Apache Spark developer mailing list, "[VOTE] Officially Deprecate GraphX in Spark 4," October 2024.



THANK YOU

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu



BIG DATA
Case Study

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

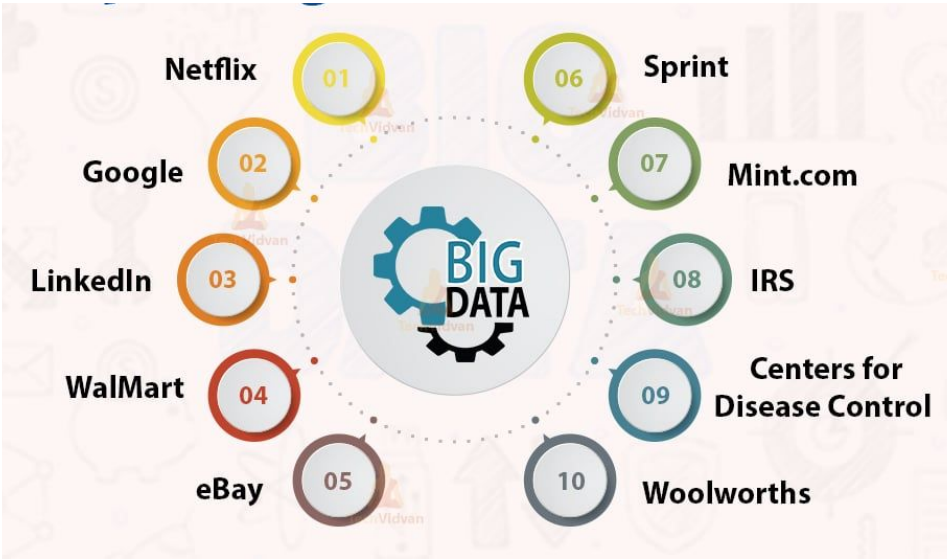
Acknowledgements:

Significant information in the slide deck presented through the Unit 1 of the course have been created by **Dr. K V Subramaniam's** and would like to acknowledge and thank him for the same. There have been some information which I might have leveraged from **Dr. H L Phalachandra's** slide contents too. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

BIG DATA

Introduction

Big Data is a big thing and this case study collection will give you a good overview of how some companies really leverage big data to drive business performance. They range from industry giants like Google, Amazon, Facebook, GE, and Microsoft, to smaller businesses which have put big data at the centre of their business model, like Kaggle and Cornerstone.



Google











BIG DATA

Google

- Google has not only significantly influenced the way we can now analyse big data (think MapReduce, BigQuery, etc.) – but they are probably more responsible than anyone else for making it part of our everyday lives.
- Although these days Google's big data innovation goes well beyond basic search, it's still their core business. They process 3.5 billion requests per day, and each request queries a database of 20 billion web pages.
- Initially it was PageRank which included information about sites that linked to a particular site in the index, to help take a measure of that site's importance in the grand scheme of things. Previously leading search engines worked almost entirely on the principle of matching relevant keywords in the search query to sites containing those words. PageRank revolutionized search by incorporating other elements alongside keyword analysis.
- Their aim has always been to make as much of the world's information available to as many people as possible (and get rich trying, of course...) and the way Google search works has been constantly revised and updated to keep up with this mission.

BIG DATA

Google

- Moving further away from keyword-based search and towards semantic search is the current aim. This involves analysing not just the “objects” (words) in the query, but the connection between them, to determine what it means as accurately as possible.
- In 2010 it launched BigQuery, its commercial service for allowing companies to store and analyse big data sets on its cloud platforms. Companies pay for the storage space and computer time taken in running the queries.
- Another big data project Google is working on is the self-driving car. Using and generating massive amounts of data from sensors, cameras, tracking devices and coupling this with on-board and real-time data analysis from Google Maps, Streetview and other sources allows the Google car to safely drive on the roads without any input from a human driver.
- Google may not quite yet be ready to predict the future – but its position as a main player and innovator in the big data space seems like a safe bet.

Kaggle



BIG DATA

Kaggle

- If you're looking for a company which seems to embody all the principles of big data entrepreneurship under one roof, then look no further than Kaggle. Crowdsourcing, predictive modelling, gamification – Kaggle has it all - and has worked out how to turn a profit from them.
- Chief scientist at Google (which has itself benefitted from Kaggle's research) and Kaggle investor, Hal Varian, describes it as "a way to organize the brainpower of the world's most talented data scientists and make it accessible to organizations of every size."
- As with all emerging sciences, there is a shortage of trained data scientists at the moment – but Kaggle has 150,000 of them, ready to farm out to the highest bidder.
- As well as charging companies they work with (including Amazon,Facebook, Microsoft and Wikipedia) up to $300 per hour for consultancy work, the company organizes competitions – which is where the gamification comes in.

BIG DATA

Walmart



BIG DATA

Walmart

- Walmart is the largest retailer in the world and the world's largest company by revenue, with more than 2 million employees and 20000 stores in 28 countries. It started making use of big data analytics much before the word Big Data came into the picture.
- Walmart uses Data Mining to discover patterns that can be used to provide product recommendations to the user, based on which products were brought together.
- WalMart by applying effective Data Mining has increased its conversion rate of customers. It has been speeding along big data analysis to provide best-in-class e-commerce technologies with a motive to deliver superior customer experience
- The main objective of holding big data at Walmart is to optimize the shopping experience of customers when they are in a Walmart store.
- Big data solutions at Walmart are developed with the intent of redesigning global websites and building innovative applications to customize the shopping experience for customers whilst increasing logistics efficiency.
- Hadoop and NoSQL technologies are used to provide internal customers with access to real-time data collected from different sources and centralized for effective use.

BIG DATA

Netflix



BIG DATA

Netflix

- It is the most loved American entertainment company specializing in online on-demand streaming video for its customers.
- Netflix has been determined to be able to predict what exactly its customers will enjoy watching with Big Data. As such, Big Data analytics is the fuel that fires the ‘recommendation engine’ designed to serve this purpose. More recently, Netflix started positioning itself as a content creator, not just a distribution method.
- Unsurprisingly, this strategy has been firmly driven by data. Netflix’s recommendation engines and new content decisions are fed by data points such as what titles customers watch, how often playback stopped, ratings are given, etc. The company’s data structure includes Hadoop, Hive and Pig with much other traditional business intelligence.
- Netflix shows us that knowing exactly what customers want is easy to understand if the companies just don’t go with the assumptions and make decisions based on Big Data.

Uber

Uber



BIG DATA

Uber

- Uber is the first choice for people around the world when they think of moving people and making deliveries. It uses the personal data of the user to closely monitor which features of the service are mostly used, to analyze usage patterns and to determine where the services should be more focused.
- Uber focuses on the supply and demand of the services due to which the prices of the services provided changes. Therefore one of Uber's biggest uses of data is surge pricing. For instance, if you are running late for an appointment and you book a cab in a crowded place then you must be ready to pay twice the amount.
- For example, On New Year's Eve, the price for driving for one mile can go from 200 to 1000. In the short term, surge pricing affects the rate of demand, while long term use could be the key to retaining or losing customers. Machine learning algorithms are considered to determine where the demand is strong.

BIG DATA

eBay



BIG DATA

eBay

- A big technical challenge for eBay as a data-intensive business to exploit a system that can rapidly analyze and act on data as it arrives (streaming data). There are many rapidly evolving methods to support streaming data analysis.
- eBay is working with several tools including Apache Spark, Storm, Kafka. It allows the company's data analysts to search for information tags that have been associated with the data (metadata) and make it consumable to as many people as possible with the right level of security and permissions (data governance).
- The company has been at the forefront of using big data solutions and actively contributes its knowledge back to the open-source community.

BIG DATA

Sources

https://data-flair.training/blogs/big-data-case-studies/



THANK YOU

Prafullata Kiran Auradkar
Department of CSE
prafullatak@pes.edu



BIG DATA

Streaming analytics use cases

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Significant information in the slide deck presented through the Unit 1 of the course have been created by **Dr. K V Subramaniam's** and would like to acknowledge and thank him for the same. There have been some information which I might have leveraged from **Dr. H L Phalachandra's** slide contents too. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

Introduction

Streaming analytics are used to gain real-time insights from continuous data streams, enabling faster decisions and actions across various industries. Common use cases include **fraud detection, real-time customer behavior analysis for personalized marketing and support, supply chain optimization, IoT device monitoring, cybersecurity, and predictive maintenance in manufacturing.**

BIG DATA

Log Analysis



BIG DATA

Log Analysis

Log analysis is one of the most common use cases of IT operations, and it can be applied to a wide range of real-time log analyses to gain insights. This tool is designed to allow for analysis of depth logs with visualization and includes out-of-the-box filters to help you save time.

The technical approach is based on distributed processing of log streams in real-time. Stream processing is used to query continuous data streams and process received data. It can generate analyses of streams and transactions, then take data from existing streams and create new streams for additional use cases.

With log analysis, we can analyze the network logs to detect anomalies and incidents before it reaches the user. Also, log analysis of routers can give you a good picture of the activity on your network such as monitoring the internet activity of your applications and router attack detection. So when something goes wrong it will appear on logs and alerts you as they occur.

Log analysis tools are crucial for effective monitoring because they allow us to extract meaningful data from log files. Popular log analytics tools such as Splunk and Elasticsearch can be used with Upsolver to provide the streaming and enrichment capabilities for real-time log analytics.

BIG DATA

Fraud Detection



BIG DATA

Fraud Detection

A stream processor must help developers write applications that respond to incoming data. The combination of these powerful technologies ensures that fraud can be detected and prevented.

Streaming transaction data can detect anomalies that signal fraud in real-time, and stop fraudulent transactions, or identify and stop them before they are even completed. Fraudulent transactions can also be stopped as they occur by inspecting, correlating, and analyzing the data which can occur in many industries.

Using machine-learning algorithms and by analyzing transactions in real-time we can Identify patterns to detect fraudulent transactions. It uses complex algorithms that iterate over large data sets and analyze the patterns in data. One of the algorithms is a binary classification which can detect if a transaction is fraud or not.

You can also identify fraud and send notifications by email, SMS, social media, and other forms of communication.

BIG DATA

Cyber Security



BIG DATA

Cyber Security

Cyber attacks will always be a concern in all computing technologies, but there will always be ways to combat them. Strong cybersecurity systems rely not only on cyber-defense technology but also on people making smart cyber-defense decisions.

At the same time, the use of streaming data to detect anomalies in a data stream allows you to identify security issues in real-time to isolate threats. The advantages of stream processing include the ability to filter data from storage and the integration of data into a more robust analytics platform.

Using stream processing in real-time for traffic comes in, We can Identify a DDoS attack by analyzing traffic to see if a suspicious amount of traffic comes from a single IP address or from huge traffic that comes from a user who has a single profile.

BIG DATA

Sensor Data



BIG DATA

Sensor Data

Another use case Is real-time processing data coming from sensors and devices. For example, it can be useful for aircrafts in gaining insights on faults as they occur before coming to a major problem which also results in fewer maintenance delays and improves flight safety.

Also, stream processing of information in the oil and gas industry can help in monitoring various processes in petroleum production and refining and different modalities including temperature, pressure, and pressure in order to guarantee the integrity of oil and gas production.

Data in modern IoT applications are usually real-time data in a stream, the calculations used in streams must be performed in real-time to reduce processing latency and provide accurate inventory and supply chain management. Near real-time analyses are possible with Upsolver and powerful databases and analytics engines.

BIG DATA

Online Advertising



BIG DATA

Online Advertising

Using stream processing on streaming data is very useful in the online advertising industry, It is used in social networks that tracks the user behaviour, user clicks and interests and based on this collected data for each user. It promotes ads which the users might be interested in. So, stream processing helps in advertising campaigns by processing the user clicks and interest in real time and showing sponsored contents.

BIG DATA

Database Migration



BIG DATA

Database Migration

A streaming architecture is recommended for the modern cloud data stacks. It is a long and difficult process for organizations to move away from the traditional on-prem databases architecture to the cloud. A streaming tool like Upsolver allows data teams to migrate data from an on-prem data solution to a cloud-native environment by providing schema-on-read, built-in transformation functions, automatic partition and compaction capabilities. It provides native CDC support to allow the latest view of the data in near real-time. If the users want to move from a traditional database to a cloud data lake architecture, it’s essential to minimize complexity of data processing and maintenance requirements.

BIG DATA

Sources

https://www.upsolver.com/blog/6-most-common-streaming-data-use-cases



THANK YOU

Prafullata Kiran Auradkar
Department of CSE
prafullatak@pes.edu